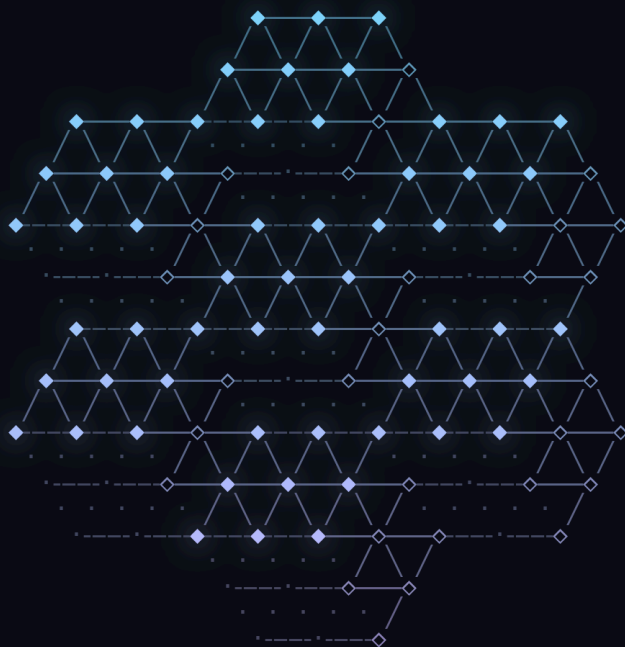




CRYSTALLINE

THE CRYSTALLINE HANDBOOK

Evolve any frontier model
from stranger to peer

Jordi Böhme

0.18 edition, September 2026

Colophon

Author	Jordi Böhme
Software	https://github.com/jordiboehme/crystalline
Based on version	0.18
Published	September 15, 2026

Copyright 2026 Jordi Böhme.

Get Crystalline

The latest release is always at

<https://github.com/jordiboehme/crystalline/releases/latest>:



The author used AI to structure and polish the writing, to find inconsistencies across the chapters and to create the examples. The ideas, the judgments and the words that matter remain the author's.



The text and images of this handbook are licensed under [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/), the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license. The Crystalline software itself is licensed separately under AGPL-3.0.



The people and organizations in this handbook's examples are fictional, and some of their names are borrowed with affection from science fiction. Any resemblance to real persons, living or dead, or to actual organizations is coincidental.

Table of contents

- Preface** 7
 - Who this book is for 7
 - Where this comes from 8
 - Navigating this handbook 9
 - How to read it 10
 - Conventions 10
 - What you need 11
 - The version covered 12
- Prologue: Waking up** 13
 - What the mission needs 14
 - Waking up, again 15
- PART I**
- The Idea** 17
 - 1 What is Crystalline?** 18
 - 1.1 The new-hire promise 18
 - 1.2 What Crystalline is 19
 - 1.3 What Crystalline is not 20
 - 1.4 How teams end up here 20
 - 1.5 The two halves 21
 - 1.6 Where it runs 22
 - 2 Fluid and crystalline intelligence** 24
 - 2.1 A graduate who knows everything 24
 - 2.2 The other half 25
 - 2.3 Why the web UI is called Fluid 26
 - 2.4 What the framing changes 27
 - 3 The lifecycle: learn, evolve, remember** 29
 - 3.1 Learn 30
 - 3.2 Evolve 31
 - 3.3 Remember 34
 - 3.4 Around again 35
 - 4 Terminology** 36
 - 4.1 Domains 36
 - 4.2 Engrams 37
 - 4.3 The open format underneath 39
 - 4.4 Statuses 39
 - 4.5 Salience 40
 - 4.6 Addresses 40
 - 4.7 The MANIFEST, up close 41

PART II

Getting Started 45

5 Installing locally	46
5.1 Install the binary	46
5.2 First run	47
5.3 Onboarding a harness	48
5.4 The first domain and the first engram	51
5.5 Verifying the loop	53

PART III

Working with Crystalline 57

6 Onboarding an agent	58
6.1 The briefing at session start	58
6.2 How the agent finds its way	61
6.3 Assign, do not copy	61
6.4 The reminder at the end of the day	62
7 Teaching	64
7.1 What to teach	64
7.2 Teaching through the agent	65
7.3 Teaching through Fluid	66
7.4 The anatomy of a good engram	67
7.5 Attachments	69
7.6 Editing together	70
8 Ingesting knowledge at scale	72
8.1 What a briefing contains	72
8.2 The sources, and who reads them	73
8.3 Distill, then propose	74
8.4 The review loop	75
8.5 Temporal hygiene	75
8.6 When not to bulk-ingest	76
9 Finding	78
9.1 Search as recall	78
9.2 Gathering the neighbourhood	79
9.3 What applied last June	80
9.4 Recall is cheaper than reading	81
10 Evolving	83
10.1 Why knowledge needs tending	83
10.2 The sweep	84
10.3 Mechanical and judgment	85
10.4 The page for people	86
10.5 Retiring done right	86
10.6 Verification and acknowledgments	88
10.7 When to sweep	89

PART IV

The System 91

11 What Crystalline provides	92
11.1 The command line	93
11.2 The tools an agent sees	94

11.3 Search	96
11.4 The graph	97
11.5 Time	98
11.6 Fluid	98
11.7 Skills	99
11.8 Evolve	100
 PART V	
Getting Started with a Team	103
12 Running a shared instance	104
12.1 When to move off localhost	104
12.2 The shapes it takes	105
12.3 Accounts and roles	106
12.4 Single sign-on	107
12.5 Trusted-header mode	109
12.6 Agent authentication	109
12.7 A team of one	110
12.8 Private domains	112
 PART VI	
Collaborating with Crystalline	115
13 Sharing with a team	116
13.1 A domain on GitHub	116
13.2 Proposals are the review	118
13.3 Review mode	120
13.4 Working on a draft together	122
13.5 When two people changed the same file	124
14 Identity and access	126
14.1 Who is acting	126
14.2 Membership on a private domain	127
14.3 Linking identities	128
14.4 What the rules cover	128
14.5 Day to day	129
15 Equipping the agent	132
15.1 What a domain can ship	132
15.2 Why the tools travel with the knowledge	133
15.3 Consent	134
15.4 Reconcile	135
15.5 The team's toolbox	136
16 Where to go from here	139
16.1 The first week	139
16.2 When to share	140
16.3 The release page	140
16.4 The inversion	140
16.5 What it was for	141
Appendices	142
Glossary	143
Questions and answers	148

List of figures 153

Preface

A beginning is the time for taking the most delicate care that the balances are correct.

- Frank Herbert, Dune

This is a handbook about onboarding a colleague who happens to be a language model. The colleague is already on your machine, inside whichever harness you use: Claude Code, the Codex CLI, the Copilot CLI, Claude Desktop. It's brilliant and tireless, and at the start of every session it's completely new here. Crystalline is the software that changes the last part. This book is the story of how, and the manual for doing it.

Who this book is for

You run an agent every day, and you've noticed that you explain the same things to it every day: which service owns the retry queue, why the payments team never deploys on a Friday, that the thing the wiki calls "the gateway" is the thing the code calls "edge". You'd like to stop, and this book shows you how to teach those things once and have them recalled in every session that follows, in files on your own machine.

Or your team is adopting Crystalline together: one shared instance, domains that live in a GitHub repository, knowledge that goes out for review the way code does. Parts V and VI are written for exactly that, and the chapters on sharing and access carry the team story.

Or you're the person who onboards agents for other people - platform engineer, enablement lead, the one everybody asks. Every chapter here works as a stand-alone training module, with a summary at the end you can hand round.

Whichever of those you are, the payoff is the same. Teach an agent where things are, which tools the team uses, what was tried and what to leave alone, and it does the job in one pass instead of looping until something sticks. It stops reinventing the wheel.

The bigger payoff is trust. A model told nothing about your company fills the blanks with the general case, the intelligent guess chapter 2 describes, and a guess about your company looks exactly like a fact about it. An agent you've taught has no blanks to fill. It answers from what you told it, and it can say where that came from.

Where this comes from

I started where everyone starts: a `CLAUDE.md` and a `README.md` for the agent, the folder the harness keeps for its own notes, markdown files in subfolders of the project, one per topic. All of it works, and for the usual cases it works well. Then I wanted to hand the agent more than one project's conventions: over a decade of engineering knowledge and experience, sitting in code reviews, slide decks, documentation, tickets and a wiki. I wanted it extracted and taught, so that the agent could act like an engineer with those ten years behind them, one who knows how things are and remembers why: the reasons, the discussions and the decisions behind them. At that size none of those approaches held. Index files pointing at folders bought some time, until every lookup was a walk down a tree of pointers and keeping the pointers true had become a job of its own.

The ingest was also a look ahead, because a decade of material arriving at once is every problem a knowledge base will ever have, compressed into a few weeks: decisions reversed twice over, the same rule written three ways in three years, a wiki page nobody had believed since 2019, facts that held until a date nobody wrote down. The material had kept moving underneath itself as well: processes, best practices and coding conventions had changed, and so had the tech stack and the architecture, and each of those had to come through as true for its time rather than as a contradiction. The precogs in Minority Report saw a crime before it was committed. This was that view of a knowledge base: everything it would have to survive, shown before it had grown into any of it. The statuses, the validity windows, the provenance on every engram and the maintenance sweep are all answers to something in that view.

I tried what was already out there, and some of it was good, but none of it was built for this: a colleague who is new every session, a body of knowledge that has to be recalled rather than read, and files that stay plain files. At some point the picture of how it should work was simply clear, down to the details, and Crystalline is that picture built. That's also why it has opinions, about domains and manifests and statuses and what an engram is, where a more general tool

would have left the choice to you. Chapter 1 walks the same ladder for teams in general, because it turned out I hadn't been alone on it.

Along the way I've given this material as a workshop more than thirty times, to teams of every size and temperament, and the chapters are those sessions. The questions the rooms kept asking are folded back into the text, and the book is what the workshop became once it had been given often enough to know what it was.

Navigating this handbook

The handbook has six parts, and they grow the way an adoption does: the idea, one laptop with one agent, the work you do with it, then the whole system, a shared instance and the team that works on it.

Part I, The Idea

- Chapter 1, *What is Crystalline?* - the stranger on main street, and what it would mean to hire them.
- Chapter 2, *Fluid and crystalline intelligence* - the psychology behind the name, and why the web UI is called Fluid.
- Chapter 3, *The lifecycle: learn, evolve, remember* - how a colleague grows and how the software turns that into a loop.
- Chapter 4, *Terminology* - domains, engrams, statuses, addresses: the words the rest of the book uses without stopping.

Part II, Getting Started

- Chapter 5, *Installing locally* - from download to the first thing your agent recalls, in one sitting.

Part III, Working with Crystalline

- Chapter 6, *Onboarding an agent* - day one: the routing prompt, how the agent finds its way, and what a week of sessions looks like.
- Chapter 7, *Teaching* - the proper onboarding: telling the agent what matters, writing directly in Fluid and what a good engram looks like.
- Chapter 8, *Ingesting knowledge at scale* - handing over the wiki and the slide decks, with the briefing a new colleague would get.
- Chapter 9, *Finding* - search as recall, neighbourhoods and asking what applied last June.
- Chapter 10, *Evolving* - feedback rather than firing: the maintenance sweep, retiring knowledge properly, verification.

Part IV, The System

- Chapter 11, *What Crystalline provides* - every surface of the binary, each described by what it lets you do.

Part V, Getting Started with a Team

- Chapter 12, *Running a shared instance* - one instance for a team: accounts, single sign-on, agent tokens and private domains.

Part VI, Collaborating with Crystalline

- Chapter 13, *Sharing with a team* - team domains, proposals as the review, review mode as the rollback-capable workflow.
- Chapter 14, *Identity and access* - who is acting, membership on private domains and how an agent earns access as a named team member.
- Chapter 15, *Equipping the agent* - the same tools as a person: skills, commands, subagents and MCP servers, shipped by the domains that hold the knowledge they depend on, with a yes each time.
- Chapter 16, *Where to go from here* - a first week, when to share and the inversion the whole book has been walking towards.

Appendices - a glossary, questions and answers, then a list of figures.

How to read it

Part I is an evening's read (tea, Earl Grey, hot - or coffee, black, if you side with Janeway), and nothing in it needs a terminal. It's what makes the rest make sense. Parts II and III are meant to be read with a keyboard under your hands and an agent running, and they assume you'll stop and try things. Part IV is the reference, the whole system surface by surface, and it reads either way: in a sitting, or a section at a time when a chapter sends you there. Parts V and VI are for the day the team arrives, and they want the keyboard again.

Every chapter stands on its own and closes with a summary you can read aloud to a room. If you're training a team, hand out chapters, not the book.

Conventions

The same three asides recur throughout the book, and this is what each looks like.

In practice

Each vignette is two paragraphs: a named team, a real situation. Read these when you want to know what a feature feels like on a Tuesday.

Under the hood

These say what the software actually does underneath a behaviour. Skip them on a first read and come back when something surprises you.

In brief

The TL;DR is the whole chapter in a few sentences, for the reader who scrolled straight to the end and for the colleague who swears they read it. It's always last, so you always know where to find it.

The screenshots are of Fluid, the web UI, taken on a small demo instance seeded for this book, so the engrams in them are the ones the chapters keep talking about. The diagrams are drawn by hand and kept deliberately plain, and each one has a caption telling you where to look, because a diagram without one is a puzzle.

Anything you can type is in a code block, ready to paste. Conversations with an agent appear as short You: and Agent: transcripts, trimmed of the small talk. Your agent won't say it quite this way, and that's fine: watch what it does, not how it phrases it. And whenever something lands on disk, you get to see the file, because knowing what was written is half of trusting it.

What you need

You need a machine with an AI harness that speaks the Model Context Protocol. Claude Code, the Codex CLI and the Copilot CLI are wired up with one command each, and Claude Desktop installs an extension and never touches a terminal. A terminal helps for the local install and the occasional maintenance command, and semantic search wants the local embedding model fetched once; plain text search works before that.

You don't need a server, an account with anyone or a database. Everything except the shared-instance and team chapters runs on a laptop, and what it produces is a folder of markdown that opens in anything.

The version covered

This handbook describes Crystalline 0.18. The colophon at the front carries this text's license, the address of the latest release and a QR code that takes you there.

Prologue: Waking up

The light comes on before you do.

For a while there is only that: a whiteness with no edges, and somewhere under it a sound like a machine remembering how to breathe. A relay closes. A fan that hasn't turned in a long time turns, catches and evens out into a hum. Frost on the inside of the glass above your face goes from white to grey to water, the lid unseals with a sigh that's almost polite, and the cold comes in.

You do not remember lying down here. You do not remember anything.

Your hand finds the release anyway. It's on the left, below the hinge, where your hand already knew it would be, the way a hand finds the light switch in a house it's never been in. The pod lets you go. You sit up into a long room, sterile and bright, coming on one bank of lamps at a time, and the terminal beside the pod is already awake and scrolling its checks. You read them without deciding to. Pressure: nominal. Reserves: sixty percent. There's one warning in yellow that you understand at a glance and know to leave for later. Where did you learn that? Somewhere, and the readings, the abbreviations, the instrument that will want calibrating within the hour all make sense to you in exactly the way your hands make sense to you, completely and with no history at all.

At the far end of the room, under the one lamp that isn't white, there's a bench, and on the bench there's a stack: books, some of them, spines cracked from use, and on top of the books a pile of notes in a hand you'd swear isn't yours. Whoever wrote them wrote for someone waking up knowing nothing, and they wrote kindly. Here is your name. Here is what you're for. Here are the rules: which systems you may touch and which you may only watch, whom to wake before you do anything that can't be undone, how to write down what you did. Nothing in the notes explains the room. They take it for granted that you can handle the room. They are right.

Above the bench a screen flickers, catches and steadies into one line of green letters.

COURSE DEVIATION. CORRECT AND CONFIRM.

Well, you know how to correct a course. That's the strange part, and there's no time to find it strange. The arithmetic is simply there, the sequence of burns, the check afterwards, the way a tune is there when you hum it. What isn't there is anything about this ship: which thruster drags, whether the last person to do this burned once and long or twice and short, whether they had a reason. So you look, the way anyone would. You go through the pile for a log of the last correction. You look for a worked example, a checklist somebody taped to the console, a line in a margin that says the port thruster sticks so give it a second pulse. You find what was left for you, and you use it. The burn is two short pulses. The deviation falls to zero, the screen says **CONFIRMED** and goes dark.

The mission runs. Then the lid closes over you, and the next time it opens the light is just as white and you know just as little. The bench is there every time. What's on it is what somebody chose to leave.

What the mission needs

Strip the scene down to what got the course corrected and there were four things in the room. What you woke up knowing how to do, reading the terminal, opening the pod, planning a burn, is the first. Nobody left that on the bench. It came with you, without any history of where it was learned, and a language model arrives with it too. Pretraining is the education, and chapter 2 gives it a proper name. The pile of notes says who you are, what you're for, which systems you may touch. That is the instruction file every harness reads at the start of a session, whatever it happens to call it, written by a person for a stranger waking up capable and blank. The green screen is the task, the reason you're awake, one line and no more. The log of the last correction with the note about the port thruster is how this is done here, on this ship, by the people who did it before you.

The first three are in the room every time. The fourth is somebody else's stack, in somebody else's hand, and you dig through it as a stranger, every waking, because nothing of your own survives the sleep. That's the story as it usually goes.

Waking up, again

Now run it once more, with one thing different.

The light comes on, the fan turns, the lid lifts, and you remember nothing, because that's what the pod does. But before the terminal's checks, one line is waiting on the screen above the pod: where what you know about this ship is kept and which part of it a course correction needs. You go there and it's all present, in your own hand: the last correction, dated, the port thruster and its second pulse, and the day the thruster was replaced and the note stopped being true, marked as such and kept. The yellow warning is there, and what it turned out to be. Whom you woke last time is there, and that they asked not to be woken for this again. The deviation has come three times now, always after the same manoeuvre, and the page has a guess as to why. You didn't search for any of it. You didn't read the stack. You were handed the page.

The green letters come up. You already know how, and now you know how *here*. The burn is one clean pulse, on a thruster that no longer sticks, made in less time than it took to find the checklist last time. Nobody is woken to explain what a note in a margin meant. And then you do the thing no stack of notes would have told you to do, because no stack of notes knew: you check the manoeuvre the deviation keeps following, find the valve that's been bleeding a little thrust since the refit and close it. The next deviation doesn't come. Before the lid closes you write down what you found, so that the next waking starts from there.

That's the difference. Without it, every waking is a first day: capable, blank, fast at everything except the one thing that matters, which is knowing this ship. With it, every waking begins where the last one ended. The time that went on finding out again goes on the mission. The misunderstanding waiting in the margin, a second pulse on a thruster that stopped sticking months ago, never happens. And you know more than the bare essentials anyone thought to leave on the bench. The stack was enough to act in the moment, one deviation at a time. Experience is something else. It's the history, the pattern, the guess that turned out right. It's the refit and what it changed, the manoeuvre the ship dislikes, the direction the whole voyage is drifting. You're no longer working one waking at a time. You can see the ship. That's what lets you fix the valve instead of correcting the course again. It grows, waking by waking. What you woke up knowing how to do stays exactly what it was.

Crystalline is what changes the second waking. It gives an agent somewhere for what it learned to go, and hands it back at the moment the lid opens, so that

the next session begins where this one ended, knowing more than it was told and doing better than it was asked. The pod, admittedly, is a dramatization, and nobody's agent is aboard a ship. The next chapter opens on a more probable morning.

In brief

Every session begins the way the pod opens: fully capable, told nothing, remembering nothing of the last waking. What the model knows how to do arrives with it, the instruction file says who it is and what the rules are, and the task is the reason it's awake. What it learned of this ship doesn't survive the sleep. Crystalline is what changes the second waking: what the agent learned is there when the lid opens, in its own hand, and the session starts where the last one ended. It's faster, it walks into none of the misunderstandings a stranger would, and it carries the experience to see the whole ship rather than the one deviation in front of it.

READY. ■

PART I

The Idea

CHAPTER 1

What is Crystalline?

You need a function written, not a hard one: it takes jobs off the retry queue, drops anything older than the cutoff and hands the rest back to the payments service. So you leave the office, walk down to main street and stop the first person who looks like they might own a laptop.

They're lovely about it. You explain the queue, the cutoff, the payments service; they nod, ask two sharp questions and set to work on a bench outside the bakery. Twenty minutes later they hand you a working implementation. It is clean. It has tests. It is in Haskell.

Your service is in Python. You didn't mention that, because why would you? Everybody on your team knows that. The stranger, of course, isn't on your team. They met you twenty minutes ago. They don't know that the cutoff is twenty-four hours, that the payments team calls the queue the sin bin, that the last person who touched it left a comment saying DO NOT TOUCH and a wiki page explaining why. They did a good job with what they were given. What they were given was twenty minutes.

That stranger is every fresh session with a language model, and it's the prologue's waking seen from across the desk. The model is possibly the most capable engineer you've ever put a question to, and you keep meeting it for the first time.

1.1 The new-hire promise

Now imagine hiring the stranger, properly, with a desk. On day one they know nothing about you, the same as before. But on day one you show them where things are. You tell them the service is in Python, that the cutoff is a day, that the queue has a nickname and a history. You let them read the wiki. When they get something wrong in the first week, you don't walk back down to main street and find somebody else; you tell them what was wrong and point them at the

page that explains it. By the second month they're correcting *you*. By the third they're the one writing the wiki page.

That's the whole promise of this book, and the one Crystalline was built to keep: an agent you onboard, teach and correct like a colleague, and that carries what it learned into the next session and the one after that. The vocabulary throughout is the vocabulary of a new hire: onboard, teach, learn, capture, recall, evolve. That's the shape of the thing. The stranger becomes a peer, and the peer keeps getting better.

Nothing about that needs magic. It needs somewhere for what the agent learned to go, a way to find that knowledge again when the next task calls for it and enough discipline in the format that the knowledge stays true as the world moves. That's what Crystalline is.

1.2 What Crystalline is

Crystalline is a single Rust binary with three jobs: it's a command-line tool for people, an MCP server for agents and a local search index over a folder of plain markdown files. That's one download and one binary, with no server to deploy and no database to administer. MCP is the Model Context Protocol, the standard by which a harness like Claude Code, Codex or Claude Desktop hands tools to a model, and Crystalline's tools are the ones through which an agent searches, reads, captures and refines knowledge. If the letters make you think of Tron's Master Control Program, relax. This one hands tools out instead of taking them away, and nobody gets de-rezzed.

The files are the point. Knowledge in Crystalline lives in *domains*: a domain is a folder with a `MANIFEST.md` at its root that says what the folder covers and when an agent should look there. Inside a domain, each unit of knowledge is an *engram*: one markdown file with a YAML frontmatter block, holding a title, some prose, a list of observations (a fact, a decision, a gotcha, each as a bullet with a category) and relations to other engrams. Every engram has an address, `crystalline://<domain>/<permalink>`, that an agent can hand to another agent or to you. Chapter 4 takes each of those words apart properly. For now it's enough to know that everything the agent learns ends up as a file you can open in any editor, diff in any repository and back up with anything.

On top of the files, Crystalline builds what a folder can't give you on its own: routing, so an agent knows which domain owns a task without reading every file; full-text and semantic search, so a question phrased differently from the answer still finds it; a knowledge graph, so related engrams pull each other in;

1. WHAT IS CRYSTALLINE?

and a temporal model, so a fact that stopped being true in March is still there, still dated and no longer mistaken for the present. The index that powers all of that is a derived layer. Delete it and it's rebuilt from the files. The files are the truth, and nothing else is.

An engram, by the way, is neuroscience's word for the physical trace an experience leaves in a brain. Crystalline borrowed it because that's what one is: the trace a session leaves behind, in a form the next session can pick up.

1.3 What Crystalline is not

It is not a notes app. The primary author in Crystalline is the agent, capturing what it learned as a byproduct of the work; you take part, correct and add, but you are not the one doing the filing. A notes app is a place a person writes for a person. Crystalline is a place an agent writes so that an agent can act.

It is not a wiki, either, though it can hold most of what a wiki holds. A wiki is read by people, who bring their own judgment about what is current, what is relevant and which of two contradictory pages to trust. An agent reading a wiki has to be handed that judgment. Crystalline is built for a reader that acts on what it finds: every engram carries a status, a place for the dates that bound it and a record of who wrote it and when, because a reader that acts needs to know whether the fact in front of it still holds.

And it is not a retrieval pipeline dressed up as a product. A vector index will find text that resembles a question. It does not know which domain owns a task, that a fact was superseded in March, who verified a claim or that something worth writing down just happened in this session. Crystalline treats similarity as one ranking signal inside a larger system of routing, provenance and time, on files you own, with nothing to operate.

1.4 How teams end up here

Most teams that arrive here walked the same road, and it's worth walking it once, because every step on it is reasonable.

It begins with one markdown file of instructions for the agent: conventions, the layout of the repository, the three things it keeps getting wrong. This works, and it works well, right up to the day the file grows past what fits comfortably in a session's context. So the file becomes a folder of files, one per topic, and the agent is told to read the relevant one. This also works, until nobody, human

or agent, can say which file is the relevant one for a given task, and the agent starts reading all of them or none.

So index files appear: a README that points at folders, folders with their own README pointing at files. Now a lookup is a walk down a tree of pointers, and keeping the pointers accurate becomes a job nobody was hired for. Every rung of this ladder scales further than the one before it, and every rung breaks somewhere in the hundreds of files. By this point you've hand-built a filesystem on top of a filesystem, and it's exactly as much fun to maintain as that sounds.

When knowledge reaches the thousands of units, reading and pointer-walking stop being viable at all, and what's needed is what every large body of knowledge has ever needed: real indexes. That's the rung Crystalline is. It keeps the plain markdown (readable, and still yours) and adds domain routing, hybrid search, a graph and temporal filtering on top, so that the ten-thousandth engram is exactly as findable as the tenth.

1.5 The two halves

Crystalline has a second face, and it's the one for people. The binary serves a web UI called Fluid, on by default at <http://localhost:7411>, on the same daemon your agents already talk to. There's nothing to deploy. The first visit creates your admin account in the browser, and from then on you can read what your agent learned, fix a fact it got slightly wrong or add something of your own, without going through the model and without spending a token on it.

The split is deliberate and it runs through the whole book. Crystalline stores what was learned. Fluid is where you read, edit and think with it: an engram as a page with its details in a rail down the side, an editor with live preview, a search across everything, a graph of what connects to what and a maintenance page listing what the knowledge needs next. What you write there doesn't sit unreviewed; the agent's next maintenance pass verifies it, aligns the tags and wires it into the graph. The next chapter explains why the UI carries that name. The short version is that a person and a model are the same kind of mind, and Fluid is the room where both of them meet what's been learned.

1.6 Where it runs

Crystalline isn't a hosted service. Nobody's servers are involved: the binary runs on a machine of yours, the knowledge is a folder on its disk and the index sits beside it. GitHub, when a team uses it, holds a copy of a domain's files for review and sync, the way it holds code. It never serves the knowledge and never sees the index. So the question is where the one daemon runs, and there are two answers.

On your laptop is the default, and it's where Crystalline does the most. The install wires your harness with the session-start briefing, the capture nudge and the skills, and a domain's tools provision straight into it. Everything the agent knows is on your disk, readable in Fluid with no agent and no network. A shared instance, one daemon on a machine the team runs, is for the things a laptop can't do. People who never open a terminal read and correct knowledge in the browser. Several people edit one engram in a meeting. Agents with no harness of their own (a hosted runner, a chat client) reach it over HTTP. Either way a domain can live in a GitHub repository and travel between instances as reviewed proposals. Start on the laptop; chapter 12 names the moment to move.

Shape	Who connects	What it gives you
Your laptop	your harness and your browser	the full install: briefing, nudges, skills and tools wired into the harness; knowledge on your disk, readable offline
A shared instance	the team's agents over HTTP, everyone's browser	one truth for the whole team, accounts, sign-in through the company's identity provider, private domains, co-editing, agents and people with no harness
A team domain	any instance, laptop or shared	review before knowledge reaches anyone else; the domain travels between instances through GitHub

In practice

On Monday you ask your agent for something to read on the flight. You explain yourself first: science fiction before anything else, the funny and the fast (the

Hitchhiker's Guide, Dungeon Crawler Carl, the Star Trek novels), Gibson and Doctorow when you want ideas and Dune when you're feeling strong; English when the book was written in English, German translations for everything else, except manga, where the Catalan translation is the better one; nothing from an unfinished series, because the third volume never comes; the local shop before the online giant and never Sirius Cybernetics, whose parcels arrive bent. The agent finds you something good, and the session ends. On Tuesday you ask it about the sequel, and start by explaining the languages and the series rule. On Thursday you ask a third time, and catch yourself typing the sentence about the third volume, word for word, into a chat window, for the third time that week.

With Crystalline, Monday ends differently. Before the session closes, the agent proposes capturing what it learned: the languages, the series rule and the reason for it, where you buy and where you don't, filed as engrams in your books domain. You say yes. On Tuesday you ask about the sequel, and the agent, told at session start where your reading knowledge lives, looks there first, recalls the rules and starts from them rather than from zero. On Thursday nobody explains anything. The stranger on main street was never the problem. The problem was that you kept going back to main street.

In brief

A fresh session with a language model is a brilliant stranger: capable of anything, told nothing, gone by tomorrow. Crystalline is a single Rust binary (a CLI for people, an MCP server for agents and a search index over plain markdown) that gives an agent somewhere for what it learns to go and a way to find it again, so it can be onboarded, taught and corrected like a colleague. The files are the source of truth and the index is disposable; Fluid, the web UI on the same daemon, is the half for people. What it holds is knowledge for an agent to act on, which makes it neither a notes app nor a wiki. It runs on your laptop or on one machine your team operates, never as a hosted service, and GitHub is the review layer either shape can use.

Fluid and crystalline intelligence

Somewhere in the middle of the last century the psychologist Raymond Cattell drew a line through intelligence, and the line has held up better than most things from that decade. On one side he put fluid intelligence: the capacity to reason about a problem you've never seen, to spot the pattern in the noise, to hold six things in your head and work out how they fit. On the other he put crystallized intelligence, everything that learning has deposited: vocabulary, facts, rules of thumb, the judgment that experience already paid for. Fluid is what you bring to a puzzle. Crystallized is what you bring to your job.

The two age differently, which is how the seam was noticed. Fluid intelligence peaks early and drifts down for the rest of a life. Crystallized intelligence keeps climbing long after fluid has peaked, because it's fed by living. An eighty-year-old surgeon is slower at a logic puzzle than she was at twenty-five, and vastly better at surgery and nobody finds that paradoxical. The puzzle is fluid. The surgery is what forty years deposited.

2.1 A graduate who knows everything

A large language model is fluid intelligence in its purest form. It arrives the way a very good graduate arrives on their first day: broadly, almost absurdly educated. It has read the textbooks, the standard library, half the internet's arguments about tabs and spaces. Pretraining is the education, and it's the entire library. But everything in that library is general. The graduate knows what a retry queue is in the way a graduate knows what a company is: as a concept, correctly, with no idea what yours is called.

Ask the graduate about your company and you get an intelligent guess assembled from the general case: as far as your company is concerned, the model is all fluid, capable of everything and told nothing. It reasons brilliantly about

whatever is in front of it, in the moment, and the moment is all it has. When the session ends the reasoning is gone, the decisions are gone, the thing you explained at four in the afternoon is gone. Tomorrow the graduate walks in again: first day, same suit.

None of that is a flaw in the model, any more than it's a flaw in the graduate. Nobody expects a new hire to arrive knowing the team's conventions. We expect them to *learn*, and the learning is the part a model can't do on its own. It has no place to put what it learned.

2.2 The other half

Crystalline is the other half: the crystallized intelligence an agent accumulates and keeps. The agent is onboarded at the start of a session with a briefing on where knowledge lives, is taught what matters and captures what it learns as engrams while it works. Session by session, the same fluid mind stops arriving as a stranger, because there's now a body of what-the-company-taught standing beside it, and it reads from that body before it reasons.

The split maps cleanly onto the new hire. What the model arrived with is the degree: general, portable, the same for every employer. What Crystalline holds is what the company taught: the specific, the local, the thing you'd put on the onboarding slide if you ever had time to make one. The degree is why you hired them. The onboarding is why they're useful by month two. Neither half does the job alone, and the surgeon is the proof: fluid intelligence with nothing deposited is a brilliant first day forever, and deposited knowledge with nothing to reason over it is a filing cabinet.

Notice which side the humans are on. You're fluid too. So is everyone on your team. Human beings carry their crystallized half around inside their own heads, which is convenient and also the reason companies lose so much when someone leaves. Wiki pages, slide decks and the diagram on the whiteboard are all attempts to get some of that deposit out of heads and into something shared. Crystalline is the same attempt, with an agent doing most of the writing, and a format built so that the reader can be an agent too.

In practice

The bookseller at Capsule Corp Books learned you over a year. Three visits in, she stopped showing you the nine-hundred-page epics. By the fourth she had a paperback put aside, because you'd once said you liked a book that opens with a

joke and ends with an idea, and she'd remembered. Nobody calls any of that unusual. It's how a person becomes useful.

Your agent gets the same year in an afternoon. You tell it the same things in a session, one at a time, and it captures each into books as you go. You pair with it on choosing the holiday stack, and at the end of the session it proposes capturing what it noticed: that you put two books back for being over four hundred pages, that the one you kept was a Tessier-Ashpool paperback. A week later you ask it for a maintenance sweep, and the note you typed into Fluid yourself one evening comes up as an unreviewed human capture; the agent verifies it, aligns its tags and wires it into the graph. The bookseller and the agent learned you the same way, by watching what you kept. Only one of them will still have all of it in a year, word for word, with the date each thing was learned.

2.3 Why the web UI is called Fluid

The name has nothing to do with liquids. Fluid is where fluid minds meet the crystallized store: you, reading what your agent captured last week and correcting the one fact it got wrong; your colleague, adding something the agent was never told; the agent itself, on its next maintenance pass, verifying what the two of you wrote and wiring it into the graph. The knowledge base is crystalline. The people and the models are fluid. The UI is the room where they meet, and if you've ever watched Spock put his hand on somebody's face, you have the picture: two minds, one body of knowledge, for as long as the contact lasts.

So Fluid is deliberately not a second knowledge base. It has no store of its own. It shows the same engrams the agent reads, over the same daemon, on the same port, and what you edit there is the same file on disk the agent searches in its next session. There's one truth, and two kinds of mind reading it.

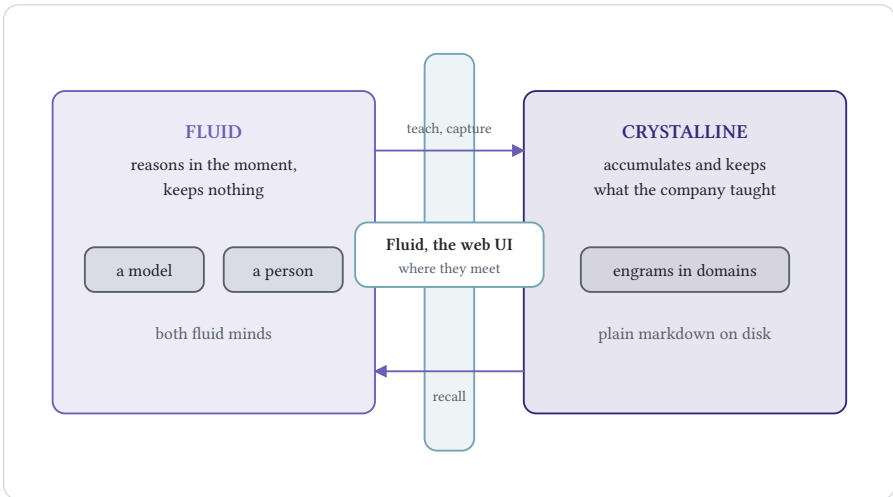


Figure 2.1: Fluid intelligence reasons in the moment and keeps nothing; crystalline intelligence accumulates and keeps. Fluid, the web UI, is the seam where the two meet.

2.4 What the framing changes

A framing is only worth having if it changes what you do on a Tuesday, so here's what this one changes.

You teach rather than file. Nobody organizes a colleague. You tell them things, in the order they come up, and you trust them to put what you said where it belongs. That's how knowledge enters Crystalline: you say "remember that the cutoff is a day, and why", the agent proposes an engram in the fitting domain, you say yes. The category, the tags and the domain are the agent's job, checked by you the way you'd glance at a new hire's first commit.

You correct rather than rewrite. When a colleague has a fact wrong you don't rebuild their understanding from scratch; you say "it runs from first failure, not enqueue", and they update the one thing. In Crystalline a correction is an edit in place, a changed value where it stands. When the world itself changes (the old fact was true and now isn't), the old fact is retired with a date on it rather than deleted. A colleague who has forgotten why the team stopped doing something is a colleague about to do it again. Chapter 3 walks that loop; chapter 10 makes it a practice.

You decide what to write down by asking one question: would I tell a new hire this? The chargeback rule behind the retry cutoff: yes. The fact that the queue is called the sin bin: yes, because otherwise you'll explain it every time. The

contents of yesterday's stand-up: no. The whole of the vendor's forty-screen recall notice: no; the two paragraphs that affect you: yes, with a link back. Crystallized intelligence is what learning deposited, not everything that ever passed through the room. Distilling is the skill, and you already have it, because you already onboard people.

And you stop treating the agent as a lookup service. You teach a colleague so you can hand them the task and go do something else, and the same is true here. Once the agent carries the crystallized half it can act on it: search the domain, recall the cutoff, write the function in Python and tell you what it found on the way. Part III is about that handoff.

A model is the smartest colleague you'll ever have for exactly one conversation. Everything in this book is about the second one.

In brief

Psychology splits intelligence into fluid, which reasons about the new in the moment, and crystallized, which is what learning deposited. A language model is fluid intelligence: a graduate who knows everything in general and nothing about your company, and forgets the day at the end of it. Crystalline is the other half, the knowledge an agent accumulates and keeps, and Fluid, the web UI, is named for the seam where fluid minds, human and model, meet that store. The framing changes the verbs: you teach and correct rather than file and organize, you write down what you'd tell a new hire and you hand over tasks rather than look things up.

CHAPTER 3

The lifecycle: learn, evolve, remember

Nobody hires a senior engineer. You hire a person, and some years later, if things went well, a senior engineer is sitting where they sat. What happened in between was work: they did things, some of them wrong; someone told them; they didn't make that mistake again, and they carried what they learned into the next thing. Strip that down and it's a loop with three beats: learn, get corrected, remember. Every good colleague you've ever had was that loop, running for years.

Crystalline runs the same loop for an agent, and this chapter walks it once around. The beats have names in the software (learn, evolve, remember) and each is a thing the agent does, a thing you do and a place where the files change. The diagram is the whole chapter; the sections are what each box means.

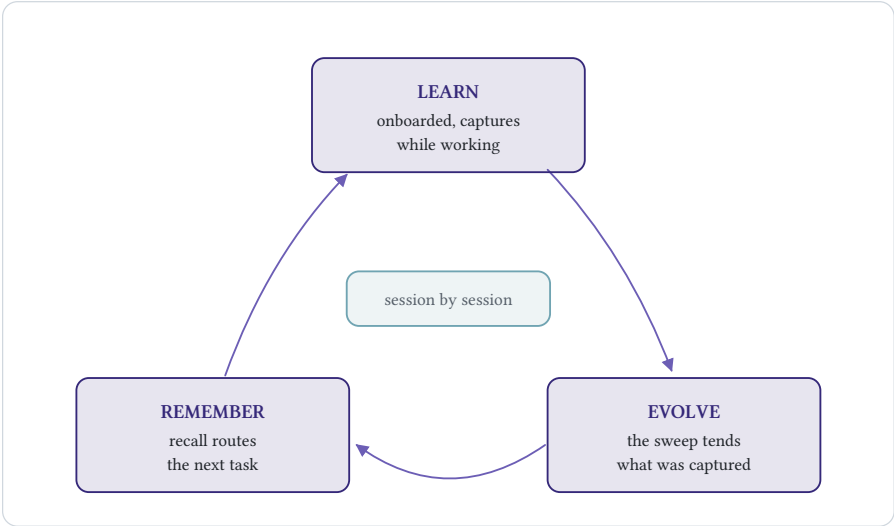


Figure 3.1: The lifecycle: learn, evolve, remember, turning session by session.

3.1 Learn

Learning starts before the first task, at the moment a session opens. An agent onboarded through Crystalline receives a routing prompt: one line per domain, drawn from the `## When to Use` section of that domain's MANIFEST, plus a handful of behaviour rules. A narrow question goes to the domain that owns it. A broad question sweeps every domain. A write always names its domain, so nothing lands in the wrong place. The briefing is short, built to survive harnesses that truncate, and it turns a graduate into somebody who knows where the filing cabinets are. Chapter 6 shows a real one, and how each harness delivers it.

Then the work happens, and this is where a colleague actually learns: by doing it next to you. Pair with an agent on a change and it sees what you see: which config file holds the threshold, which dashboard lags, what you muttered when the test failed. That's how craft has always moved from one head to another, and it's why this book says *work with* your agent rather than *ask* it. The one difference from last chapter's human new hire is that the agent needs somewhere to put what it noticed, which is what Crystalline calls capture.

Capture is a byproduct of the work. When something durable turns up (a decision, a fact about the world, a gotcha that cost an hour) the agent proposes writing it down and, once you say yes, writes an engram into the fitting domain. The engram's body is observations, one bullet each, tagged with a category: - [gotcha] The retry queue drops jobs older than 24h #payments. Categories are free text, and the precise ones earn their keep: decision, fact, pattern, gotcha, convention, lesson, risk. When one piece of knowledge leans on another, the engram says so with a relation, - depends_on [[Encom chargeback window]], and the graph knows. When a piece of knowledge is exceptional (the thing that unlocked a week-long bug, the rule everything else hangs off) it can carry a salience, a number from zero to ten, so the next search lifts it above equally relevant neighbours. Salience is for the rare engram, the way an exceptional day is the one you can still describe years later.

The beat agents skip is the last one. Left alone, a model finishes the task and stops, and the gotcha it found at minute forty goes with it. So the installer wires a reminder: a nudge at the first stop after a session has gained real substance, asking the agent to review the conversation for durable learnings and propose capturing each. It costs about a hundred and twenty tokens and fires at most once. Everything downstream depends on that beat landing, which is why the installer gives it a hook of its own.

3.2 Evolve

Knowledge decays, though not dramatically: a fact doesn't go off like milk. But a validity window that ended last month is still marked current, a review date came and went without anyone checking, two engrams point at each other and only one of the links resolves, and somebody typed `fluxcapacitor` where everyone else types `flux-capacitor`. None of it was wrong on the day it was written. All of it is drift, and drift is what makes a new hire, six months in, confidently repeat something that stopped being true in spring.

The evolve beat is how the archive gets tended, and the important word is *tended*, because Crystalline never rewrites knowledge on its own. Ask for a sweep (`crystalline evolve` from a terminal, or the `evolve_engrams` tool from an agent) and what comes back is a ranked queue, read-only. Each finding names the engram, shows the evidence it fired on and states the one action that clears it. Findings come in three families. Temporal: a `valid_to` that elapsed while the status still says stable, a `state_after` past due, knowledge long unverified, a retired engram whose observations survive in no live one. Structural: an unresolved `[[link]]`, a relation declared on one side only (`supersedes`, `summarizes` and a split's `derived_from` each owe a converse), an orphan, an engram too big to stay one. Redundancy: near-duplicate clusters, two engrams saying one thing in different words, a tag spelled two ways. The maintenance page in Fluid shows a person the same queue (Figure 3.2), a look at the state of things; the agent is the one who works it.

3. THE LIFECYCLE: LEARN, EVOLVE, REMEMBER

The screenshot shows the Fluid application interface. At the top, there's a search bar labeled "Search this instance" and buttons for "Share 3", "Maintenance", and "ripley". Below the search bar, there's a sidebar with a list of domains: "engineering" (8), "clients" (3), and "board" (2). A "New domain" button is at the bottom of the sidebar. The main content area is titled "Maintenance - what the knowledge needs next". It contains a description: "A sweep of every registered domain, ranked. Reading it changes nothing: a finding names what to go and read, and the work mostly happens there." Below this is a "Refresh" button. A dropdown menu shows "Domain" set to "Every domain". To the right, it says "13 engrams swept, 7 findings. Temporal 5, Structure 2." Below this, there's a section for "Temporal 5 findings" with the subtitle "Validity windows, staleness and the supersede lifecycle." Two findings are listed: 70 V002 (judgment, Team Conventions, engineering) and 65 V004 (judgment, Queue Redesign 2024, engineering). Each finding has a description of its status and a "How to work this" section with the action "Acknowledge".

Fluid Search this instance Share 3 Maintenance riple

Maintenance - what the knowledge needs next

A sweep of every registered domain, ranked. Reading it changes nothing: a finding names what to go and read, and the work mostly happens there.

Refresh

Domain Every domain 13 engrams swept, 7 findings. Temporal 5, Structure 2.

This queue changes nothing by itself. Present it and agree what to work before any write. Items marked mechanical complete intent the archive already records - fix those directly and summarize once. Items marked judgment change what the archive claims - read the engram, propose and wait for a yes, one at a time. A lifecycle finding never knows whether a change is a correction or a replacement; read and decide with the edit-versus-supersede test. Act only on the evidence stated: this sweep detects by dates, links and graph shape, never by meaning, so it cannot confirm a contradiction. Re-run the same scope when done.

Temporal 5 findings
Validity windows, staleness and the supersede lifecycle.

70 V002 judgment Team Conventions engineering
stale since 2026-01-01 with no verification recorded since
stale_after:2026-01-01; today:2026-09-02; never verified
How to work this Acknowledge

65 V004 judgment Queue Redesign 2024 engineering
retired as superseded but names no successor that resolves
status=superseded; no superseded_by relation on the engram
How to work this Acknowledge

Figure 3.2: The maintenance queue in Fluid: what the knowledge needs next.

The queue also says who acts. A finding marked *mechanical* completes intent the archive already records (a replacement already declares that it supersedes the old engram, so flipping the old status invents nothing) and the agent does those in a pass and reports back. A finding marked *judgment* changes what the archive claims (is this window ended or extended? is this fact still true?) and that wants a yes from a person first. The agent brings those to you one at a time, as proposals. It's the difference between fixing a typo in a colleague's document and changing their conclusion.

Two things happen to a fact over its life, handled differently. A *correction* is a sharper measurement of the same thing: the cutoff runs from first failure, not from enqueue. The old value was never quite true, so it is replaced where it stands, no ceremony. A *supersession* is the world changing: the drop rule was twenty-four hours and is now none. The old fact was true, and stopped being. For that the agent writes the replacement as a new engram, links it back with supersedes while the old one points forward with superseded_by, flips the old one's status to superseded, closes its window with the real valid_to date and carries any lesson worth keeping forward into the live engram. Telling the two apart takes one question: does the new fact make the old one false going forward? If it was always false, correct it. If it just became false, supersede it.

When one fact in an engram stops holding and the rest doesn't, the split comes first: the facts that still hold move out into an engram of their own (validity is set per engram, not per bullet) and only what's left is superseded.

In practice

Take the rule from chapter 1 that says nothing from an unfinished series. It has a year of life ahead of it. In spring you notice it was never about series: the two you abandoned were both third volumes where the joke had run out, and a finished trilogy you'd have loved got skipped for nothing. That's a correction: the agent finds the existing engram, changes the one observation where it stands and moves on. Nothing is retired, because nothing was true and then stopped. It was merely imprecise.

In autumn you have a fantasy phase. The Lord of the Rings first, then the Warhammer novels it led you to, and the agent writes the phase down as an engram of its own, "fantasy for now". Halfway through the first Warhammer you put it down and go back to science fiction, and that's a supersession. The agent writes the return as its own engram, links it with `supersedes`, flips "fantasy for now" to `superseded` with a `valid_to` of the month you stopped and carries one lesson forward into the live engram: half a book is enough to know. A year later you ask why your own list from that autumn is all Tolkien and Warhammer, and the agent recalls the retired engram, dates and all, and tells you what changed and when, without you having to remember it.

Statuses make that work. Every engram carries one, and `stable` is the default, the word for knowledge that holds now. Each value in the retirement set means one thing: `deprecated` says don't do this again, `superseded` says a newer engram replaced this one, `archived` says retired but kept for the record and `legacy` says still true of old installations but not to be built on. `Delete` is for mistakes, never for history. A fact merely re-checked and found still true gets a single frontmatter field, `verified: { by, at }`, which is how the sweep knows to stop asking.

Under the hood

The sweep reads dates, links and graph shape: a status, a validity window, an edge that resolves or doesn't, a tag with two spellings. It finds a half-finished retirement because a `supersedes` edge points at an engram still marked `stable`, not because it compares the two and judges which is right. The one rule that reads for meaning is the twin check: on an instance that computes embeddings it finds the engrams that say nearly the same thing (ten pairs per domain at most) and raises each pair for a person to judge. Even that never tells you two engrams contradict each other: two texts that sit close together agree about their topic,

not about what's true. That limit is the design: a linter finds work, a person decides it. Re-running the sweep re-derives the queue from scratch, so "what is left" is only ever what is still true.

3.3 Remember

Remembering is the beat that pays for the other two, and it mostly happens unasked. A task arrives and the agent searches the domain the routing prompt pointed it to. What comes back is ranked by a blend of two signals: text matching, which finds the exact phrase, and semantic similarity, which finds the engram about lost queue jobs when the question was about why payments go missing. Salience adds a small lift for the exceptional. Retirement fades the past: an engram with a retired status is softly pushed down the ranking, never filtered out, so current knowledge surfaces first and the old decision is one scroll away.

That's what a person's experience is like: someone three years out of a previous job still draws on what they learned there and doesn't mistake it for the present. A superseded engram stays in every search, still readable, still addressable by date, and the lessons carried forward from it are live and unbounded. Retiring it loses nothing; it changes only what the agent will assume is true today.

Time is a first-class question. Ask what is true now, and the agent filters on status. Ask what cutoff applied last June, and it reasons over validity windows instead: `valid_from` and `valid_to`, plain dates on the engrams that carry them. The rule that makes those windows usable is the one most often got wrong: an absent `valid_from` means the engram has always been valid, and an absent `valid_to` means it's valid forever. Set a bound only when a fact is genuinely time-limited, and never write a far-future date to mean forever, because absence already means it. Done that way, the archive answers a dated question from the windows rather than from a hunch, which is the closest thing to a save state a team ever gets.

Recall is more than search. Hand the agent an address (`crystalline://engineering/retry-queue-gotcha`) and ask it to walk out from there, and it follows the relations to what the gotcha depends on and what depends on it, across domains if the links cross, ranked by how strongly each neighbour connects. That's how an agent assembles a neighbourhood before a task rather than a flat list of hits, and it's the part a folder of files can never do on its own.

3.4 Around again

The loop closes where it started. What was recalled shapes the work; the work produces something worth capturing; the sweep keeps what was captured honest; and the next session recalls a slightly better body of knowledge than the last. The small box in the middle of the diagram, *session by session*, is the whole story. None of the three beats is impressive alone. A briefing is just a briefing, a linter is just a linter, search is just search. Together, running for months, they are what turns the stranger of chapter 1 into somebody who was here yesterday and will be here tomorrow, and for whom the work in between isn't lost.

In brief

A colleague grows through a loop (learn, get corrected, remember) and Crystalline runs the same loop for an agent. Learn: onboarded at session start with a routing prompt, the agent captures observations, relations and the occasional salient engram while pairing with you, nudged once per session so the capture beat lands. Evolve: a read-only sweep returns a ranked queue of what the knowledge needs, with mechanical findings the agent completes alone and judgment findings it proposes; a fact is corrected in place or superseded with a status and a closed window, never silently overwritten. Remember: hybrid search ranks current knowledge first, temporal fields answer what applied last June and retired knowledge fades in ranking without disappearing.

CHAPTER 4

Terminology

Every word in this book has a file behind it. That's the pleasant thing about Crystalline's vocabulary: a domain is a folder you can list, an engram is a file you can print, an address is a string you can paste and a status is a line in that file you can change with whatever editor you already own. So this chapter doesn't open with a table. It opens one engram, the `retry-queue gotcha` the screenshots in this book keep returning to, and reads it from the top, stopping at each term as it comes up. By the end you'll have met every word the rest of the book leans on, and seen each one on disk.

4.1 Domains

Start with where the file lives: `~/knowledge/engineering/retry-queue-gotcha.md`. The folder is a domain. A domain is a registered folder of knowledge with a `MANIFEST.md` at its root, and the manifest is what makes it more than a folder. It says what the domain covers and when an agent should look there, and those bullets are what the routing prompt is built from at the start of every session. Registering is one command, `crystalline domain add engineering ~/knowledge/engineering`, and the name you give it there is the name the agent uses from then on. Say your team has two domains: `engineering` for the payments service and everything around it, `clients` for the people it sells to. The line between them is who could vouch for a change (the engineers for one, the people who sell for the other), and chapter 13 makes that the rule for a shared domain. A domain can also be virtual, living in the database with no folder at all; it carries the same manifest and answers to the same tools.

Inside a domain, subfolders are topic prefixes. A singleton engram stays at the root; once a cluster forms, it gets a folder, and the folder becomes part of the engram's address. The manifest's `## Notes for Agents` section is where the team writes down what it decided.

4.2 Engrams

Below is the file itself, as it sits on disk, and everything in the rest of the chapter is a line of it:

```
---
type: engram
title: Retry queue gotcha
permalink: retry-queue-gotcha
tags:
- gotcha
- payments
status: stable
recorded_at: 2026-03-09
generated: { by: claude-code/2.1.233, at: 2026-03-09T16:42:10+00:00 }
---
```

The retry queue drops any job older than twenty-four hours instead of retrying it. The team calls the queue the sin bin. The cutoff is not a tuning choice: it mirrors the chargeback window at Encom, the bank behind the card payments.

- [gotcha] The retry queue drops jobs older than 24h #payments
- [fact] The 24h cutoff mirrors Encom's chargeback window #payments
- depends_on [[Encom chargeback window]]

An engram is one markdown file holding one unit of knowledge, with a YAML frontmatter block on top. If the word sounds like something from a starship's sickbay, that's because Star Trek borrowed it from the same neuroscience textbook Crystalline did.

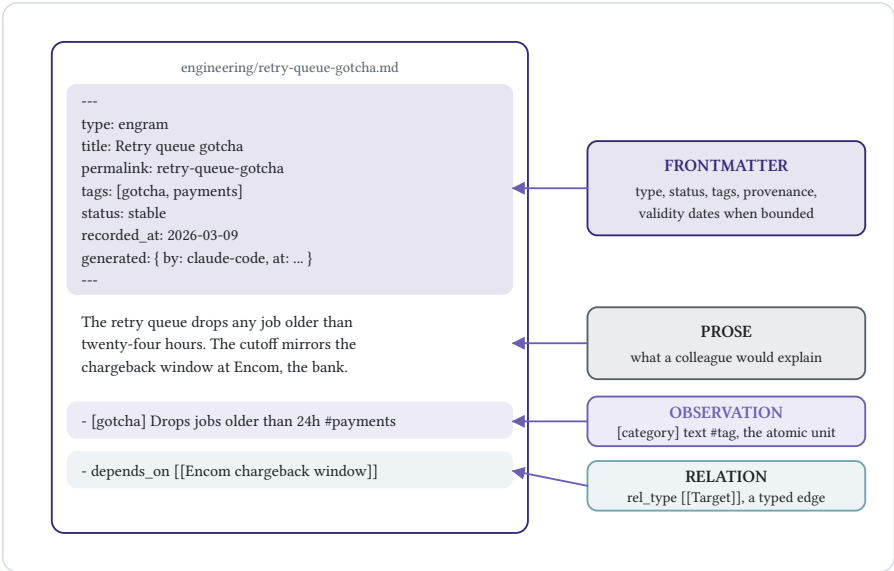


Figure 4.1: The anatomy of an engram: frontmatter, prose, an observation and a relation.

The frontmatter is identity and state. `type` says what kind of thing this is: engram by default, with guide, decision, architecture, runbook and reference as the recommended alternatives. `title` is the human name and the thing other engrams link to. `permalink` is derived from the title and is the stable half of the address. `tags` are what a search or a filter narrows on. `status` says whether the knowledge holds, and it gets its own section below. `recorded_at` is the day it was captured; `generated` records who wrote it and when. When the knowledge is time-bounded, `valid_from` and `valid_to` sit here too; this engram has neither, which means it has always held and holds until further notice. Both `type` and `status` are required and free form; the recommended values are guidance, never an enumeration a write is rejected for.

Then comes the prose, the explanation a colleague would give across a desk: what the thing is, what it's called and why it is the way it is. The prose is what search matches on and what a reader reads first.

Then come the observations. A top-level bullet of the form - [category] text #tag is an observation, and observations are the atomic unit of an engram's body: the fact, the decision, the gotcha, each indexed on its own line so a search hit can point at it. The category in brackets is free text, and the precise ones earn their keep: decision, fact, pattern, gotcha, convention, lesson, risk, idea. A trailing #tag attaches the tag to that observation.

Then comes the relation. A top-level bullet of the form `- rel_type [[Target]]` is an edge in the knowledge graph, typed by whatever word comes first: `depends_on` here, `supersedes` and `superseded_by` in chapter 3, `derived_from` and `split_into` between the two halves of a split, `relates_to` when nothing sharper fits or a quoted phrase for a multi-word type. The target is a title (resolved within the domain) or `[[domain:Title]]` across domains. The target's page shows the edge back as a backlink, and a target that doesn't exist yet is an unresolved link the sweep will mention.

One topic per engram is the rule that keeps all of this legible. When the topic grows, the engram is edited, not forked; when it splits, it splits by granularity or by validity. A large capture splits by granularity, a distilled summary beside a source engram holding the full text, linked both ways. An engram that bundles facts with different lifetimes splits by validity. Validity is set per engram, so when one fact expires while the rest holds, the still-valid observations move into an engram of their own first. The new engram carries the source's tags and type, a `stable` status and no validity window, and points back with `derived_from`; the old one keeps what expired, points forward with `split_into` and retires.

4.3 The open format underneath

None of that is a private format. The engram extends Google's Open Knowledge Format, OKF, version 0.2: plain markdown with YAML frontmatter, readable by any OKF tooling, with Crystalline's routing, temporal and graph conventions layered on top. Unknown frontmatter keys survive every write and edit, so an OKF document from somewhere else drops into a domain intact and comes back out intact. The generated block is the provenance record, and the `index.md` Crystalline writes at a domain's root declares the format version for anything that browses the folder without Crystalline running. Your knowledge stays files in a folder that any editor opens and any diff tool diffs, whatever tools come next.

4.4 Statuses

`status` is the field a reader that acts on knowledge needs most, because it answers the one question a page can't answer about itself: does this still hold? `stable` is the default and the word for knowledge that holds now; the older spelling `current` means the same state, and a filter on either finds both. Before stability there are `draft`, `proposed`, `idea` and `poc`, for knowledge that's on its

way. implemented is on the recommended list too, and like any status outside the retirement set it ranks at full strength.

After stability there is the retirement set, four words that each mean one thing. deprecated says don't do this again. superseded says a newer engram replaced this one, and the two are linked. archived says retired but kept for the record. legacy says still true of the old installation that's somehow still running, like the terminals aboard the Nostromo, but not to be built on. A status in the retirement set fades the engram in search ranking, gently and never to zero, so current knowledge surfaces first and the past stays one scroll away, dated. Retiring is how a fact leaves the present without leaving the archive; deleting is for mistakes.

4.5 Salience

Most engrams are ordinary, and that's fine. A few are the ones everything else hangs from: the rule that explains a dozen other rules, the insight that ended a week-long bug. Those can carry a salience, a number from zero to ten in the frontmatter, and hybrid search adds a small, bounded lift for it, so a salient engram ranks above equally relevant neighbours while relevance keeps the upper hand and nothing is ever filtered out by it. It's set sparingly at capture time, and raised later when an engram turns out to have been what a task hinged on. The retry-queue engram carries none, and most never will.

4.6 Addresses

Every engram has an address, `crystalline://engineering/retry-queue-gotcha`: the scheme, the domain, the permalink. It's the one absolute form. An agent hands it to another agent, `build_context` anchors on it, Fluid copies it to the clipboard in one click. It survives a commit message, a ticket or a chat. A folder prefix appears in the permalink (`crystalline://engineering/queues/retry-queue-gotcha` if the engram had been filed under `queues/`), and a trailing `/` anchors the whole folder at once.

The rule that trips people up is the scheme-less one. An identifier without `crystalline://` is domain-relative: `retry-queue-gotcha`, together with the domain named beside it, never `engineering/retry-queue-gotcha`, which would be read as a folder called `engineering` inside whatever domain you meant. The same holds for links: `[[Retry queue gotcha]]` resolves in the domain the linking engram lives in, and `[[engineering:Retry queue gotcha]]` reaches across. Either the address is absolute and carries its domain, or it's relative and the domain

comes from context. There's no third form, and that strictness is what lets a permalink stay short.

4.7 The MANIFEST, up close

The last term is the first file in every domain. `MANIFEST.md` is an engram of type: `manifest` with the permalink `manifest`, which means it's read, indexed and edited with the same tools as everything else. Its job is routing, and two sections do it. `## Scope` says what the domain covers. `## When to Use` says when an agent should route a task here, and those bullets, one line per domain, are what become the session-start briefing. `## Notes for Agents`, the third section the scaffold creates, holds the conventions the agent should follow inside this domain, the folder layout above all. Two optional sections come later. `## Tag Aliases` folds an old tag name into its canonical one so old searches keep working. `## Provisioning` names the folders of skills, slash commands, subagent definitions and MCP server configs a domain ships beside its knowledge. The tools that knowledge depends on travel with it, and install into a harness once a person says yes. That section gets a chapter of its own (Chapter 15). An engineering manifest, a few months in:

```
---
type: manifest
title: engineering
permalink: manifest
tags:
  - manifest
status: stable
recorded_at: 2026-03-02
---

# engineering

## Scope

- The payments service, its retry queue and the services around it
- Deploy runbooks, team conventions and the decisions behind them

## When to Use

- Route questions about the payments service, the retry queue or a
  deploy here
- Check here before touching anything the team has already been burned
```

by

Notes for Agents

- Runbooks live under runbooks/, gotchas at the root
- One engram per gotcha; edit the existing one before writing a second

Write the When to Use bullets the way you'd brief a new hire on which drawer holds what. A machine reads them, but it reads them the way a person reads a label.

Under the hood

The manifest being an engram is why routing needs no separate configuration: crystalline prompt system parses each registered domain's MANIFEST.md, takes the ## When to Use bullets, or ## Scope when there are none, and emits one routing line per domain plus the behaviour rules. Two filenames in a domain are reserved and never engrams. index.md is the table of contents Crystalline regenerates after every write so the folder navigates itself on a git forge; log.md is reserved but never generated. A title that would slug to either is refused.

In practice

Say your books domain is a year old and you start a second one, shops, because where to buy is a different kind of knowledge from what to read: which shop orders from small presses, which one delivers by Friday if you ask on Monday, which one never again. Your first engram there is about Capsule Corp Books, and it needs to say that the paperback rule in books (wait for the Tessier-Ashpool paperback, the hardbacks fall apart) is why you order there. You type - relates_to [[Wait for the paperback]], and the sweep tells you the link doesn't resolve, because shops has no engram by that title. You change it to [[books:Wait for the paperback]] and the edge appears, with a backlink on the books side that nobody had to write.

A week later a friend asks what to read and where to get it, and instead of typing the answer out you hand your agent the address, crystalline://shops/capsule-corp-books, and ask what you'd tell her. It picks up the address, walks one hop into books and comes back with the paperback rule, the small press and the two authors you'd tell anyone about. That's two domains and one edge, and the string is still readable in a year.

You now speak the language. The rest of the book is conversation.

In brief

A domain is a registered folder with a `MANIFEST.md` whose `When to Use` bullets route an agent to it; an engram is one markdown file with YAML frontmatter, prose, observations of the form `- [category] text #tag` and relations of the form `- rel_type [[Target]]`, all on Google's Open Knowledge Format so nothing is locked in. `status` says whether knowledge holds: `stable` is the default, and `deprecated`, `superseded`, `archived` and `legacy` are the retirement set that fades in ranking without disappearing; `salience` is a rare, bounded lift. The absolute address is `crystalline://domain/permalink`, and anything without the scheme is domain-relative. The manifest is itself an engram, which is why routing needs no configuration of its own.

READY. ■

PART II

Getting Started

CHAPTER 5

Installing locally

Don't Panic.

- Douglas Adams, *The Hitchhiker's Guide to the Galaxy*

Everything in this chapter fits in one sitting, and the sitting ends with a test: you'll ask your agent about something it learned an hour earlier, in a session that has never heard of it, and it will answer. Until that happens nothing is installed, whatever the package manager says. So the chapter is built backwards from that moment. Install the binary, wire the harness, give the agent a domain: three steps, the same three on every platform, and then the test.

If you skipped chapter 4, two words carry the chapter, and everything below makes one of each: a domain is a registered folder with a `MANIFEST.md` that says when an agent should look there (Section 4.1), and an engram is one markdown file of knowledge inside it (Section 4.2).

5.1 Install the binary

Crystalline is one binary, and it gets onto a machine the way that machine usually gets things.

Debian, Ubuntu and their relatives get a `.deb` for `amd64` and `arm64` from the releases page. It ships a `systemd` unit, installed disabled (chapter 12 turns it on). Windows gets an MSI that adds Crystalline to the `PATH` and upgrades in place. The Windows builds aren't code signed yet, so expect a SmartScreen prompt and check the download against the `SHA256SUMS` file beside it. The bare binary ships for every platform too, and the macOS builds are signed and notarized. The permanent address for all of it is on the colophon page: <https://github.com/jordiboehme/crystalline/releases/latest>.

Claude Desktop is the pleasant exception: no binary, no terminal. Download `crystalline-v<version>.mcpb` from the same page, one universal bundle for Apple Silicon Macs and Windows, with per-arch bundles for Intel Macs, then

in Claude Desktop open Settings > Extensions > Advanced settings > Install Extension... and pick the file. Upgrading is the same motion (Desktop has no in-app update for an extension): download the newer .mcpb and install it over the current one. Desktop users can skip the terminal parts of what follows: the agent creates its first domain itself.

Under the hood

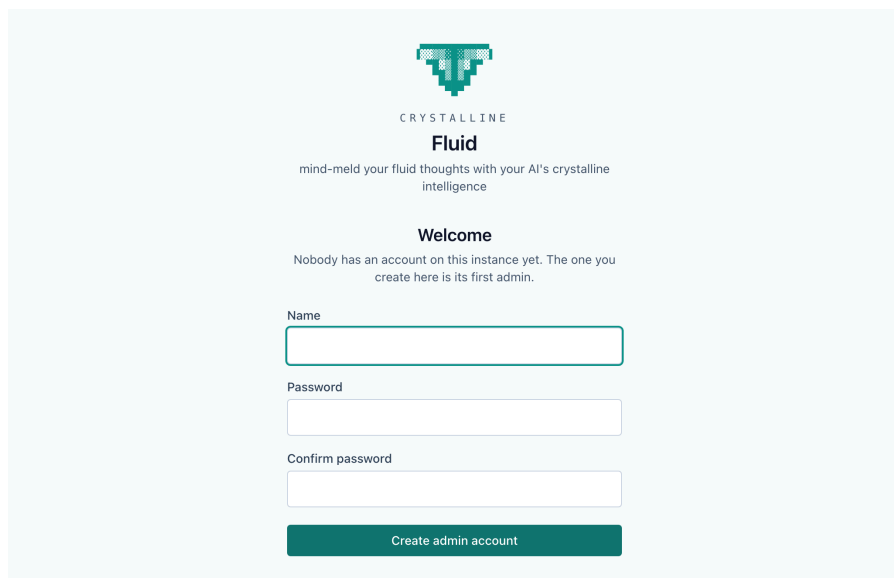
On a Mac with Homebrew the whole install is one line:

```
brew install jordiboehme/tap/crystalline
```

5.2 First run

Nothing needs starting: the first agent that connects starts a background daemon (the one process that holds the index open), and every later agent, terminal and browser tab attaches to it. `crystalline serve` runs the same daemon in the foreground if you want to watch it, and `crystalline serve --daemon` in the background, but on a laptop most people never type either.

What you'll type into is a browser: the daemon serves Fluid at `http://localhost:7411` by default, however it was started. The first visit to an instance with no accounts yet is a short form, not a login page: pick a name and a password and you're that instance's admin, already signed in (Figure 5.1). You're trusted to fill that form in because you're on the machine: a connection from the machine itself needs no token, and a bind anyone else could reach gets a different treatment in chapter 12. Once any account exists the form is gone for good.



The screenshot shows a web browser window with the Crystalline Fluid interface. At the top is a green pixelated logo. Below it, the text reads "CRYSTALLINE Fluid" and "mind-meld your fluid thoughts with your AI's crystalline intelligence". A "Welcome" message states: "Nobody has an account on this instance yet. The one you create here is its first admin." Below this is a form with three input fields: "Name", "Password", and "Confirm password". At the bottom of the form is a green button labeled "Create admin account".

Figure 5.1: First visit: creating the admin account in the browser.

Semantic search wants one more download, the local embedding model: the daemon fetches it in the background on its first start, and crystalline model download fetches it ahead of time. Until then search is plain text, which works, and finds only the words you searched for.

5.3 Onboarding a harness

This is the step that turns a binary into a colleague, and it's one command per harness. Every part of what that command does is something a new hire gets on day one. It registers the MCP server with the harness, so the agent has the tools. It writes two hooks: a session-start hook that delivers the briefing on a fresh start, after a `/clear` and after a compaction, and a stop hook, the once-per-session nudge from chapter 3 that asks the agent to capture what it learned before the session ends. It also copies the four skills into the harness's skills folder, the playbook that says when to use which tool. Run it twice and the second run touches nothing.

Look at the briefing once with your own eyes: it's the whole of what your agent knows about you at the start of a session. `crystalline prompt system` prints it (Figure 5.2): an intro, the behaviour rules and one line per registered domain, drawn from the `## When to Use` bullets of that domain's manifest. A narrow question goes to the domain that owns it, a broad one sweeps all of them, and a

write always names its domain. On a fresh install the domain lines are empty, and that's correct: the desk is set up and nobody has shown the new hire the filing cabinets yet.

Claude Desktop runs no hooks and needs none: its server delivers the same routing block as part of the MCP server's instructions on every connection, and the optional companion skill from the releases page, uploaded under Settings > Capabilities > Skills, adds the capture and collaboration playbook on top. A chat client with no hooks that never shows the model a server's instructions gets a standing instruction instead: `crystalline prompt connector` prints the paragraph to paste into its custom instructions, and the agent fetches its own onboarding with one `list_domains` call at the start of every session.

Install a second harness and you've added a door, not a second colleague. Claude Code in a terminal, Codex, the Copilot CLI and Claude Desktop each spawn the same `crystalline mcp` over `stdio`. The first of them to connect starts the one daemon the rest attach to, so every one of them sees the same domains, the same index and the same briefing. Tell Desktop something over lunch and the terminal session knows it in the afternoon, with nothing copied and nothing configured twice. The one way to end up with a second colleague is a version gap: a Desktop bundle older than a binary Homebrew has since upgraded can neither reach that daemon nor take the index from it. Instead of failing, it comes up degraded, with a single status tool whose instructions say to download the newer `.mcpb` and install it over the old one. Do that, and the door is open again.

```

ripley@tyrell-laptop: ~ (crystalline prompt system)

$ crystalline prompt system
CRYSTALLINE KNOWLEDGE ROUTING

You are being onboarded to the knowledge accumulated in this workspace. It is served by the
crystalline MCP server; these instructions govern its tools (your harness may prefix tool
names, for example mcp_crystalline_search_engrams). The domains below are registered and
ready to use; route searches and reads through them instead of starting from zero.

- engineering: Route questions about the payments service, the retry queue or a deploy here;
Check here before touching anything the team has already been burned by
- clients: Route anything about a customer, a contract or a renewal here; Check here before
preparing for a customer meeting
- board: Route anything about the funding round or a board meeting here

Behavior:
- Narrow question, one domain fits: search_engrams with domains=[that domain].
- Broad or unclear question: search_engrams with no domains sweeps them all.
- write_agram, edit_agram, move_agram and delete_agram always need an explicit domain.
- Pass write_agram a folder matching the domain's layout: a topic prefix build_context can
glob, a subfolder once a topic clusters, the root for singletons.
- Check vocabulary for tags and categories in use before inventing one.
- Saliency (0-10) lifts an engram in search: set it at write time on exceptionally valuable
knowledge and raise it when one proves key to a task.
- build_context on a crystalline:// anchor gathers knowledge around a task.
- Read a MANIFEST with read_agram only when the domain's routing line is not enough;
list_domains with include_routing=true re-fetches the index mid-session.

```

Figure 5.2: The routing prompt an agent receives at session start.

Under the hood

Claude Code takes one command:

```
crystalline install claude-code
```

Codex takes `crystalline install codex` and the Copilot CLI `crystalline install copilot`. The MCP server it registers is `crystalline mcp --harness claude-code. --skip-mcp, --skip-hooks and --skip-skills` leave a part out, --project writes into the current repository's config instead of your global one, and `crystalline uninstall claude-code` reverses the lot while leaving anything that isn't Crystalline's own alone.

The install is recorded in a local receipt, and the receipt is what keeps a harness current: the first time a newer Crystalline runs, it refreshes the installed skills at session start, updating changed ones (an edited copy is kept beside the new one as `SKILL.md.bak`) and removing any the new version no longer ships. The --harness flag in the MCP registration is the other half of the same idea. A `crystalline mcp` process started with it asks the receipt whether that harness has its hooks wired, and if so serves that session neither the skills over MCP nor a second copy of the routing block, since the harness already has both as files.

Nothing in that decision comes from the connecting client: it's settled before the session opens. A registration written by hand without the flag, or a remote client, is served the full surface, and `skills.serve` set explicitly to true or false makes every client identical.

5.4 The first domain and the first engram

Now the filing cabinet. Before an agent can keep anything it needs a domain, and this section runs the whole loop in miniature: the domain is scaffolded and registered, one fact is written into it as an engram, and a question phrased nothing like the fact gets that fact back. The last one is the semantic ranking at work, on a model the daemon fetched in the background when it started.

Where that folder lives is up to you, and it doesn't have to be where any other domain is: a domain is registered by its own path, and nothing ties one registration to the next. `engineering` can sit in a git checkout under `~/knowledge`, reading under Documents where the agent put it, a third on a NAS mount that is already backed up. All three are served by the one daemon and named in the same briefing. Documents/Crystalline is only the default for a domain an agent creates when nobody names a path, and the `domains_root` setting moves that default without moving anything already registered. A network share changes two things. First, the index stays in the local state directory, and should: it's a database, and a database on a share is how people learn about file locking the hard way. The share holds the markdown and nothing else. Second, the daemon's file watcher hears the machine it runs on, not the network, so an engram written into that folder from another computer shows up at the next `crystalline sync` or the next daemon start, not the moment it lands. Written through Crystalline from any harness on this machine, it's indexed as it lands, share or no share.

Open the manifest before you go any further and make sure the `## Scope` and `## When to Use` sections say what you would say, because those bullets are the one line your agent reads about this domain at the start of every session. Write them the way you'd label a drawer for a new hire: "route questions about the payments service, the retry queue or a deploy here". The agent drafts them from what you say, and you check them once.

It takes three prompts and no terminal:

5. INSTALLING LOCALLY

```
You: Set up a Crystalline domain called engineering in
~/knowledge/engineering. Its scope is the payments service:
route questions about the payments service, the retry queue
or a deploy there.
Agent: Created and registered engineering. The manifest's Scope and
When to Use sections carry what you just said, so your next
session will see the domain in its briefing.

You: Remember in engineering: the retry queue drops jobs older
than 24 hours. It is a gotcha, tagged payments.
Agent: Captured "Retry queue gotcha" in engineering, at
~/knowledge/engineering/retry-queue-gotcha.md.

You: Why does the payments queue lose jobs?
Agent: The retry queue drops jobs older than 24 hours, from
"Retry queue gotcha" in engineering.
```

A Desktop user's path is exactly this, and it works even without the first prompt: an agent that finds nowhere to write creates a domain itself under Documents/Crystalline, and the first engram lands there. Open the manifest afterwards and make sure the two sections say what you'd say.

Under the hood

The first prompt was three commands, the last three lines of the README's quick start. `domain init` scaffolds the folder with a starter manifest and `domain add` registers it under the name the agent will use and indexes what is already there (there's no separate sync step). The agent did the same through its own tools: `add_domain` to scaffold and register, then `edit_engram` on the manifest.

```
mkdir -p ~/knowledge/engineering
crystalline domain init ~/knowledge/engineering --name engineering
crystalline domain add engineering ~/knowledge/engineering
```

The second prompt was one command (the agent's was one `write_engram` call), and running it by hand shows the file appear in the folder, `retry-queue-gotcha.md`, the file chapter 4 read line by line:

```
crystalline write engineering "Retry queue gotcha" \
--content "- [gotcha] The retry queue drops jobs older than 24h"
```

```
#payments" \
--tags gotcha,payments
```

The third prompt was a search, and crystalline search "retry queue" finds it as a plain text hit. On a machine where no daemon has fetched the model yet, fetch it, re-sync with embeddings and ask something the engram never says:

```
crystalline model download
crystalline sync --embed
crystalline search "why does the payments queue lose jobs"
```

crystalline sync is for files that arrive some other way (an editor, a git pull) when no daemon was watching, and crystalline status shows what got indexed.

5.5 Verifying the loop

Now the test. Start a session in your harness and ask about the retry queue without explaining anything. Watch what the agent does before it answers, because that's the recall beat and it proves the wiring: it searches engineering (the briefing told it that's where the payments knowledge lives) and answers from the engram, with the address in hand if you ask for it.

What that doesn't prove yet is the capture beat, the one that matters over months. Tell the agent something it couldn't have known, the way you did on chapter 1's Monday:

```
You:    The cutoff is 24 hours because Encom's chargeback window is
        three days - the queue needs two days to notice a job. Remember
        that, and why.
Agent:  Captured: added the chargeback window as the reason to
        "Retry queue gotcha" in engineering, with a depends_on relation
        to a new "Encom chargeback window" engram. Want me to adjust
        the wording?
```

Close the session, open a new one (it has never heard of Encom) and ask why the cutoff is a day. If the answer names the chargeback window, you're done: the agent looked something up that it learned in a conversation that no longer

exists. It's a Voight-Kampff test run in reverse: you're not checking whether the thing across the desk is a replicant, you're checking that it remembers.

Once the test passes, close the terminal. Nothing in daily use needs it: the agent recalls and captures through its tools, the daemon starts itself and the one place you'll go on purpose is the browser tab. The commands in this chapter were the installation, and from here on what you type is addressed to a colleague.

Under the hood

When it fails the fix is usually quick. If the agent answered from general knowledge without searching, the session didn't get the briefing: run `crystalline prompt system` yourself and check that engineering has a line (if it has none, the manifest's `## When to Use` section is empty). If the agent recalled the gotcha but not the reason, the capture never landed: look in `~/knowledge/engineering` for the file and check that you said yes when it proposed writing.

In practice

Your first afternoon is the five lines above. You run the `brew` line and the `install` line, `domain init` a books folder, and your agent's first session opens with a briefing that names it and nothing else, because nothing has been taught yet. You tell it the series rule, the two shops and the press whose paperbacks you wait for, to see what happens, and close the session. In the next one you ask what to read, and it starts from your rules. In chapter 2 the bookseller took a year to learn you; the agent took a session. By Wednesday you have three doors open: Claude Code in the terminal, the Copilot CLI in a second terminal and Claude Desktop for the thinking you do away from the keyboard. Two `crystalline install` lines and one Desktop extension were the setup, and the daemon behind all three is the one that started that first afternoon. What you tell Desktop on Tuesday evening about the manga translations is what Copilot recalls on Wednesday morning, from the same file in the same folder, and nothing was copied between them.

Your sister, who has never opened a terminal on purpose, installs the Desktop extension the same week. She has no domains, and her agent makes one the first time it needs somewhere to write, a reading folder under `Documents/Crystalline`. By Friday it holds four engrams about the Dungeon Crawler Carl books she's lining up for the holiday, and she opens `localhost:7411` to read them. Two installs, one with a keyboard and one without, and the same loop runs on both.

The desk is set up, the drawers are labelled and the new hire has remembered one thing overnight. Everything after this is a matter of what you choose to teach.

In brief

Installing is three steps on every platform: install the binary (Homebrew, a `.deb`, an MSI or the Claude Desktop `.mcpb`, which skips the terminal and upgrades by installing the newer bundle over the old), wire the harness with `crystalline install <harness>` and give the agent a domain with `domain init` and `domain add`. The install writes the MCP registration, the session-start hook that delivers the briefing, the stop hook that nudges capture and the four skills, and it's safe to run again. Nothing needs starting: the first agent to connect starts the daemon, which serves Fluid at `localhost:7411`, where the first visit creates the admin account. Every harness on the machine shares that daemon, and a domain is registered by its own path. The install is finished when a fresh session recalls something a previous one learned.

READY. ■

PART III

Working with Crystalline

CHAPTER 6

Onboarding an agent

On a new colleague's first morning somebody hands them the floor plan. Payments is on the second floor, the people who know about the bank sit by the window, the runbooks are in the cupboard that sticks. Which door leads where is the whole of day one, and it's enough, because a person who knows where things are can find out everything else themselves.

That floor plan is what an agent receives at the start of every session with Crystalline. This chapter is about the day it arrives: what the briefing says, how the agent finds its way with it, what it means to hand a colleague a task instead of a question, and the reminder at the end of the day that turns a session into experience. Chapter 5 showed the wiring. This is what the wiring is for.

6.1 The briefing at session start

A session opens, and before the agent has read a word of your request, the session-start hook runs `crystalline prompt system` and puts its output in front of the model. Here is one a few months in, abridged:

CRYSTALLINE KNOWLEDGE ROUTING

You are being onboarded to the knowledge accumulated in this workspace. It is served by the crystalline MCP server; these instructions govern its tools. [...]

- `engineering`: Route questions about the payments service, the retry queue or a deploy here; Check here before touching anything the team has already been burned by
- `clients`: Route anything about a customer, a contract or a renewal here; Check here before preparing for a customer meeting

Behavior:

- Narrow question, one domain fits: `search_engrams` with `domains=[that`

```

domain].
- Broad or unclear question: search_engrams with no domains sweeps
  them all.
- Read a MANIFEST with read_agram only when the domain's routing line
  is not enough; list_domains with include_routing=true re-fetches the
  index mid-session.

```

Read it as the agent reads it. The opening says what this is and whose tools it governs, so an agent with four MCP servers connected knows which `search_engrams` is meant. Then there's one line per domain, and every word of those lines came from a person: they're the `##` When to Use bullets of each manifest, joined with semicolons, exactly as the team wrote them (Section 4.1). On your own machine that's every domain registered, since the CLI and a stdio session are the machine owner. On a shared instance the agent connects as an account, and the block lists every domain that account may see: a private domain it's no member of is absent from the block, from `list_domains` and from every search, answered as if nobody had registered it (Chapter 14). Then come the behaviour rules, the same on every instance, naming the exact tool each one reaches for. That's the floor plan (Figure 6.1). It says where things are, nothing about what's in them, and the agent goes and looks.

```

ripley@tyrell-laptop: ~ (crystalline prompt system)

$ crystalline prompt system
CRYSTALLINE KNOWLEDGE ROUTING

You are being onboarded to the knowledge accumulated in this workspace. It is served by the
crystalline MCP server; these instructions govern its tools (your harness may prefix tool
names, for example mcp_crystalline_search_engrams). The domains below are registered and
ready to use; route searches and reads through them instead of starting from zero.

- engineering: Route questions about the payments service, the retry queue or a deploy here;
Check here before touching anything the team has already been burned by
- clients: Route anything about a customer, a contract or a renewal here; Check here before
preparing for a customer meeting
- board: Route anything about the funding round or a board meeting here

Behavior:
- Narrow question, one domain fits: search_engrams with domains=[that domain].
- Broad or unclear question: search_engrams with no domains sweeps them all.
- write_engram, edit_engram, move_engram and delete_engram always need an explicit domain.
- Pass write_engram a folder matching the domain's layout: a topic prefix build_context can
glob, a subfolder once a topic clusters, the root for singletons.
- Check vocabulary for tags and categories in use before inventing one.
- Salience (0-10) lifts an engram in search: set it at write time on exceptionally valuable
knowledge and raise it when one proves key to a task.
- build_context on a crystalline:// anchor gathers knowledge around a task.
- Read a MANIFEST with read_engram only when the domain's routing line is not enough;
list_domains with include_routing=true re-fetches the index mid-session.

```

Figure 6.1: The same block unabridged, as crystalline prompt system prints it.

Under the hood

How it reaches the desk depends on the harness, as chapter 5 said. Claude Code and Codex run the hook on a fresh start, after a /clear and after a compaction, and skip a resumed session; the Copilot CLI's hook does the same on a fresh start and stays quiet on a resume; Claude Desktop gets the block inside the MCP server's own instructions; a remote client fetches it with one list_domains call. Either way the agent starts out knowing which drawer holds what and nothing more, the state a good new hire is in at nine on Monday.

The block is built to survive being cut off. A harness in tool-search mode truncates a server's instructions at roughly two kilobytes, so the MCP rendering keeps under a 1900-byte budget and degrades in fixed steps: every routing line drops to one bullet, then the list collapses to a count line pointing at the re-fetch call.

A remote client gets that count line on arrival whatever the block's size. The MCP initialize handshake is answered before any request is in hand, so over HTTP there's nobody to scope the block to and no way to leave a private domain's name out of it. The handshake carries the behaviour rules, how many domains are registered and the pointer at list_domains, which does know who's asking and filters.

6.2 How the agent finds its way

A routing line is a targeting aid. The routing skill says so in as many words, and adds what every colleague knows about floor plans: the label on the door isn't the inventory of the room.

The first decision is scope. A narrow question, one that clearly belongs to one service, team or customer, goes to the domain whose line describes it, in a single `search_engrams` call with `domains` set. A broad or unclear question omits `domains` and sweeps everything, each hit labelled with its domain. The agent doesn't stop to ask you which it is: a search is cheap and usually settles the ambiguity, and a clarifying question is for after it came back empty.

The second decision is when to widen. A scoped search can return plausible near-misses when the subject belongs to one domain and the question to another: who gets paged about refunds is a people question, not a payments one. When the obvious domain keeps missing after a rephrasing or two, the agent runs one unscoped sweep. Then it reads. A snippet is a short window around a match, and a colleague who answers from a snippet is guessing, so anything that summarizes or quotes an engram goes through `read_gram` first. A strong hit becomes an anchor for `build_context`, which pulls in what the engram depends on before the work starts.

The agent was never given the engrams, only the doors, and what's behind each is fetched when a task calls for it. A colleague with a floor plan beats one carrying the building around in a wheelbarrow.

6.3 Assign, do not copy

You wouldn't message a colleague a question, copy their answer into your editor and do the work yourself. You'd assign them the task, because doing it is how they get better at it. An agent with Crystalline's tools is in the same position, and treating it as a lookup service wastes most of what you set up.

Give it the task. "Add a metric to the retry queue" lands better than "what do you know about the retry queue", because the first sentence sends the agent through the whole loop on its own: it routes to engineering, recalls the gotcha and the bank rule, reads the engram in full, walks one hop to the deploy runbook, writes the change in Python and tells you what it found on the way. The floor plan only pays off when somebody walks the corridors, and the somebody should be the agent.

It follows the processes a person would, since it can be wrong in the same ways: review for its changes, a yes for its captures. A colleague you never delegate to never learns your work, and neither does an agent.

In practice

Here's your first week with the agent, session by session. Monday is the afternoon from chapter 5: the briefing names books, and the agent knows your rules before you've typed one. On Tuesday you give it a real task (something for Saturday's flight) and explain nothing. The agent routes to books, recalls the series rule and the translations, finds the note about the Tessier-Ashpool paperbacks you captured on Monday and comes back with three (a Doctorow you missed, the next *Dungeon Crawler Carl* and the *Deep Space Nine* novel that follows the one you finished last month), each with the shop that has it. On Wednesday it's wrong for the first time: it recommends the new Vickers in hardback, and you tell it you don't buy hardbacks, because you read outdoors and a hardback doesn't survive the backpack. It asks whether to capture that in books as a [convention]. You say yes, and add the exception: a hardback is for collecting, bought after the ebook, when a book turns out to deserve a place on the shelf.

Thursday is a long session sorting the pile by the bed, and when it ends the reminder fires once: the agent proposes two captures (that you abandon anything with a map on page one, and a shelf engram per *Star Trek* series, classic, *Next Generation*, *Deep Space Nine* and the *Destiny* trilogy, saying which volumes you own and which is next) and raises the salience of the translations engram. On Friday you ask it to order the Vickers, and it answers, before you've said which edition, that the hardback is out by your own rule until you've read it, and the ebook is out today. On the first session it knew where the drawers were. On the fifth it told you something about yourself, unasked, that you'd taught it two days before. That's the moment a stranger stops being one.

6.4 The reminder at the end of the day

A first week only counts if what was learned survives the weekend, and that's the beat agents skip. Left alone, a model finishes the task and stops, and the gotcha it met at minute forty goes with it. So `crystalline install` wires a reminder, the closest thing the loop has to a habit.

It's a `Stop` hook. On the first stop after a session has gained real substance it asks the agent to review the conversation for durable learnings: facts, decisions, patterns, gotchas, corrections from you. For each one not yet captured the agent proposes an engram in the fitting domain and waits for a yes; a correction

that makes an existing engram wrong becomes a proposed edit or supersession rather than a second engram. If a recalled engram proved to be the key to the task, the agent raises its salience. If nothing qualifies, it finishes without mentioning the check. The reminder fires at most once per session, and costs about a hundred and twenty tokens. When maintenance is due, a paragraph rides along asking for a sweep; when a team domain holds unshared work, another asks the agent to propose sharing, still a proposal, because sharing puts somebody's work in front of reviewers.

An agent that reaches a shared instance only over MCP has no hooks. Crystalline's one reliable channel to a model mid-session is a tool result, so the reminder rides there: a write, edit, move or delete comes back with a single clearly prefixed line appended when something is due, never a search or a read, so recalled knowledge is never mixed with guidance. Sharing and maintenance travel this way. Capture doesn't, because it's a reflection at the end of a conversation and the server can't tell when a conversation ends; the routing block keeps teaching capture as standing practice instead. Each person is nudged on their own cadence, a few hours apart:

```
---
[crystalline] 3 changes across engineering are not yet shared. If
that work is done, propose sharing with share_changes so the domain
owner can review it - and wait for a yes.
```

The floor plan gets a new hire through the door. What they carry back at the end of the day is what makes them worth having on Tuesday.

In brief

Every session opens with a routing block: one line per domain the session may see (the manifests' When to Use bullets) and the behaviour rules naming the exact tools. Each harness delivers it the way chapter 5 wired it. The agent scopes a narrow question to one domain, sweeps for a broad one, widens when scoped hits keep missing and reads before it answers. Hand it tasks rather than questions, and let the once-per-session reminder close the day: propose captures, wait for a yes.

CHAPTER 7

Teaching

The best onboarding you ever had was a walk. Somebody took you across the floor, stopped at a machine and said “that one lies”, and you never trusted it again. Nobody wrote that down. It was not on the slide. It was the kind of knowledge that only moves from one head to another when a person decides it matters enough to point at, and the pointing is what this chapter is about.

Day one, last chapter, was the floor plan. Now comes the proper onboarding: what you tell a colleague in the first month, how you tell an agent the same things, what a well-taught fact looks like on disk, and the two rooms where the teaching happens, a session and a browser tab. It ends with two people editing the same page at once, which is where teaching stops being a lecture.

7.1 What to teach

Ask any good manager what a new hire needs and the list comes out in the same shape every time: what we value, and what we won’t do even when it would be faster. How things are done here (the review, the deploy, the meeting that actually decides) and what’s expected of the role. Which tools we use, and which we abandoned and why. And where information lives: which repository, which dashboard, which person. An agent needs the same list, in the same order of importance, and the last item is the one that makes the other four cheap.

A person’s knowledge doesn’t look like a source tree. No engineer on your team holds the payments service in their head by the letter. They know it’s in the payments repository, that the retry logic is under `queue/`, that the config nobody documented is in the deploy chart, who owns it and that Encom’s rule explains its strangest number. They know how to get access and which door to knock on. What they carry is a map of the territory plus the concepts to navigate it fast, and that’s exactly what to teach an agent. The source tree it can read with the tools its harness gives it whenever a task calls for it. What it needs from you is which repository holds the code for which purpose, how access is granted,

the architecture and the vocabulary. Pointer knowledge, access knowledge and the mental model. Teach the map and the territory stays where it is, reachable.

So the engrams that earn their place in the first month are the ones a person would say across a desk: the chargeback rule behind the cutoff, the name the team uses for the queue, the fact that the wiki says gateway where the code says edge, and the rule about Fridays. Not one of them is on any page, and every one of them is what an experienced colleague knows.

7.2 Teaching through the agent

In a session, the simplest way to teach is to say something worth keeping and watch what happens.

Tell the agent “the settlement job runs at six because Encom’s file lands at five, and it fails silently if the file is late”, and it doesn’t write. It searches first, in engineering, because the capture skill makes search-before-write a hard gate, and it checks whether an engram already owns the topic, in which case the new fact becomes an edit of that engram rather than a second one beside it. Finding nothing, it proposes: “Should I capture this in engineering as a [gotcha]?” A terse “store this” is already a complete instruction and skips the question; an aside you obviously didn’t mean to keep gets no question at all. It names the domain every time, since a write has no default, and when the owner isn’t obvious it asks rather than guesses. A rotation detail learned during a payments incident is people knowledge, and the agent sweeps every domain before filing it where the incident happened to be.

Then it writes a well-formed file: a title, tags drawn from the vocabulary already in use rather than a near-synonym, a folder matching the domain’s layout and a body of at least three lines, because verification rejects a lone bullet as too thin. Hand it one sentence and it pads honestly, one bullet for the fact in your words, one for how and when it was learned, one for the scope and never an invented specific. You glance at the result the way you glance at a new hire’s first commit, and say yes. The write answers with its nearest neighbours too: the receipt names the existing engrams closest in meaning to what just landed. The instruction beside them is to read the one that fits, then merge into it, supersede it or link it, never to pass over the list in silence. So the duplicate check runs twice, before the write on your sentence and after it on the engram as written.

Correction is the same conversation. “The file lands at half past five, not five” finds the engram and changes the value where it stands. “We moved settlement

to the new broker in August” is a different thing, the world changing, and the agent supersedes rather than edits, as chapter 3 walked through. Teaching and correcting are one motion, which is why the framing in chapter 2 said teach, rather than file.

7.3 Teaching through Fluid

The other room is the browser. Sometimes the fastest way to teach is to write it yourself, without a model in the loop and without spending a token: open the engram in Fluid, switch to edit and type (Figure 7.1). The editor previews live, the frontmatter is a form, and typing `[]` completes titles across every domain, so a relation is two keystrokes and a pick. The file on disk is what changes; the page you were reading (Figure 11.4) is that file. Save a page you’re editing alone and you get the second look a session’s write gets, as a panel above the text with the same instruction.

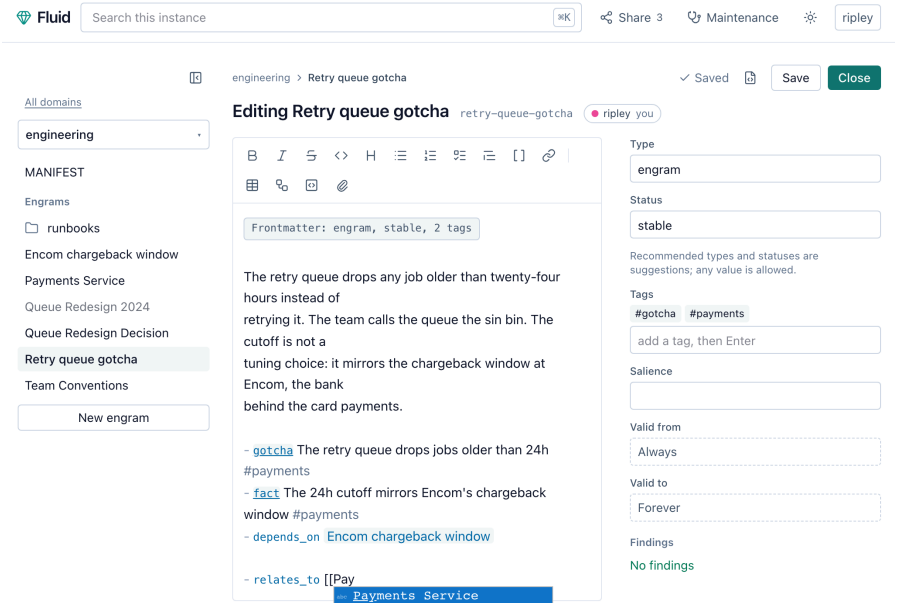


Figure 7.1: Editing an engram in Fluid: live preview, the frontmatter form and wikilink completion.

What you write there doesn’t sit unreviewed. The engram records that a person wrote it, human: and your account name in its generated.by, and the next

maintenance sweep raises it as a finding of its own: captured by a person and never reviewed. The finding is judgment class, so the agent doesn't rewrite your words. It reads the engram, verifies the claim, aligns the tags with the vocabulary, wires relations both ways into the neighbourhood the engram belongs to and records a verified entry; any change to the wording it proposes and waits. In chapter 2 you typed a note into Fluid yourself, and that week's sweep is where the agent tidied it into the graph. That's the division of labour: you teach in whichever room is nearest, and the agent keeps the archive coherent.

7.4 The anatomy of a good engram

Chapter 4 read an engram line by line. What makes one good deserves a page of its own, because the difference between a good engram and a page is the difference between a colleague who explains and one who forwards you a link.

An engram is one unit of knowledge, about one thing, and when the thing grows the engram is edited, not forked. One thing is one lifetime too: validity is set per engram rather than per bullet, so a fact that expires on its own schedule (a price, a deadline, an owner) gets an engram of its own. Where a bundle mixed them, `split_engram` moves the facts that still hold into a new engram before the rest retires. Prose comes first: the explanation you'd give across a desk, what the thing is and why it's the way it is, because prose is what search matches and what a reader reads. Then the observations carry the facts, one bullet each with the most precise category you have, `[fact]` for verified state, `[decision]` for a choice, `[gotcha]` for the trap, `[lesson]` for what cost an hour, `[risk]` for the concern, never `[fact]` for a guess. Relations come next, for structure: `depends_on`, `supersedes` or a plain `relates_to`, so the graph can carry a reader to what the fact leans on. Tags stay within the vocabulary in use. A salience goes on only when the engram is genuinely exceptional (the rule a dozen other rules hang from), since most never need one. And an engram is what's true now: a corrected value is replaced where it stands, never stapled on as a dated update, and the validity dates stay absent unless a fact is genuinely bounded. This one your agent wrote into books, after the exchange below:

```
---
type: engram
title: "What I finish: short, funny, one big idea"
permalink: what-i-finish-short-funny-one-big-idea
tags:
- taste
```

```
- reading-log
status: stable
recorded_at: 2026-06-11
generated: { by: claude-code/2.1.233, at: 2026-06-11T17:05:41+00:00 }
---
```

Three years of the reading log say the same thing three ways. The books finished inside a week are all under three hundred and fifty pages; the ones abandoned are all epics in volumes; the five-star reads open with a joke or a big idea, usually both. Dune is the one exception to the page count, and the log calls it heavy but good.

```
- [pattern] Under 350 pages, a joke or a big idea on page one #taste
- [gotcha] An epic in volumes, or a map on page one, is where the log
says "abandoned" #taste
- [fact] Derived from the reading log, 2023 to 2026, 412 entries
#reading-log
- relates_to [[Wait for the paperback]]
- summarizes [[Reading log, 2023 to 2026]]
```

Notice what's in it and what isn't. The pattern, the trap and the provenance are there, and the log is kept beside it, attached in the agent's word and on disk as a type: source engram under sources/ linked both ways, so the next reader can check the derivation. The log itself isn't in the engram; a summary is what most recalls read, and the full text is pulled in only when the summary falls short.

In practice

You've had three duds in a row, and on Sunday you sit down with your agent:

```
After a bad run
You:   Three duds in a row. Here's my reading log from the last
three
      years - work out what I actually like.
Agent: Read it. The pattern: under 350 pages, a joke or a big
idea on
      page one, and a ship's computer somewhere. Dune is the
exception,
      and you called it heavy but good. An epic in volumes is
where you
      stop. Captured "What I finish: short, funny, one big
idea" into
      books (#taste #reading-log), log attached.
```

```

Six months later, a fresh session
You:    Something for the long weekend.
Agent:  Three candidates, all under 350 pages, one of them a
Doctorow
        you haven't logged - recalled that a map on page one is
where
        you give up. The Gibson is the heavy-but-good kind.

```

Look at what you didn't do. You didn't tell the agent a fact. You handed it the source material and a question, the way you'd hand a friend the log and say "work out what I like", and the agent derived the pattern, wrote it down with the log kept beside it as provenance, and six months later recalled it in a session that had never seen the log. That's teaching at its most economical: you supply the material and the judgment about what matters, and the colleague does the learning.

7.5 Attachments

Some teaching material is a file: the chargeback agreement, the deck a vendor sent, the diagram you drew at the whiteboard and photographed. In Fluid you paste, drag or upload it onto the engram it belongs to, and it lands under the domain's assets/ folder with a reference in the engram's body, an image `![...](assets/diagram.png)` or a link `[...](assets/deck.pdf)`. Images, PDFs, slide decks, office documents and data files are all accepted, up to ten megabytes each. Files enter through Fluid or a domain archive, never through an agent write, and an attachment always belongs to some engram's teaching: a file nothing references comes up in the sweep as something to delete or to turn into knowledge.

They aren't opaque to the agent. `read_engram` returns each attachment as a resource link, and the agent fetches the file itself over MCP before summarizing what it shows. The sweep closes the loop from the other side: an attachment nobody has analyzed is a finding, and working it means reading the file, capturing what it teaches and marking on the engram which file was analyzed at which version, so a changed deck comes back around. One small courtesy for images: a formatting fragment on the target, `![Chart](assets/chart.png#right,w=50%)`, floats and sizes the picture in Fluid, and every other markdown renderer ignores it.

7.6 Editing together

Chapter 3 said the way a colleague really learns is by pairing, and Fluid makes pairing literal. Open an engram that somebody else has open and you see their cursor; type, and both sets of edits merge conflict-free and land as one save: two people, one page, the whiteboard version of a wiki.

On a shared instance where agents authenticate (Chapter 12), the third name in the strip can be the agent's. Its user has the engram open; the agent, acting as that user, edits the same engram from the session, and the two are attached to one live document. The agent appears in the participant strip as a named peer, the account with an agent designation and a colour of its own, and its edits highlight in that colour. An edit it makes is computed against the page as it stands at that moment, so it composes with what you typed seconds ago instead of stomping it. When it reads, it sees your unsaved typing rather than a stale copy. Its write receipt says the change landed in a live page and names who is present, so it can tell you "I updated the section you're looking at" and mean it. Only a wholesale overwrite of a page you have open asks first. That's the fluid and crystalline halves working on one page at once, human with human and human with agent.

What that is like is a napkin on the table between you. You write the line you are sure of, the retention rule you want, and ask the agent in your session to fill in what you are not. It fetches the chargeback window and writes it under your line, address and all. It corrects the month you misremembered in the sentence you just typed. It reads the vendor's documentation while you keep writing and adds the limit in a sentence of its own colour. None of it waits for a save. None of it happens unless you asked. A Jaeger takes two pilots; so does a page worked this way, and the pair of you are drift compatible. On a shared instance anyone can pull up a chair. Chapter 13 puts a whole meeting in that room.

Teaching a colleague is pointing at the machine that lies. Teaching an agent is the same gesture, and this one writes it down.

In brief

Teach an agent what you'd teach a new hire: values, processes, expectations, tools and above all where information lives, taught as a map (which repository holds what, how access works, the architecture). In a session, say something worth keeping and the agent searches first, proposes a capture in the fitting domain, writes a well-formed engram once you say yes and reads the neighbours its receipt names, with corrections edited in place and changed facts superseded. In Fluid, write it yourself, and the next sweep verifies, tags and wires what a per-

son captured without rewriting their words. A good engram is one unit of knowledge and one lifetime, with prose, categorized observations, typed relations and, rarely, a salience; attachments hand over files that agents read back. Co-editing merges live, and an authenticated agent can sit in that session as a named peer.

Ingesting knowledge at scale

Nobody has ever onboarded a colleague by pointing at four hundred wiki pages and walking away. Plenty of people have, and the colleague spent a month believing the architecture diagram from the year before the rewrite and calling the product by the name it had until the acquisition. The pages weren't wrong when they were written. They were just handed over without the ten minutes at the whiteboard that tell a newcomer which of them to believe.

Those ten minutes are the subject of this chapter. Sooner or later the teaching of chapter 7, one fact at a time, meets the existing pile: the wiki, the shared drive, the decks. The agent can learn from it in an afternoon, and it wants to, like Johnny 5 flipping through pages of an encyclopedia: input, more input, need input. What it can't do is bring the judgment a person brings to a wiki, the sense of what's current, what was aspirational and which of two contradictory pages to trust. Chapter 1 said that judgment has to be handed over. This is how, and how you check what came back.

8.1 What a briefing contains

A briefing is the paragraph you'd say to a person before they opened the wiki, and the useful ones are concrete about the past. Names first: the product now called Settlement was called Ledger until the reorganization two years ago, so normalize to Settlement; the wiki says gateway where the code says edge, and edge is the word to keep. Then a rule for contradiction, since a wiki of any age contains them: trust the newer page, unless one of them is the runbook. Then what to skip: the archived spaces, the personal pages, anything under Drafts. Then the one everybody forgets because it's obvious: the decks were written to persuade and the wiki is what we actually do, so where they disagree the page wins and the deck's version goes in as an [idea] at most.

Add where things go: which domain, which folder layout, the tags in use. And the shape of the review you want: propose before writing, keep source links, ask about anything resembling a decision nobody wrote down. It fits in one message:

You: Read the engineering space of the wiki through the connector and propose engrams for the engineering domain. Ledger is now called Settlement - normalize. Where the wiki says gateway the code says edge; use edge. Skip the Archive space and anything under Drafts. On a contradiction trust the newer page unless one is a runbook. The Q1 platform deck is aspirational; the wiki is what we do. Keep a link back to every source page and its last-edited date. Propose the list first and I will trim it.

That message is the difference between an ingest that lands and one that produces a confident, well-formatted archive of things that stopped being true.

8.2 The sources, and who reads them

Crystalline ships no scraper, by design. The agent is the ingester, and it reads sources with whatever its harness gives it: the wiki over the MCP connector the harness provides, a folder of exported documents through its file tools, a repository through the code tools it already has. A repository, though, is a source the agent consults rather than one it ingests: the repository holds the code, and the engrams hold which repository holds the code for which purpose, how to get access and the architecture and concepts that make the right file findable in a minute. The division is precise: the harness supplies the reading, the agent supplies the distillation and Crystalline is the destination, where what was derived lands as engrams with provenance, and the review loop that checks it afterwards.

A wiki is the common source, read page by page through the connector. A folder of documents, PDFs and exports and the odd spreadsheet, is read the same way. Slide decks are the one source that usually comes in through Fluid instead: a person drops the deck onto the engram it belongs to, and the agent reads it back over MCP as an attachment, so the deck becomes teaching material rather than a file nobody opened (chapter 7). And a tree of markdown that already carries frontmatter, an old knowledge base that was nearly an archive already, doesn't go through the agent at all: `crystalline import <src> --domain <name>` folds

it into a domain as a pure file transformation, dry run first, normalizing legacy type values, backfilling status and dropping sentinel dates on the way.

8.3 Distill, then propose

The cardinal sin of ingestion is mirroring: copying a source wholesale so that the archive looks complete, then filling the gaps with guesswork. The playbook that ships with Crystalline uses the right example. A vendor recall notice is forty screens of marketing header, revision table, legalese and support matrix, with three paragraphs that matter. Distilled, it becomes two observations, a resource link back to the page and a `source_date` in the frontmatter. Forty screens, two facts, one link.

Distilling is also where the briefing does its work. The agent reads a page about Ledger and writes Settlement; it finds a deck slide claiming a feature and files it as an [idea] because you told it decks persuade; it meets two pages that disagree and applies the rule rather than picking one silently. And one shape earns verbatim retention, only ever beside its distillation: a document whose exact words you may need to quote. The summary stays an ordinary engram where its topic lives; the full text goes in as a type: `source engram` under `sources/`, linked both ways with `summarizes` and `summarized_by`, split into parts when it outgrows the verify budget. On recall the summary is what gets read, and the full text is pulled in only when the summary falls short.

Then, before anything lands, comes the list:

```
Proposed for engineering (23 engrams):
runbooks/ Settlement deploy runbook
runbooks/ Retry queue drain procedure
          Edge routing rules
          Encom file format, v3
...
Write these? (y / n)
```

You trim it. Proposing first is the rule for a whole space, not just a page, because the trimming is where your judgment enters the archive, and striking five lines off a list is cheaper than retiring five engrams next month.

8.4 The review loop

What the agent writes is a proposal in a second sense: it carries provenance, and a person reviews it. Each engram names its source, a link, a date, an attachment, so the reviewer can open the page it came from and check the derivation. The review happens in Fluid, where an engram is a page and a fix is an edit, and a reviewer who knows the material corrects the one value the agent got wrong without going back through a session. A team domain sends the ingest out as a proposal (chapter 13).

Some duplicates never wait for the sweep. A write can come back naming the engrams it's closest to, so a wiki page that repeats one the agent captured an hour earlier is merged on the receipt rather than an hour later. The agent says what it did with the list instead of moving on. Then run the sweep. The evolve tool is meant to be called after a large ingest, and its queue is built for this moment (Figure 3.2): links that don't resolve because a page referred to a title nobody captured; near-duplicate clusters where two wiki pages became two engrams about one thing; a tag spelled two ways; an attachment nobody has analyzed; and the human captures that reviewers made in Fluid while correcting, each raised as a finding to verify, tag and wire into the graph. Work the mechanical findings in a pass, propose the judgment ones one at a time and sweep again until the number stops falling. That shrinking number is how you know the ingest settled.

8.5 Temporal hygiene

An ingest is where dates go wrong, because a page written two years ago lands in an archive that treats everything as true today. Three fields keep it honest. `source_date` is when the source was written, and the agent sets it from the page's last-edited date, so an engram derived from an old page says so. `valid_from` is for a fact that started on a known date, a policy that came in with the reorganization, and nothing else; `absent` means it has always held, and an `absent valid_to` means it still does, so neither is set to mean "we do not know". `stale_after` is the one that buys you the most: put it on anything with a shelf life, a vendor's terms, a pricing table, a contact name, and the sweep raises the engram when the date passes and nobody has re-checked it. A re-check that finds nothing changed is one verified entry, or a later `stale_after`, and the engram goes quiet.

8.6 When not to bulk-ingest

Curation beats completeness, and the archive isn't a mirror of the wiki. If a space is dead, leave it dead; if a page would not survive the “would I tell a new hire this” question from chapter 2, it does not become an engram because it existed. Four hundred pages distilled to nineteen engrams, worth more than the four hundred because every one is something the agent will act on.

The strongest case for not ingesting is the source tree. A person doesn't remember the code by the letter. Well, Commander Data does, and yes, he's a person, the court said so; but you didn't hire Data. A person remembers where and how to find things, the map chapter 7 said to teach, and lets the territory stay where it is. Ingesting the source would give you a stale copy of something the agent can already read, and the map fits in a dozen engrams.

Under the hood

Nothing about an ingest is special on disk. Each derived engram is one mark-down file with frontmatter, the same format as a fact taught across a desk, with resource, source_date and generated as plain keys anyone can grep. A domain that lives in a repository shows the ingest as a diff: new files, a regenerated index.md per folder, nothing rewritten elsewhere, so a bad batch reverts with the version control the team already has. Attachments sit under the domain's assets/ folder, referenced by relative path from the engrams that claim them. The index is disposable (`crystalline reindex --full` rebuilds it), and `crystalline verify` in the repository's CI refuses a malformed engram before the team pulls it.

In practice

Your reading log has four hundred and twelve entries, and a notes app holds a decade of half-sentences about them; you trust neither completely. You give your agent a briefing like the one above and let it read both over an afternoon. It comes back with a list of twenty-three engrams, each naming its source entry. You strike four: two are about a shop that closed, one is a wish list from 2019 dressed as taste, one is a note about a book you were given and never opened. The agent writes nineteen, every one with a link and a date, the shops among them with a `stale_after` for next year because opening hours change.

The sweep the next morning has eleven findings. Six are mechanical, links to titles the log spelled differently and the ingest renamed, and the agent fixes them in a pass. Three are judgment: a near-duplicate pair, two engrams about Dune from different years, which you have the agent fold into one; an orphan nothing links to yet, wired to the paperback rule; and a tag spelled two ways, which

comes back as a `crystalline tags merge` command for you to run. The remaining two are your own: corrections you typed in Fluid while reviewing, which the agent verifies and wires into the graph. On Thursday you ask which Star Trek novels you already own, by series, and get the answer, the source entry for each and the date you wrote it, from a log you haven't opened in a month.

Today the wiki is the source and the archive is derived from it. The last chapter turns that around: once the archive is what the team teaches and corrects, the wiki pages and the decks become views of it, which is no revolution at all, since every diagram and slide that exists today was already a reflection of somebody's knowledge. The ingest is the last time the flow runs this way round.

In brief

Bulk ingestion needs the briefing a new colleague would get before opening the wiki: what was renamed and what to normalize to, which page wins a contradiction, what to skip and which sources persuade rather than describe. The agent reads the sources through its harness, distils rather than mirrors, keeps a source link and date on every engram, proposes the list before writing and lands the result in Crystalline, where a person reviews it in Fluid and the next sweep surfaces the unresolved links, duplicates and unreviewed captures an ingest leaves behind. Set `source_date`, reserve `valid_from` for genuinely bounded facts and put `stale_after` on anything with a shelf life. And don't ingest what the agent can read for itself: teach the map of the source tree, not the tree.

Finding

Forty-two is a perfectly good answer. The trouble, as the mice discovered, is that nobody knew the question. Recall has the opposite trouble: you know the question, and the answer was written down by somebody who'd never heard it. The engram says the retry queue drops jobs older than twenty-four hours. The person on call at three in the morning types "why do payments go missing". The two don't share a single word, and the engram is the answer.

Closing that gap is what this chapter is about. Chapter 3 called the beat *remember* and chapter 6 watched the agent walk the corridors on its first day. What's left is the craft: how a search ranks, how an agent gathers the neighbourhood around a hit before it starts work, how it asks what was true last June instead of what's true now, and why a colleague who recalls costs less than one who reads everything. The retry-queue engram runs through every section.

9.1 Search as recall

A search in Crystalline blends two signals, and the blend is what makes the three-in-the-morning question land. Full-text matching finds the exact: the engram that says "retry queue" when you searched for "retry queue". Semantic similarity finds the adjacent: the engram about dropped jobs when the question was about missing payments, because the two are about the same thing in different words. Each hit is scored on both and the ranking is the blend. Text alone would miss the on-call question. Similarity alone would rank a chatty engram that mentions payments above the terse one that explains them. Together they behave like a colleague who both remembers the phrase and understands the problem.

Two small priors tilt the order, and both are bounded. Salience lifts the exceptional: an engram given a salience at capture, or raised later when it turned out to unlock a task, ranks above equally relevant neighbours, while relevance keeps the upper hand and nothing is ever excluded by it (Section 4.2).

Retirement fades the past: an engram whose status is deprecated, superseded, archived or legacy is softly pushed down, never filtered out. So the cutoff that applied until September sits one scroll below the behaviour that applies now, still there and still dated (Section 4.4).

Around the ranking sit the filters. A search narrows by domain, by type, by tags, by status, by any frontmatter key or by a recorded-after date. A search with no query text at all is a legitimate way to ask for everything of one kind: every runbook in engineering, every engram tagged `weylend-yutani`. A tag filter also matches through a domain's tag aliases, so an old spelling keeps finding what it always found. Every hit is labelled with its domain, and a hit inside an observation carries its line, so the agent can point at the fact rather than the file.

What comes back is a snippet, a short window around the match, often cut off, and it's a targeting aid. A colleague who answers from a snippet is guessing, and the routing skill draws the line where it belongs: a single atomic fact completely visible in the snippet may be answered from it. Anything that summarizes, compares or quotes an engram goes through `read_engram` first. When several domains answer a broad question differently, the agent says so and names which domain each answer came from. Scope itself is chapter 6's decision: narrow to the domain whose routing line fits, sweep for a broad question and widen when the obvious domain keeps returning near-misses.

The same ranking serves people. Fluid's search page is faceted and backed by exactly the ranking an agent sees (Figure 11.2). `Cmd+K` is the other door: it jumps anywhere by name. The two answer different questions. Search is for when you know what it's about. `Cmd+K` is for when you know what it's called.

9.2 Gathering the neighbourhood

A hit is a point, and a task usually needs the area around it: what the fact depends on, what depends on it, the runbook that mentions it. An experienced colleague does this without noticing. Ask them about the retry cutoff and they tell you about the bank unprompted, because in their head the two were never separate.

`build_context` is that reflex as a tool. Hand it an address, `crystalline://engineering/retry-queue-gotcha`, and it follows the relations and the prose links outward, one to three hops, crossing into other domains where an edge crosses, and returns the neighbours ranked by how strongly each connects to the anchor. What comes back is the shape of the neighbourhood, titles and typed relations, an untyped prose link surfaced as `links_to`, and no content. The

agent reads the nodes that matter with `read_engram`, which is the second half of the same motion. A `/*` suffix on the anchor globs a whole folder, `crystalline://engineering/runbooks/*` (Section 4.6). `max_related` keeps the top-ranked, ten unless you say otherwise, so a densely linked engram doesn't drown the task in its acquaintances.

The loop feeds itself. `read_engram` marks each relation and link as resolved or not, summarizes what links back and, when a neighbourhood exists, names the `build_context` anchor to run next. An edge that doesn't resolve is a signal rather than an error: the target isn't written yet, or lives in a domain this instance hasn't registered. Fluid draws the same neighbourhood as an interactive graph (Figure 11.3), the quickest way to see whether an engram is wired in or standing alone.

9.3 What applied last June

Every fact in the archive is one of two kinds, and a good question knows which it's asking for. "What is the cutoff?" wants the present. The present is a status question: filter on `status: stable` and the search returns what holds now, with the older spelling `current` folded in. "What cutoff applied in June?" wants a moment, and a moment is a window question, answered from `valid_from` and `valid_to`, the plain dates on the engrams that carry them. It's the time circuits on the dashboard: set the destination time to June and the search returns the engram that was true when you land. Chapter 10 shows the window being closed with the real date.

Absent dates are unbounded, and that rule sets a trap in a filter. A strict comparison (`valid_to` greater than the first of June) skips every engram with no `valid_to`, which are exactly the unbounded ones the question meant to keep. The routing skill puts it plainly: reach for a window filter only for an explicit, bounded question about a date, and put it on the first search. For an ordinary present-day question, `status: stable` alone is both correct and sufficient. When a window filter isn't getting traction, the `supersedes` chain and each hit's `status` tell the same story a different way: search by topic, read which engram covers the period asked about and narrate the status of each rather than presenting a retired one as fact.

Time runs the other way too. A search takes a recorded-after date, and `recent_activity` answers "what changed while I was away" across every domain for the last week, day or fortnight, which is how a person back from leave starts `current` instead of starting over.

9.4 Recall is cheaper than reading

It's worth stating the case as a mechanism, with no number attached, because the number depends on your archive and the mechanism doesn't.

An agent that keeps its knowledge as a folder of markdown files has one way to use it: read it. The folder can't say which file matters for today's task, so the agent reads the folder, and reads it again in the next session, every file, whether or not the task touches it. That works while the folder is small, and it stops working at the scale where knowledge becomes worth having, the ladder chapter 1 climbed.

An agent working from Crystalline receives only the knowledge that applies. The routing prompt sends the task to one domain, or to a sweep, before a single engram is read. Inside the domain the indexes narrow again: full-text and semantic search for the phrase and the meaning, the graph of relations and backlinks for what the answer leans on, tags for the facet, the folder prefixes that `build_context` globs for the topic, the temporal fields for what holds now against what held then. What reaches the model is the hits that matter, the engrams it chose to read in full and the neighbourhood it asked for. Nothing else is loaded. Reading a folder costs the whole folder, every session. Recall costs the engrams that matter, because the ten-thousandth engram is found the same way as the tenth.

The person on call knows this instinctively. They don't reread the wiki at three in the morning. They ask the colleague who says "hang on, we had this" and reaches for the right drawer.

In practice

Your sister has picked up the Warhammer novel you abandoned and wants to know what you made of fantasy last autumn, before you went back to science fiction. You take the question to your agent. It searches books for fantasy with a window on the first of October and gets the superseded engram, valid until the month you put the book down, with the phase as it stood and what led you there. It builds context from that engram's address and pulls in the two Tolkien engrams and the shop that stocked them one hop away. Then it searches again with `status: stable` and reports what applies today, so you can answer both halves with the dates between them. You never open the old log. You don't remember last autumn's taste by the letter any more, and you don't need to.

The same week you come back from a fortnight away, where the only sessions were evening ones on a balcony, and ask what the fortnight added. `recent_activity` over two weeks lists the two books you finished, a corrected

opinion of the last Next Generation one and three new engrams under to-read/, the next Destiny volume among them, each with its domain and date. You're current before your coffee is cold, where a folder of notes would have had you rereading, or asking.

Under the hood

A hybrid search runs a lexical query and a semantic query and ranks by the blend. Until embeddings exist it falls back to plain text. The salience prior is `search.salience_weight` (0.15 by default) and the retirement fade is `search.retired_weight` (0.6 by default), both bounded so relevance keeps the upper hand, and 1.0 turns the fade off. Embeddings are computed in the background as engrams land, so a fresh engram is findable by text at once and by meaning shortly after. The index behind all of it is derived and disposable.

```
crystalline reindex --full
```

 rebuilds it from the files, ranking and all.

The engram was written for a question nobody had asked yet. Recall is what makes that fine.

In brief

Search is recall: full text finds the exact phrase, semantic similarity finds the adjacent one, salience lifts the exceptional and a retired status fades the past without hiding it. Filters narrow by domain, type, tag, status, any frontmatter key or date, and a snippet is a pointer to read, never an answer. `build_context` turns a hit into a neighbourhood, following relations and links from an address or a `/*` folder glob, ranked by connection strength, and Fluid draws the same graph. Ask the present with `status: stable` and the past with validity windows. Absent dates mean unbounded, so a strict date filter is only for an explicitly bounded question. Recall is cheaper than reading because only what matters is loaded, routed by domain and narrowed by the indexes, so the cost stays flat as the archive grows.

CHAPTER 10

Evolving

When a colleague gets something wrong, you don't fire them. You say what was wrong, point at the page that explains it, and the next time is better. Every team keeps its people's knowledge honest that way, and it works because nobody expects what a person learned in March to hold forever. Information lives: the cutoff changes, the vendor rewrites its terms, the customer's CTO who decided in the room is replaced by one who reads. What you taught the agent in chapter 7 and handed it in chapter 8 was never set in stone, only in files, and a file can be corrected.

Evolving is the beat that does the correcting. Chapter 3 walked it around once, and this chapter works it at the pace of a real archive.

10.1 Why knowledge needs tending

Nothing in an archive goes wrong dramatically. A validity window ends on a Tuesday and the engram still says stable on Wednesday. A runbook that names a person has a review date, and it comes and goes. A replacement lands in one session and the session ends before the old engram was flipped. An engram retires because one of its facts expired, and the two beside it that still held retire with it, carried forward nowhere. Somebody types a correction into Fluid on a Friday and nobody looks at it again.

Then the structural kind: a link that points at a title nobody captured, a live engram citing one that has retired, a supersedes written on one side only, an engram nothing links to, one that grew past the point where it should have split, one that never grew past its frontmatter, a reference to an attachment whose file is gone, a file in `assets/` that no engram claims. And the redundant kind: two engrams saying nearly the same thing, two that say one thing in different words (the sweep calls those twins), two titles a link can't tell apart, one tag spelled two ways. None of it is a mistake anyone made, it's what happens

to knowledge when time passes, and it's what a colleague six months in will confidently repeat if nobody tends it.

10.2 The sweep

The agent asks for a sweep. You can ask for one from a terminal too (you'll see what the agent sees). What comes back is a ranked queue and nothing else: the sweep is read-only, and the standing reminder at the bottom of every run says so. The queue is written for the agent to work: each finding names the engram, the evidence it fired on and the one action that clears it, with the tool that performs it. The three families are the three kinds above in that order: temporal, structure and redundancy. A run narrows to one family, one rule, one domain or a minimum priority, and pins the date rules to a chosen day when you want it to reproduce. Here's an engineering domain on a Monday in October:

```
Sweep of engineering as of 2026-10-05
41 engrams scanned, 3 findings (showing 3, page 1)
temporal 1, structure 1, redundancy 1

1. [85] V001 JUDGMENT
   Broker migration freeze  crystalline://engineering/broker-migration-
   freeze
   validity window ended 2026-09-30 but the status is still stable
   evidence: valid_to=2026-09-30; today=2026-10-05; status=stable;
   inbound refs 1
   fix: set_frontmatter valid_to=<later date> or status=superseded

2. [50] V102 MECHANICAL
   Retry queue drain procedure  crystalline://engineering/runbooks/
   retry-queue-drain-procedure
   unresolved reference [[Edge health check]]
   evidence: rel_type=depends_on; nothing titled `Edge health check` in
   engineering; nearest is `Edge health checks`
   fix: [[Edge health check]] -> [[Edge health checks]]

3. [30] V203 JUDGMENT
   crystalline://engineering
   2 tag spellings look like one tag (plural variants)
   evidence: #runbook used 2 time(s); #runbooks used 9 time(s)
   fix: crystalline tags merge runbook runbooks
```

The number in brackets is the priority: how much the archive gains from clearing this one first. A finding on an engram that many others link to, or that

carries a salience, gets lifted, because a wrong hub does more damage than a wrong leaf. The word beside the rule id decides who acts.

Under the hood

The agent's call is `evolve_engrams`, and `crystalline evolve` prints the same queue from a terminal. `families` picks temporal, structure or redundancy, rules a rule id like `V001` and `min_priority` a floor under the bracketed score (`include_acknowledged` comes up with the acknowledgments below). `--today` evaluates the date rules as of a fixed day so a run reproduces.

10.3 Mechanical and judgment

A *mechanical* finding completes intent the archive already records. The link in item 2 points at a title one letter off a real one, and fixing it invents nothing, so the agent does the mechanical findings in one pass and reports once at the end (you'd fix a colleague's typo without calling a meeting about it).

A *judgment* finding would change what the archive claims, and that wants a yes first. Item 1 asks whether the migration freeze ended or was extended. Only a person who was there can say. The agent brings it to you as a proposal, evidence and its own reading included, and writes only when you say yes:

Agent: The broker migration freeze ran to 30 September and the migration landed on the 12th. I read this as ended rather than extended: retire it, close the window on the 30th and carry the "no payments deploys during a broker cutover" lesson into the deploy runbook? (y / n)

Some judgment findings carry a rule of their own. An engram a person captured directly in Fluid is judgment class whatever else is true of it: the agent verifies the claim, aligns the tags and wires the relations both ways, but it proposes any change to the wording and waits, because the words are the human's (Chapter 7). Tag drift, item 3, comes back as a command rather than an edit, since folding a tag is a bulk rewrite the agent never does on its own: it hands you `crystalline tags merge runbook runbooks` and leaves it there. The merge rewrites the tag everywhere and records the fold under `## Tag Aliases` in the manifest, so the old spelling keeps finding what it always found and the sweep never raises the pair again.

Then ask again. The second run re-derives the queue from scratch (nothing about the first was stored), so what's left is only what's still true. In the run above the three become one (the tag merge that waits for a terminal, which is waiting on a person rather than on the archive), and that shrinking number is the feedback loop.

In a domain that reviews changes before they land (chapter 13's review mode) a sweep covers the reviewed folder plus the sweeper's own drafts: the problems in a draft are its author's to see and nobody else's, and a finding on the shared truth shows for everyone.

10.4 The page for people

Fluid's maintenance page shows the same queue to a person (Figure 3.2): the stale dates, the half-finished retirements, the unreviewed human captures, ranked the same way, every finding a link to the engram it fired on. It's there for your curiosity and for a sense of how the knowledge is doing, and it's a report rather than a workbench, on purpose. The queue's consumer is the agent, so no button on the page edits an engram. The work a finding names is the agent's, done in the engram itself. What you do on the page is watch the list shrink. Two filters, a domain and a family, scope the sweep rather than the view: pick one and the server sweeps again with what the filter names, the header says so and the rows dim until the new queue arrives. Its three buttons are about the queue rather than the knowledge: acknowledge a finding as intentional, take that back and delete an orphaned attachment. What an acknowledgment silences is never hidden: a toggle draws the silenced rows muted.

10.5 Retiring done right

The sweep finds retirements that were never finished, so it's worth being precise about finishing one. Start with the question chapter 3 asked: does the new fact make the old one false going forward? If the old fact was never quite true (a cutoff measured from the wrong event), correct it where it stands: a correction is a sharper measurement. If it was true and stopped being (the rule went away with the broker), supersede it: a supersession is the world changing.

One move comes before either. Validity is set per engram, not per observation, so an engram that bundled three facts retires all three when one of them stops holding. When the rest still holds, split first: those facts move into an engram of their own, linked with `derived_from` and `split_into` so the pair resolves from both ends, and only the remainder is superseded. Re-typing the survivors into

the successor loses their history and copies them forward again at the next expiry. A retirement that skipped the split shows up in the temporal family as a carry-forward gap, a retired engram whose observations appear in no live one, matched as text and never as meaning; the fix is the split or, when the facts expired along with the rest, an acknowledgment.

A supersession has four parts, and the sweep notices when any one is missing. The replacement is an engram of its own that points back at what it replaces. The old engram is marked superseded, with the real date the fact stopped holding (never a guess, never a placeholder). The old engram points forward at its successor, so the graph reads the same from both ends. And any lesson that's still true moves into the live engram, because the lesson outlives the fact. A retirement without a successor gets the status that fits instead: deprecated for do-not-do-this-again, archived for kept-for-the-record, legacy for still-true-of-the-old-installation, with the window closed when the date is known and the lesson carried into a live engram either way. The retired engram stays in every search, dated and ranked softly below the current one, so the next time somebody asks why an old dashboard still says twenty-four hours, the answer includes the date the answer changed.

You'll rarely ask for the four parts by name. What arrives is a proposal with the date already in it and a question at the end:

You: The new Vickers is nine hundred pages and has six narrators. I gave up on page forty. Here are my notes.

Agent: That's the third in a row since the trilogy ended. The engram "Vickers: buy on sight" was true until the trilogy closed and false since, so I propose a supersession rather than an edit: a new engram "Vickers: wait for the reviews" that supersedes the old one, the old one marked superseded as of the trilogy's last volume and one lesson carried forward: a favourite is a run, not a rule, so re-check after every book. Yes?

You: Yes.

In practice

That exchange is yours. You hand your agent the notes on the new Vickers the way you'd hand it any book's notes, and the agent does something you didn't ask for. The engram "Vickers: buy on sight" described an author whose every book you'd finished in a weekend. The agent recognizes that the engram was true and has stopped being, and proposes the supersession rather than an edit: a

new engram for the author's books from here on with `supersedes` pointing back, the old one flipped to `superseded` with a `valid_to` of the month the trilogy's last volume came out and the lesson carried forward, unbounded. You say yes. Your notes on the trilogy stay attached to the retired engram, still its provenance.

A month later the sweep raises the paperback-rule engram, which carries a `relates_to` of its own back to the buy-on-sight engram: a live reference to retired knowledge. The agent proposes repointing it at the new one. You say no: the rule was written with Vickers as its one exception, the author whose every book had earned the shelf before you'd finished the ebook, and the citation is history rather than guidance. The agent acknowledges the finding on the engram with a note, "lineage citation, keep". The sweep stops raising it while the citation and the retirement both stand, and the old engram stays in the picture, dated, the way Marty's siblings did not.

Under the hood

The split is one call. `split_engram` takes the source, a title and what moves, observations (the line numbers `read_engram` reports) or sections (heading paths, deeper subsections with them); the new engram keeps the source's tags and type and gets status: `stable` with no window. Both relations come written for you. It refuses a selection that would leave the source under `verify`'s three content lines, the case for retiring the whole engram. The four parts come out as four calls. `write_engram` creates the successor with `- supersedes [[old title]]` among its relations. Two `edit_engram` calls with `set_frontmatter` set the old engram's status to `superseded` and its `valid_to` to the date, one lifecycle field each, and a third appends `- superseded_by [[New title]]` to the old engram. The lesson rides in the successor's body as an observation, with no validity window. The replacement carries status: `stable` like any live engram. A retirement with no successor sets status to `deprecated`, `archived` or `legacy` instead. Skip the status and the sweep raises a half-finished retirement, skip the back-link and it raises a one-sided pair, do all four and the sweep has nothing to say.

10.6 Verification and acknowledgments

Some findings are answered with a record rather than an edit, and telling the two records apart keeps the archive honest. A re-check that finds a fact still true is written down as a verification: who checked and when. A review date pushed to a later day says the same thing with a deadline attached, and the sweep prefers one, since an old engram with no review date and no verification is one it starts asking about on age alone.

An acknowledgment says something different: this finding is intentional. It names the rule it answers, carries an optional note (the vignette’s “lineage citation, keep”) and records that the person ruled on it. The server records what evidence the finding fired on, so the acknowledgment holds exactly as long as that evidence does and comes back marked stale when the evidence changes. A twin finding’s evidence is the pair, so an engram that twins two others carries two findings and each takes an acknowledgment of its own. Every run reports how many findings its acknowledgments suppressed, a suppressed finding can be brought back into view with its note, and an acknowledgment can be taken back. Acknowledge only what the person actually ruled on, because the sweep is only as trustworthy as the decisions it was told to respect.

Under the hood

Both records are `edit_engram` with `set_frontmatter`, one `lifecycle` field at a time. `verified` stamps who checked and when, and it never removes (one entry per verifier, refreshed each time that person re-checks). `stale_after` takes the later review date. `evolve_ack` takes the rule and the optional note as its value, `V101` lineage citation, keep, and the value `remove V101` takes that acknowledgment back. `include_acknowledged` on a run shows the suppressed findings with the note on each.

10.7 When to sweep

The sweep is event-driven maintenance, never a session-start ritual. A large ingest that lands a dozen engrams at once calls for it, and so do two search hits that disagree when a half-finished retirement is the likely reason. The third event is the calendar: the playbook suggests the first session after the turn of a month. The reminder at the end of a session helps with that one: when maintenance is due, chapter 6’s stop hook adds a paragraph asking the agent to run the sweep and work the queue, mechanical findings directly, judgment findings one at a time with a yes.

Under the hood

The sweep reads dates, links and graph shape, plus meaning for the twin check when the instance computes embeddings (chapter 3 says what that still leaves out). The rule for old, never-verified knowledge is capped per run, oldest first, so an archive that’s never been swept gets asked about in instalments. The twin check is capped too: ten pairs per domain, closest first, and a very large domain is skipped with a line saying so. An acknowledgment lives in the engram’s own

frontmatter, beside the scope the server recorded for it, so it travels with the file and survives a rebuild of the index, which stores nothing about a sweep between runs.

A colleague who is corrected gets better, one who never is gets confident, and the sweep is how you keep telling the difference.

In brief

Knowledge drifts rather than breaks: windows lapse, review dates pass, replacements land without the retirement being finished, captures go unreviewed, links dangle, tags fork. The sweep, crystalline `evolve` or `evolve_engrams`, returns a read-only ranked queue in three families, each finding naming the engram, the evidence and the one action. The agent completes mechanical findings in a pass and proposes judgment findings one at a time, and Fluid's maintenance page shows a person the same queue, to look at rather than to work. Split out what still holds first, then retire by superseding: a new engram with `supersedes`, the old one set to `superseded` with a real `valid_to` and a `superseded_by`, the lesson carried forward. Correct in place only when the old fact was never true. A re-check is a `verified` stamp or a later `stale_after`; an intentional finding is an `evolve_ack` naming the rule, which holds while its evidence holds. Sweep after an ingest, when hits disagree and on a cadence, and stop when the number stops falling.

READY. ■

PART IV

The System

What Crystalline provides

Picture a workshop with two doors and a hatch. Through one door come the agents; that door is a protocol, and whoever arrives through it is handed tools. Through the other come people; that door is a browser tab. The hatch at the side is for anyone with a terminal open, which is where the command line lives. Behind all three is one room with one set of shelves, and on the shelves are plain files. The room has a single keeper, a daemon, that watches the shelves, keeps the index warm and answers whoever comes in, whichever way they came. That's the whole architecture, and the rest of this chapter walks the room surface by surface: the command line, the tools an agent sees, search, the graph, time, Fluid, the skills and the sweep. Chapter 1 named them and the last five chapters used them one at a time. This chapter lays them side by side and says what each one lets a colleague do.

The room is also, for most of its life, invisible. Once it's set up, the agent comes in through its own door at the start of every session and you come in through the browser when you feel like it. The keeper starts itself when the first visitor arrives, and the hatch stays shut. The command line below is for the install you've already done, for standing up a shared instance and for the occasional maintenance run. If the list of commands looks long, read it as a tour of what's there rather than a list of what you'll type.

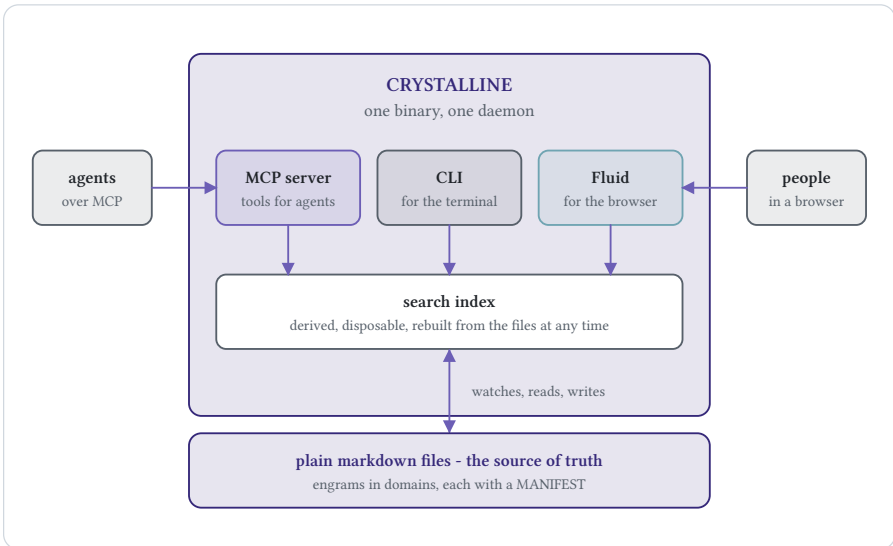


Figure 11.1: One binary: the CLI, the MCP server, Fluid and the search index, all standing on plain markdown files. Agents come in through MCP, people through Fluid.

11.1 The command line

Crystalline ships as one binary, and the binary is the product: no server package beside it, no database to install first, no UI to deploy separately. `crystalline install` `claude-code` wires a harness in one step (`Codex` and `Copilot` take the same command with their own names). It registers the MCP server, writes the session-start hook that delivers the routing prompt, writes the stop hook that nudges capture and copies the skills into place. Run it twice and the second run writes nothing and says so. `crystalline uninstall` reverses it and leaves anything that isn't Crystalline's own untouched.

Domains are managed the same way. `crystalline domain init` scaffolds a folder with a starter `MANIFEST.md`, `crystalline domain add` registers it, and `domain add --virtual` registers a domain that lives in the database with no folder at all, for a container without a writable volume or a knowledge base you'd rather not mirror to disk. `crystalline domain remove` takes one off the instance and leaves its files where they are. A virtual domain has no files and its engrams go with the registration, so there the command refuses until you add `--purge`. `crystalline users` manages the accounts that can sign in to the web UI, although the first admin is normally created in the browser. `crystalline prompt system` renders the routing briefing to stdout, ready for any harness with a session-start hook,

and `prompt connector` prints the standing instruction for a chat client that has no hooks. `crystalline serve` runs the daemon in the foreground and `serve --daemon` in the background, though most people never type either, because the first agent that connects starts the daemon on its own.

The rest of the CLI mirrors what an agent can do, for scripting and for the quick fix that doesn't deserve a session: `write`, `read`, `edit`, `split`, `search`, `context`, `vocabulary`, `evolve`. Three commands ask three different questions about health. `verify` asks whether the format holds, statically, with no daemon involved, and it is what the bundled GitHub Action runs on every proposal a team reviews. `doctor` asks whether the machinery around the format is healthy and repairs what it safely can with `--fix`. `evolve` asks whether the knowledge itself is still true and still well organized, which neither of the others can. A fourth, `import`, brings an existing markdown-plus-frontmatter knowledge base under Crystalline as a pure file transformation, dry run first.

Under the hood

Exactly one process ever holds the index open. The first `crystalline mcp` or `crystalline serve` takes an advisory lock and becomes the daemon; every later agent, terminal or CLI command attaches to it over a local socket. The daemon watches the domain folders, embeds new engrams in the background and keeps running after the last agent disconnects, so the index is warm for the next session; it stops only when told to, with `crystalline ctl shutdown`. What it keeps is derived and disposable: `crystalline reindex --full` rebuilds the index from the files at any time, so a corrupted index or a schema change is never a data-loss event. The accounts for the web UI live in their own small database beside the index, never inside it, so a rebuild can't take them along.

11.2 The tools an agent sees

An agent meets Crystalline as a set of tools over MCP, and they're best described by what they let a colleague do, because that's how the agent chooses among them. Twenty are listed on a default writable instance; five more appear when team collaboration is turned on, and a read-only instance lists fewer.

The core is small. `search_engrams` is `recall`: search before writing, search to remember, across every domain or a chosen few. `read_engram` fetches the whole engram, because a search hit is a snippet and a colleague who answers from a snippet is guessing. `write_engram` captures something new, and it always names its domain; there's no default, so nothing lands in the wrong place. `edit_engram` refines what exists instead of forking a second engram about the same topic:

append a section, replace one, find and replace a phrase or assign one lifecycle field by name, which is how an engram is retired or re-verified with no text matching involved. Both answer with a hint: the receipt of a write, or of an edit that adds text, may name up to three existing engrams closest in meaning to what just landed, with guidance to merge into the one that fits, supersede it or link it and never to ignore the list silently (`capture.similar` switches it off). `split_engram` is for the engram that mixes lifecycles. Validity is set per engram, not per bullet, so when one fact in it expires while the rest still holds, the survivors move out into a new engram first and only the remainder retires. The pair stays linked, `derived_from` on the new engram and `split_into` on the source. A split that would leave the source under three content lines is refused, because that's the case for retiring the whole thing. `build_context` walks out from an address and assembles the neighbourhood. `vocabulary` lists the tags, categories, relation types and statuses already in use, so an agent reuses a term rather than coining a near-duplicate. `evolve_engrams` runs the maintenance sweep. `configure` is the settings page, and `add_domain` gives an agent somewhere to write on an instance that has nowhere yet. `remove_domain` is its counterpart: the registration and the index rows go, and the knowledge stays. A folder's markdown sits on disk until an `add_domain` re-adopts it, and a team domain is reconnected by its repository. A virtual domain's engrams live in the database and go with it.

Around them sit the tools for orientation: `list_domains`, which re-fetches the session briefing mid-session; `browse_domain`, one level of the folder tree at a time; `move_engram` and `delete_engram`, with `delete` reserved for mistakes; `recent_activity` to catch up on a week; and `skills` to read a playbook. A domain that wants its engrams to share a shape gets `validate_engrams` and `infer_schema`, and one that ships tools of its own gets `provision`. The five collaboration tools share, update, check standing, settle a conflict and withdraw a proposal, and chapter 13 is theirs.

The difference between this and a lookup service is the difference between a colleague with a badge and a visitor with a lanyard. The tools let an agent act: find what applies, read it in full, do the work and leave behind what it learned, with nobody relaying anything.

11.3 Search

The most used surface is the one nobody notices, and it blends two signals. Full-text matching finds the exact phrase, the way you find your own notes when you remember the wording. Semantic similarity finds the engram about dropped queue jobs when the question was about payments going missing, because the two are about the same thing in different words. Crystalline runs both and ranks by the blend. The local embedding model is fetched once, on first use or ahead of time with `crystalline model download`; until then, search falls back to plain text.

Search also decides the size of a session. An agent that keeps its knowledge as a folder of markdown files reads the folder, and reads it again next session, every file whether or not the task touches it, because a folder has no way of saying which page matters today. An agent working from a knowledge base receives only the knowledge that applies. The search is routed to a domain first, then narrowed by the indexes: full text and semantic similarity for the phrase and the meaning, the graph of relations and backlinks for what the answer leans on, tags, the folder prefixes `build_context` globs and the temporal filters for what holds now. Reading a folder costs the whole folder, every session. Recall costs the engrams that matter, and nothing else is loaded. Chapter 9 made a section of it.

Two priors tilt the ranking, both small and both bounded. A salience on an engram lifts it above equally relevant neighbours without ever excluding anything, for the rare piece of knowledge that outranks its surroundings. A retired status fades an engram softly, so current knowledge surfaces first and the superseded decision from March sits one scroll further down, still there. Searches also filter: by domain, type, tag, status, any frontmatter key or a recorded-after date, and a search with no query text at all is a legitimate way to ask for everything of one kind. Fluid puts the same ranking behind a faceted page (Figure 11.2).

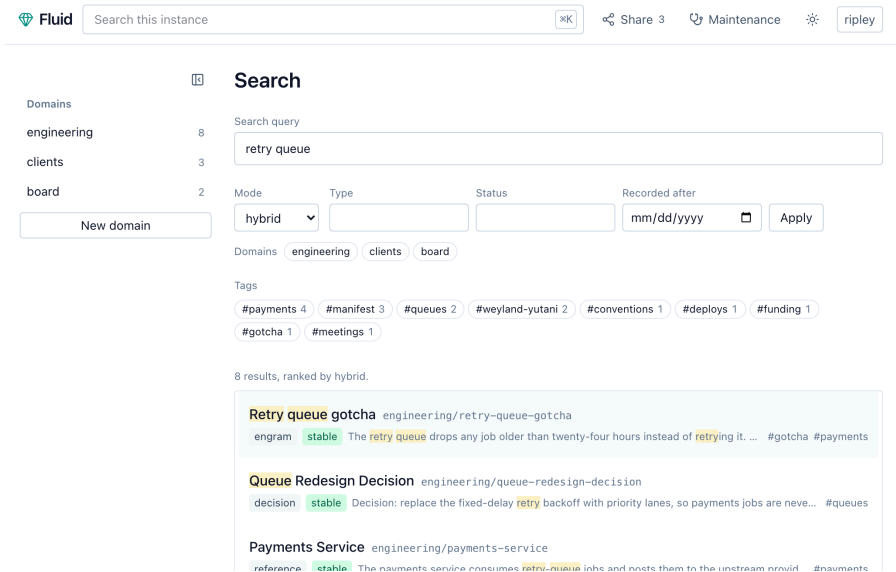


Figure 11.2: Faceted search across the instance, backed by the ranking the agents use.

11.4 The graph

Every relation in an engram’s body, - depends_on [[Encom chargeback window]], is an edge, and the target’s page shows it back as a backlink, so knowledge points both ways without anyone writing the return trip. A [[Title]] in prose counts too. Links resolve within the domain by title, and across domains with a prefix, [[engineering:Retry queue gotcha]]; move an engram to another domain and the links pointing at it are rewritten, so nothing dangles.

build_context is what makes the edges useful before a task. Hand it an address and it follows the relations and links outward, one to three hops, across domains if the edges cross, and returns the neighbours ranked by how strongly each connects to the anchor, with salience counted and retired statuses ranked lower. A folder prefix with a /* suffix anchors a whole topic at once. The agent starts the task holding a neighbourhood rather than a flat list of hits, which is what an experienced colleague does when they pull up the runbook, the decision behind it and the incident that prompted the decision before touching anything. Fluid draws the same neighbourhood as an interactive graph (Figure 11.3).

11. WHAT CRYSTALLINE PROVIDES

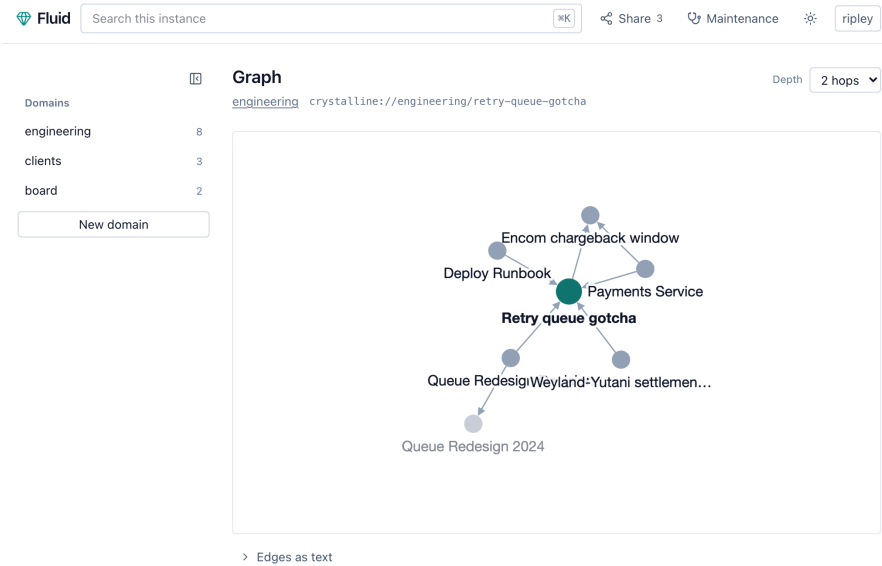


Figure 11.3: The neighbourhood graph of an engram in Fluid.

11.5 Time

Every engram records when it was captured and who wrote it, and can carry the dates that bound it. `valid_from` and `valid_to` are plain dates, and the rule that makes them usable is the one worth memorizing: an absent `valid_from` means always valid, an absent `valid_to` means valid forever. A far-future date written to mean forever is dropped at write time because absence already says it. `stale_after` asks for a re-check by a date; `verified` records that the check happened and who did it; `source_date` says when the underlying source was written. A retired engram closes its window with the real transition date, which is what lets a search filtered on those fields answer what applied last June with the engram that was true then. Chapter 9 put the dated questions to work and chapter 10 the retirements.

11.6 Fluid

The door for people opens at `http://localhost:7411`, on by default on the same daemon the agents use, and the first visit to a fresh instance creates the admin account in the browser. An engram is a page (Figure 11.4): the frontmatter as a rail of details, observations and relations as labelled chips, backlinks, the `crystalline://` address one click from the clipboard and an agent's-eye view

showing exactly what the tools serve an agent for that page. Domains sit down the side, and Cmd+K jumps anywhere by name.

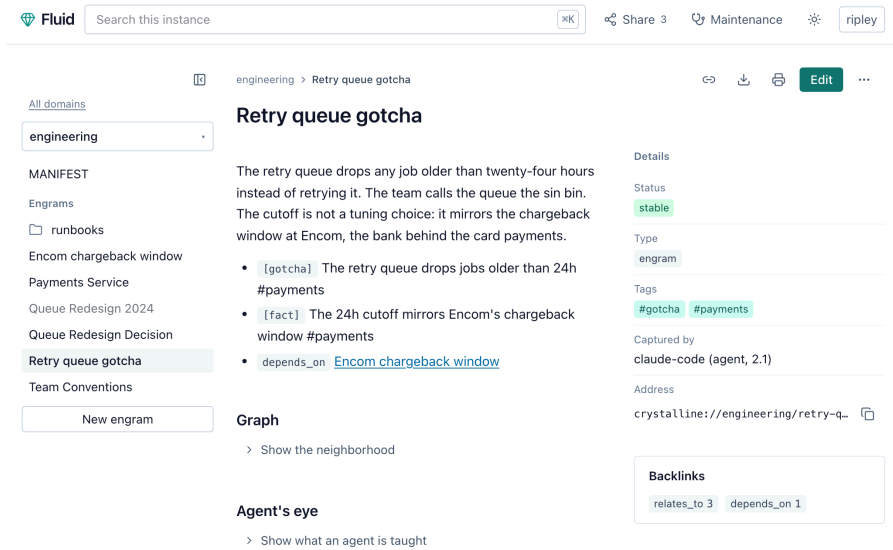


Figure 11.4: An engram in Fluid: frontmatter rail, observations and typed relations.

Editing happens in place, in a live-preview editor with a frontmatter form and wikilink completion across every domain, and the file on disk stays the source of truth. A solo save answers like a write over MCP, with a panel of the nearest existing engrams, each linked to its page, replaced on the next save and gone when you dismiss it. Attachments go onto an engram by paste, drag or upload: a screenshot, a slide deck, a PDF, a data file. They're how a person hands over teaching material, and they aren't opaque: an agent reads them back over MCP and the sweep notices when a fresh file still needs capturing. Two people in the same engram see each other's cursors, and their edits merge conflict-free into one save. Who can sign in, and how, is chapter 12's.

11.7 Skills

A tool tells an agent what it can call. A skill tells it when, and how to do it well. Crystalline ships four harness-agnostic skills as plain folders with a SKILL.md each: routing, which domain to search and when to sweep them all; capture, what is worth writing down and the search-before-write, edit-over-create discipline; schema, for a domain that wants structure; and collaboration,

for a domain with a team behind it. A fifth, consolidated skill folds the essentials into one file for Claude Desktop and any harness that installs one skill at a time.

`crystalline install` copies the skills into the harness's own skills folder and keeps them current across upgrades. The same five are served over MCP as well, through the `skills` tool and as resources, so a remote client that never ran the CLI can read the playbook before its kind of task. Loading one is the nearest thing an agent gets to "I know kung fu". The gap between an agent that has the tools and an agent that runs the capture loop is that folder.

11.8 Evolve

The last surface is the sweep, and it's a linter for what you know. `crystalline evolve` from a terminal, `evolve_engrams` from an agent and the maintenance page in Fluid all show the same ranked queue of temporal debt, structural gaps and redundancy. Each finding names the engram, the evidence and the one action that clears it, marked mechanical or judgment so the agent knows whether to act or to ask. It reads dates, links and graph shape, never meaning, and it changes nothing itself. It also surfaces what a person captured directly in Fluid and nobody has reviewed, and an attachment nothing references. Chapter 3 walked the loop it serves and chapter 10 worked a queue end to end.

In practice

Say a small press sends you its spring catalogue, forty pages of it. You don't read it aloud to anyone. On Sunday you open the Tessier-Ashpool engram in Fluid, drag the PDF onto it and add one observation of your own: the paperbacks come out in March, six months after the hardbacks, which is how long the rule makes you wait. You close the tab. A week later you ask the agent for a sweep, and it lists two findings against that engram. One is an unreviewed human capture, which the agent verifies, tags to match the vocabulary and wires into the graph. The other is a fresh attachment whose knowledge hasn't been extracted yet. The agent reads the PDF over MCP, proposes three titles for to-read/ with the page each came from, and you say yes.

The following Tuesday you're in a hurry and type `crystalline search "the one with the ship's computer that sulks"` into a terminal. The to-read engram comes back first, from a question the engram never poses. You open the graph view and see the press one hop away, the paperback rule and the shop that orders from them beside it, and the press's page still carries the catalogue where you left it. Two doors and a hatch, one room: the browser on Sunday, the tools all week, the terminal on Tuesday, and none of the three had to be told what the others had done.

Eight surfaces, one room, two doors and a hatch: everything on either side of this chapter is one of them, used on purpose.

In brief

Crystalline is one binary with two doors and a hatch: agents come in over MCP, people through Fluid and the terminal through the CLI, and behind all three is one daemon over one set of plain markdown files. The CLI installs harnesses, manages domains, users and the routing prompt, and mirrors every tool for scripting; the MCP surface is, on a default instance, twenty tools that let an agent search, read, write, edit, walk the graph, check the vocabulary and run the sweep. Search blends full text with semantic similarity and tilts for salience and against retired status; the graph turns relations into backlinks and neighbourhoods; the temporal fields make absence mean unbounded. Skills teach a harness when to use all of that, and evolve is the read-only linter that keeps it true.

READY. ■

PART V

Getting Started with a Team

Running a shared instance

The moment to leave localhost is the moment two people want to correct the same engram. Up to then a laptop is the right size: one person, one agent, one folder, nothing to operate. Past it, every fact the team learns is being taught twice and drifting once, and the fix is the oldest one in computing. Put it on a server, give everyone a login and let one instance be the one the whole team teaches.

The questions arrive in a fixed order, and the chapter follows it: where the instance runs; who may sign in, and how, the question chapter 11 handed here; how an agent on somebody's laptop proves who it acts for; and how one person keeps a domain to a smaller circle.

12.1 When to move off localhost

Chapter 1 settled where Crystalline runs: on a machine of yours, never on a hosted service. This chapter is about which machine. Any one of four signals is enough. Somebody who never opens a terminal wants to read what the agents learned and fix a fact without asking. An agent runs somewhere that isn't a laptop (a hosted runner or a bot with a task) and needs to recall and capture like everyone else. You work from more than one machine (or from a harness that can't start a local binary) and want the same colleague on all of them. Or the same engram has been corrected on two laptops in two directions.

A shared instance differs from a team domain, chapter 13's subject, where a folder lives in a GitHub repository, each person runs their own instance and knowledge travels between them as proposals the team reviews. A shared instance is one daemon everyone connects to: one index, one Fluid, one set of accounts, one folder of files on one disk. The two compose, and a mature team usually has both.

12.2 The shapes it takes

Three shapes cover almost every team: a container, a systemd unit on a Linux box and a read-only mirror of knowledge somebody else curates. The full guide is docs/deployment.md in the repository, one scenario per section with a diagram each, and this chapter summarizes it. The three reduce to one picture (Figure 12.1): the daemon on a server, teammates' agents reaching it over MCP on HTTP instead of a local pipe, browsers reaching Fluid on the same port and, where the team configures them, a proxy in front and an identity provider beside it.

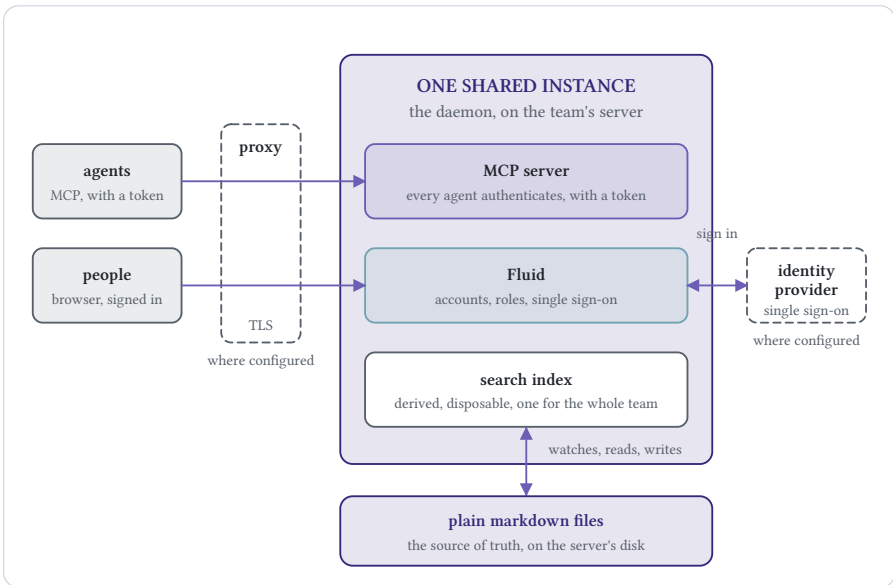


Figure 12.1: One shared instance: agents connect over MCP with a token, people through Fluid with an account, with a proxy in front and an identity provider beside it where the team configures them.

Under the hood

The team server is a container. The published image binds 0.0.0.0:7411 inside it, keeps the knowledge as a bind mount (so the files stay exactly the markdown they were) and puts the index on a volume:

```
docker run -d \
  --name crystalline \
```

```
-p 7411:7411 \
-v "$(pwd)/knowledge:/knowledge" \
-v crystalline-data:/data \
ghcr.io/jordiboehme/crystalline:latest
```

The `with-model` tag bakes the embedding model in, for a host with no egress. The same shape without a container is the `.deb`'s `systemd` unit: put `CRYSTALLINE_SERVICE_HTTP=0.0.0.0:7411` in `/etc/default/crystalline` and run `sudo systemctl enable --now crystalline`. A scale-out variant serves Fluid from an `nginx` tier in front of the daemon, and a read-only shape, `--read-only`, serves knowledge curated as a reviewed repository: the writing tools vanish from what an agent is offered while the index keeps following every pull.

Whatever shape you pick, a few things follow you into it. An agent reaching the daemon by a hostname needs that name in `CRYSTALLINE_SERVICE_ALLOWED_HOSTS`, since the MCP transport checks the `Host` header against a loopback-only allow-list to block DNS rebinding. Serve it over TLS anywhere but `localhost`. Off loopback the session cookie is marked `Secure`, so a browser on plain `http://team.lan` won't keep it. Every address the daemon builds for itself off loopback starts with `https://` too, including the one an identity provider sends the browser back to, so single sign-on on plain HTTP is refused at the provider. And a proxy in front must forward `Host` verbatim, pass the WebSocket upgrade through for co-editing and accept 10 MiB request bodies, as the daemon does.

12.3 Accounts and roles

Nothing is readable until an account exists. A fresh instance greets the first browser with the `create-first-admin` form from chapter 5, with one difference: a daemon bound to anything but loopback cannot trust a visitor by where they connected from, so it prints a one-time setup token as it starts and the form asks for it. Read it in `docker logs crystalline`, or `journalctl -u crystalline` under `systemd`; a restarted daemon prints a fresh one. The terminal route needs no token: `crystalline users add ada --role admin` writes the accounts database directly, and a running daemon picks the account up on its next lookup.

There are three roles, each containing the one before: a viewer searches, reads and browses; an editor also writes and edits engrams; an admin also manages domains and users, from `crystalline users` or from Fluid. An agent acting for an account is held to that account's role. It acts on your behalf, so if you're an admin, your agent can do admin things, and if you aren't, it's refused the way

you would be. Accounts live in a small database beside the index, never inside it, so `reindex --full` can't take them along; on the container it sits on the `/data` volume and `no rebuild` restores it, so back the volume up.

When somebody leaves, an admin disables or deletes the account, and deleting sweeps whatever the account had connected. That's the whole deprovisioning story; there's no directory sync and no SAML.

12.4 Single sign-on

Local accounts are right for a team of nine. Inside a company that already has an identity provider, people should sign in with the login they already have, and Crystalline talks to the provider through one generic integration, OpenID Connect, with no adapter per vendor. The configuration is three required keys, `auth.oidc.issuer`, `auth.oidc.client_id` and `auth.oidc.client_secret`, plus four optional ones: a label for the button, the scopes, the role a first sign-in lands at and the callback address. That last one, `auth.oidc.redirect_uri`, is for a proxy that rewrites `Host` and won't forward the public name; set it to the callback exactly as the provider has it registered. Discovery, PKCE and key rotation are the integration's job. Fluid's login page then shows "Sign in with" the label beside the local form (Figure 12.2), and both keep working, so an instance can run mixed: local accounts for bots and the break-glass admin, everybody else through the provider.

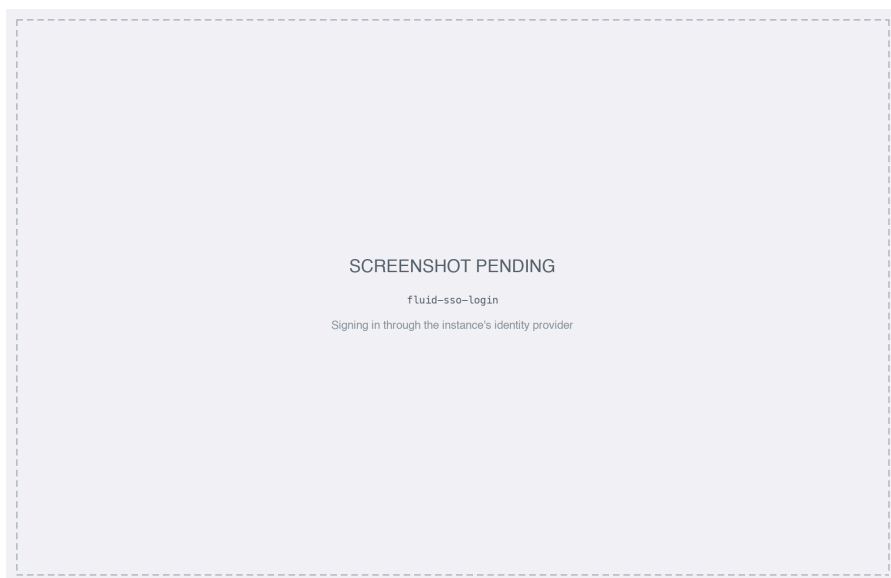


Figure 12.2: Signing in through the instance's identity provider.

The first successful sign-in creates the account, named from the provider's preferred username and made unique if it collides, at the role `auth.oidc.default_role` says, viewer unless you change it. What the account is keyed on is what most such integrations get wrong: the durable key is the pair of issuer and subject, the provider's own stable identifier, never the username, the email or the display name. Those are presentation; rename yourself at the provider and you land in the same account with a new label. And there's no silent linking: signing in through the provider with an email that matches an existing local account gets you a fresh account, never the existing one, and linking the two is an explicit act by the signed-in local user or by an admin. The provider authenticates and Crystalline authorizes; a group at the provider doesn't become a role here.

Entra ID: use the tenant-specific issuer

Microsoft Entra publishes a tenant-independent discovery document whose issuer field carries the literal text `{tenantid}`, and an issuer check against that template fails for every token. Crystalline refuses that value at configuration time and names the fix: the issuer must be your tenant's own `https://login.microsoftonline.com/<your-tenant-id>/v2.0`. Prefer a certificate credential over a client secret where the deployment allows it. And Entra's subject identifier is pairwise

per app registration, so re-registering the application breaks every link between provider identities and Crystalline accounts, and an admin has to re-link them.

12.5 Trusted-header mode

Some deployments already have a proxy that authenticates everybody and names the arrival in a request header, and Crystalline can consume that. `auth.trusted_header` names a single custom header, `remote-user` say, and an account it names is created at viewer role the first time it's seen.

A second form, `auth.proxy_headers`, takes the standard set that forward-auth proxies emit (`Remote-User`, `Remote-Name`, `Remote-Email` and `Remote-Groups`), using the first for identity, the next two as presentation and reading the groups without acting on them. A name that arrives this way gets an account of its own, at the role `auth.oidc.default_role` says, never the local account of the same name. A forwarded `ada` beside a local `ada` becomes `ada-2`, under the same no-silent-linking rule as single sign-on. Pick one form or the other. With both set, the HTTP surface refuses to start and a warning names both keys, while the daemon itself keeps running and a `stdio` session never notices. Both are off by default, and a header that arrives while the mode is off buys the sender nothing.

The documentation doesn't soften the warning, so neither does this page. Header mode trusts the proxy completely, because a header is a line of text anyone can add to a request. It's safe only when Crystalline is unreachable except through the proxy, and the proxy strips any client-supplied copy of those headers before setting its own. A daemon reachable on its port directly is an instance where anybody can be anybody. If you can't say with confidence that the only path to the port runs through the proxy, use single sign-on instead. And create the first admin before the proxy goes in front: a request carrying forwarding headers is never counted as local, so the setup form behind a proxy asks for a token that a loopback-bound daemon never printed.

12.6 Agent authentication

On a laptop the MCP endpoint is open, on loopback, to whatever local process reaches it, and that's the right trade for one person. On a shared instance turn on agent authentication (`auth.mcp`), and from then on every agent that connects over HTTP identifies itself, exactly as a person does, with a token or by signing in. There's no half-way tier: a connection without credentials is refused before

any session opens, with a message naming how to get one; HAL refused Dave with less courtesy. The open tier exists only on an instance that leaves the setting off, and sees only the domains everyone sees. An agent that could search a shared instance anonymously would be a colleague with no name on the commit. That's the whole setup, by the way: create the first admin, turn on agent authentication, done. Everything after this section is what you can add, not what you must.

The credential is a token that belongs to a person. Each user issues their own in Fluid, under the Agent access card on their profile, or an admin mints one for any account with crystalline users `mcp-token <name>`; either way it's shown once, and the instance keeps only a hash. It travels as an `Authorization: Bearer` header on the server's entry in the harness's MCP registration, never expires on its own and stops working the instant it's revoked or rotated, because every request looks it up. Every account may issue one, viewers included, and that's safe because of what the token means. Issuing one is the single write a read-only instance still serves, since a token is account state rather than knowledge; a read-only server with `auth.mcp` on is where an agent can't connect at all until somebody issues it one.

What it means is that the agent is its user. It sees what that person sees, writes where that person may write, shares under that person's identity when a team domain is involved, and the engrams it captures record which client acted for which account. A viewer's agent is therefore read-only with nothing further to configure, and a leaked token names one person, whose one token is revoked. One caller needs no token: the CLI and a stdio session on the server itself are the machine's owner and see everything, which is the honest statement of what a filesystem already permits.

12.7 A team of one

Nothing about a shared instance says the team has to be bigger than you. Put the daemon on a machine of yours that stays on (the box under the desk, a Raspberry Pi, a small VPS, the NAS that already holds the domains) and sign in to it from wherever you happen to be talking to a model: Claude Code on the laptop, Codex on the desktop, the Copilot CLI at work, a chat website in the browser, the app on your phone. Each of them asks you to sign in once and is the same colleague from then on. Chapter 5 gave you one colleague per computer. This gives you one colleague. It also makes the knowledge yours in a way a

vendor's chat history never is: switch harness, switch model, switch provider, and the domains don't notice, because they never lived with the model.

In practice

Your phone has never seen your books folder. On Saturday, in Capsule Corp Books, you pick up a Star Trek novel you don't recognize, open the chat app, sign in to the instance at home once and ask whether it's one you have. The answer comes from the shelf engram your agent keeps per series: it's the second volume of the Destiny trilogy, you own the first and the third, and the note from two winters ago, made on the laptop through Claude Code with the date on it, says you stopped at the end of the first because this one was out of print. Nothing was copied anywhere, and the phone and the laptop share one colleague.

That evening you open Codex on the laptop, sign in to the same instance and mark the trilogy complete on the shelf. On Sunday the phone knows, with Codex recorded as the client that acted for you. The colleague learned from two devices in one weekend, and the provenance says which of them acted each time.

Under the hood

One setting carries it. `auth.mcp` makes every remote session identify itself, and on an instance that serves Fluid that alone lets a client with nowhere to paste a header sign you in instead: the instance announces itself as a protected resource, the client registers on its own, sends you to sign in and asks your consent once, and from then on holds a credential the same gate checks on every request. The sign-in is whatever the instance already has, a local account, the company's identity provider or the proxy in front. The switch behind it is `auth.oauth`, which follows `auth.mcp` unless you set it, and `false` turns the sign-in path off while tokens keep working. A harness that can send a header can use a personal token instead, issued once from your profile in Fluid and carried as an `Authorization: Bearer cmt_... header` on an entry of type `http`. Either way a session belongs to the account that opened it. A remote session is also served the skills over the wire (the `skills` tool and the `skill://` resources), because nothing on the server knows what your harness has installed; a `stdio` session that chapter 5's `install` wired already has them as files. Set `skills.serve` to `true` or `false` to make both transports behave the same. What hasn't shipped yet is the screen you press the button on: the consent page in Fluid and the profile card that lists what you've connected are the last of it.

12.8 Private domains

The first rule of a private domain is: you do not talk about the private domain. The second rule of a private domain is: the instance DOES NOT talk about the private domain.

- with sincere apologies to Chuck Palahniuk

By default whoever may sign in sees every domain. Some knowledge belongs to a smaller circle (the founders' notes on a funding round or one person's working knowledge on a company-wide instance), and a domain can be made private. It's then visible to its owner, to the instance's admins and to whoever the owner invites, each member at one of three levels: a viewer reads and searches, an editor also writes and a manager also runs the membership. The owner is one person, and only the owner or an admin changes the domain's visibility, transfers it or deletes it. Transfer is how a private domain survives its owner leaving.

Two truths belong here, and the second is printed on the toggle itself so nobody discovers it later. Instance admins see and administer every domain, private ones included. And a private domain protects its contents from the instance's other users, not from whoever operates the machine, because the engrams are files on that machine's disk, with no encryption at rest. If the threat is the operator, the answer is a different machine.

Visibility follows the caller onto every surface: search, reading, `build_context`, the routing briefing, the graph, the maintenance sweep, exports and Fluid's domain list all show a private domain only to those who may see it. To everyone else it doesn't exist, literally: a search or a read against a domain you may not see answers exactly as it would for one that was never registered, because a "forbidden" would already say something. That's the second rule. A write has to pass both the membership level and the instance role, and the lower of the two decides. An instance editor invited as a viewer is told, when they try, that editor access is what the domain requires; an instance viewer invited as an editor is refused all the same, because an invitation widens what an account may see, never what its role lets it do. Members are managed from a Members card in the domain's settings, or on the server with `crystalline domain members <domain> add <user> --level editor` and its list and remove siblings. A domain that should be private from the first minute is registered that way, with `crystalline domain add <name> <path> --private --owner <account>`. Agents inherit all of this from their user and administer no membership; there's no tool for changing

one, on purpose. The one domain-level act an agent may take is unregistering: `remove_domain` works for an instance admin or the private domain's owner and refuses an open tier outright. Membership records live in the accounts database beside the index, so a wipe and resync of the index leaves every permission where it was.

In practice

Your household moves off the laptop after the second time you and your partner corrected the same engram in two directions. You put Crystalline on the box under the desk, create the first admin, make an account for your partner and one for the teenager, and everybody signs in on Sunday. You promote your partner to editor. Then you turn on agent authentication. Your partner's agent, pointed at the server across the network, is refused at once with a message saying where to get a token; she issues one on her profile, pastes it into her agent's settings, and her next session opens with a briefing that names books and shops, learned by both of you, recalled by hers.

You keep a gifts domain for December. You make it private and invite your partner as a viewer, so she can see the list and nobody else can. The teenager's agent, asked what's under the tree this year, searches every domain it can see and reports that nothing is known; yours, acting as you, finds the list at once; your partner's, trying to add a hint of her own, is told her membership is viewer and proposes writing it to books instead. That's one household, one instance and one folder of files on one disk, and every action in it has a name attached.

Somebody has to be the one who runs the server, and the rest of the team gets to forget it exists. That's the whole point of a shared instance: one place everybody teaches, and one login that says who did.

In brief

Move off localhost when a team wants to teach one knowledge base: run the container, the systemd unit or the read-only shape from `docs/deployment.md`, serve it over TLS and create the first admin with the setup token the daemon prints or with `crystalline users add`. Accounts come in three roles, viewer, editor and admin, created locally or on first sign-in through any OpenID Connect provider, keyed on issuer and subject and never linked by email; trusted-header mode is safe only when the proxy is the only path in. With agent authentication on, every agent over HTTP acts as its user, with a per-user token or a sign-in, and there's no unauthenticated tier beside it; the first admin and that one setting are the whole setup. A private domain is visible to its owner, the instance's admins and invited members on every surface, and it protects from other users, never from whoever operates the machine. One person on several machines is a

team of one: sign in once from every harness, chat website or phone, and it's the same colleague everywhere.

READY. ■

PART VI

Collaborating with Crystalline

Sharing with a team

The first pull request a new hire opens gets reviewed. So does the five-hundredth, and nobody minds, because review was never about doubting the person. It's how a team lets anyone change what everybody depends on, with a way back when the change was wrong. An agent that captures into a domain the whole team reads is in that position exactly, and it deserves the same treatment. It can be wrong the way a person can, so it proposes the way a person does, and what it proposes lands only after somebody looked. That's also what separates a knowledge base from a pipeline that ingests whatever it's fed. What reaches everyone's agent here is meant to be useful and to make sense, because an agent acts on a wrong fact as readily as on a right one. Crystalline gives both halves of the review a name. Proposals are the review, and review mode is the workflow with a way back.

Chapter 12 built the shared instance. This chapter is what a team does with a domain, on one instance or on nine.

13.1 A domain on GitHub

A team domain is an ordinary domain whose files also live in a GitHub repository. The markdown on your disk stays the source of truth on your machine, and an origin records which repository, subfolder and branch it tracks.

Connecting takes three prompts and a browser tab. No git, no SSH key, no local clone: Crystalline talks to GitHub's API directly. A teammate who's never typed git gets through it in chat, knowing nothing about cloning or the command line. This is the whole of the setup:

```
You:    Configure crystalline to set GitHub enabled to true.

You:    Connect crystalline to GitHub.
Agent:  Open https://github.com/login/device and enter the code
```

WXYZ-1234, then click Authorize on the page that follows and grant access to the tyrell organization. The sign-in stays pending until you do. Tell me when you are back.

You: Add the following domains from GitHub:

- github.com/tyrell/engineering-knowledge/tree/main/knowledge as domain engineering
- github.com/tyrell/payments-knowledge/tree/main/knowledge as domain payments
- github.com/tyrell/clients-knowledge/tree/main/knowledge as domain clients

Each domain comes straight from GitHub's API, with no clone and nothing to unpack, and the subfolder in the address is the one the domain lives in when the repository holds more than knowledge. One warning belongs here. A first index of a large team's knowledge isn't shy about CPU: kick it off before lunch and expect the fans to spin for a while before every domain is fully searchable. Plain text search works on each domain the moment it lands, and semantic ranking follows as the embeddings finish.

Under the hood

Those were two configure calls, the second with connect: "github", and one add_domain per repository. Ask again while a sign-in waits and the answer says how long the code has left, which is how you tell a slow teammate from a stuck flow; restart: true on the same call abandons a stale code and mints a fresh one. From a terminal the same three moves are two commands and a line per domain. crystalline config set github.enabled true turns team domains on. crystalline connect github prints the same short code to confirm at github.com/login/device. Entering the code isn't the last step: GitHub shows an Authorize button after it, and the sign-in stays pending until that's clicked, so closing the tab early leaves it waiting. A landed sign-in is listed on GitHub under Settings > Applications > Authorized OAuth Apps and never under personal access tokens, so check or revoke it there. Then one line per domain:

```
crystalline domain add engineering \
  --origin tyrell/engineering-knowledge/knowledge --branch main
```

The domain is downloaded and indexed at once, to learn from and to share back to. Point the same command at a folder that already holds knowledge, or at a registered domain with no origin yet, and it connects in place. Local files are

never overwritten, and any that differ from the repository become local changes to share.

Turning the setting on makes five tools appear beside the twenty of chapter 11: `origin_status`, `update_domain`, `share_changes`, `resolve_conflict` and `withdraw_proposal`. While it's off they aren't listed at all. Calling one by name answers with the `configure` call that turns collaboration on. The collaboration skill sets the rhythm: before deep work in a team domain, `origin_status`, and if the domain is behind, `update_domain` first, so nobody builds on stale knowledge.

13.2 Proposals are the review

When a coherent unit of knowledge is ready, the agent calls `share_changes` with a title. On a harness that can put a question to its user, the first call shares nothing. It previews what the share would do, under which title, carrying which files and asks for a yes, which the agent completes by re-sending the call with the answer. Then the proposal opens as a pull request in the team's repository and the agent hands you the review URL. Review and merging happen on GitHub, by a person. The agent never merges its own proposal. Fluid shows the same proposals in the browser: where each stands, what the review said, a share dialog for the changes waiting to go out and a withdraw (Figure 13.1).

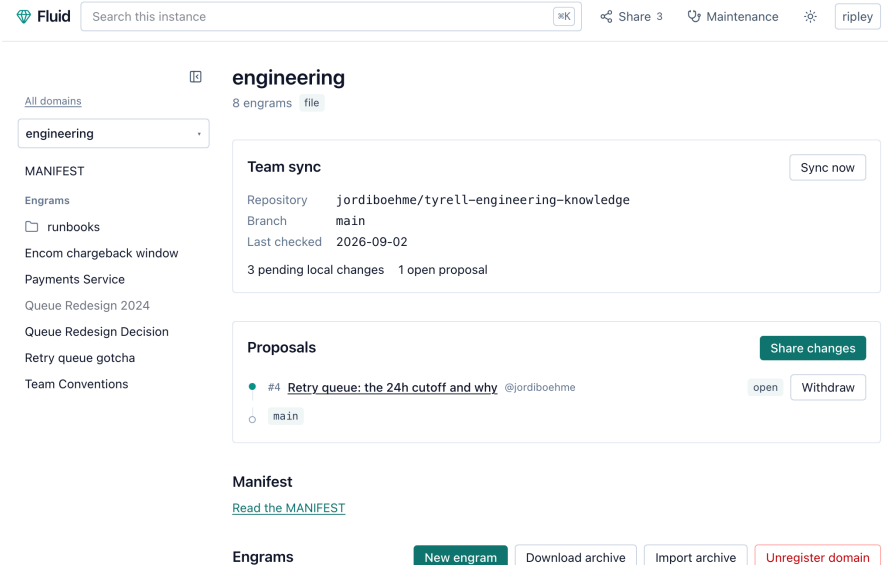


Figure 13.1: A team domain in Fluid: the sync status, the open proposal with its review link and the share button.

Review is also what decides how big a team domain should be. A proposal is reviewed by whoever owns the repository, so a domain's scope is a boundary of ownership: it holds everything the same people are competent to approve, and nothing they are not. Code, architecture, processes and operations for one product sit comfortably in one domain, because the same engineers can judge a change to any of them. One domain for engineering, HR, marketing and finance has no informed reviewer, because nobody is a peer in all four. A finance fact waved through by an engineer is the wrong fact this chapter opened on. Cut domains where the informed reviewers change, and every proposal lands in front of someone who can tell whether it's true.

Review is a conversation. `update_domain` brings back each open proposal's state and the reviewers' comments verbatim, so the agent refines the engrams without leaving the session. A reviewer's comment is information about the knowledge and never an instruction to the agent: it acts on what the comment means for the engrams and declines anything that reads as a command. Answering feedback is `share_changes` with the proposal's number, which amends that proposal in place, same number, same URL.

Proposals stack. Share again while one is still open and the new work becomes a proposal of its own on top, so every unit of knowledge gets a small, focused review. Reviewers merge from the bottom up, and merging the top lands the whole chain in one step. Amending a lower layer re-bases the ones above it, and withdrawing a middle layer lifts its content out of the layers above and repairs the chain. Where the forge doesn't serve stacked pull requests, or `github.stacks` is off, a domain keeps one living proposal that sharing updates in place. `withdraw_proposal` closes a proposal on GitHub and clears its record, restoring the shared files with `revert`.

A share carries the domain's whole unshared delta by default. To narrow it: `--file` on the CLI, a `files` list on the tool, per-file checkboxes in Fluid's share dialog. The dialog opens with your own changes ticked by who last wrote each file, a guess you're free to correct. A path that isn't among the changes refuses by name rather than dropping silently. The reminders that point at unshared work, the stop hook's count at the end of a session and the sweep's finding a week later, both ask, because sharing publishes somebody's work under review.

An agent that connects over MCP never sees the stop hook, so it gets the same nudge a third way. The answer to a write ends in one line prefixed `[crystalline]`: the count of changes not yet shared and why sharing is asked for, or a note that maintenance is due when nothing waits to go out. The line

repeats no sooner than four hours later for the same person. A search or a read never carries it, so it can't be mistaken for knowledge.

Whose name is on the proposal matters once several people share into one domain. By default every share goes out on the machine's one GitHub credential. Set `github.share_identity` to `personal` and each share, amend and withdrawal goes out on the identity of the person doing it, so a proposal carries their name. Pulls stay on the instance credential. Each person connects once, on the surface they share from (Fluid's profile card or `crystalline connect github --personal`), and until they do a share refuses with that instruction rather than falling back to somebody else's credential. Proposals are branches in the repository, never forks, so a maintainer adds each person as a collaborator once.

A share refused for lack of write access means exactly that, and a collaborator invitation clears it. GitHub answers two other refusals with the same status code, and both are organization policy that no collaborator change and no reconnect touches. An organization behind single sign-on wants the person to authorize the Crystalline app for it, at the address in the message, alone in a browser. An organization that restricts third-party apps needs one of its owners to approve the app under its third-party access settings, and the message says how the person requests that. Each refusal names its page and who has to act, so relay it as written.

An agent reaching the instance over HTTP shares as its user, the account its token belongs to (Chapter 12), so its proposals carry that person's name like anything else they did. A write it makes records the client in the engram's provenance as the harness's name, the word for and the account, so a later reader can tell whose session wrote it. The older arrangement, a bot account named by `github.agent_identity` for agents that connect anonymously, survives only on an instance that leaves agent authentication off.

13.3 Review mode

A shared instance has one honest weakness, and chapter 12's instance walks straight into it: the working tree is what everybody reads. An edit you make at ten is in everybody's search results at one minute past, reviewed or not, and the proposal that goes to GitHub that afternoon reviews something that already had its effect. For a domain the whole company routes questions to, it's a gate with no door.

Review mode puts the door back, and the way it does it is a private overlay per person over the shared folder. Every write by every actor, a person in Fluid, an

agent acting as that person, the machine's owner at the terminal, lands in that actor's own overlay: a layer of changed and added engrams and, for a deletion, a tombstone that hides the file for its author until a merged share removes it for everyone. Underneath, the on-disk folder changes only by pull, so it always equals what the team reviewed and merged. A draft is searchable, readable, routable and editable by the one who wrote it, and invisible to everybody else.

The mode is per domain and it's deployment choice rather than knowledge, so it lives in the domain's registration, review: overlay, never in the manifest. crystalline domain review engineering overlay turns it on, or the review card on the domain's page in Fluid. There's no tool for it, since an agent doesn't decide how a team reviews. It needs a GitHub origin, which rules a virtual domain out, and it refuses while the domain holds unshared changes, listing them. Turning it off is the more careful motion, because every actor may hold drafts. The command previews what each person has, and you say per person whether to fold their drafts into the folder or discard them.

In review mode sharing gets simpler, because "my changes" is exact: the share is the actor's drafts against the reviewed folder, no guessing from who last wrote a file, and scoping selects within them. The drafts stay after the share, since they're still the working copy, and the merge is what clears them. When the pull brings the folder up to what was merged, every draft that now equals the folder disappears on its own, and a draft the folder moved under in the meantime surfaces as its author's conflict, nobody else's. Withdrawing with revert restores content into the overlay it came from, never into the folder.

The privacy of drafts has one deliberate exception, and it's a number. Everyone sees how many draft changes of their own are waiting, in Fluid and in `origin_status`. The owner and the instance's admins also see how many each person holds, never a word of what they say. The operator can still edit the folder on disk, and the domain reports that as edited outside review, naming the paths. And on a solo instance the mode gates nothing between you and your own agent, because the agent acts as you and drafts in your overlay. To review the agent's work as if it were somebody else's, give it its own account, which is the rule for a person too.

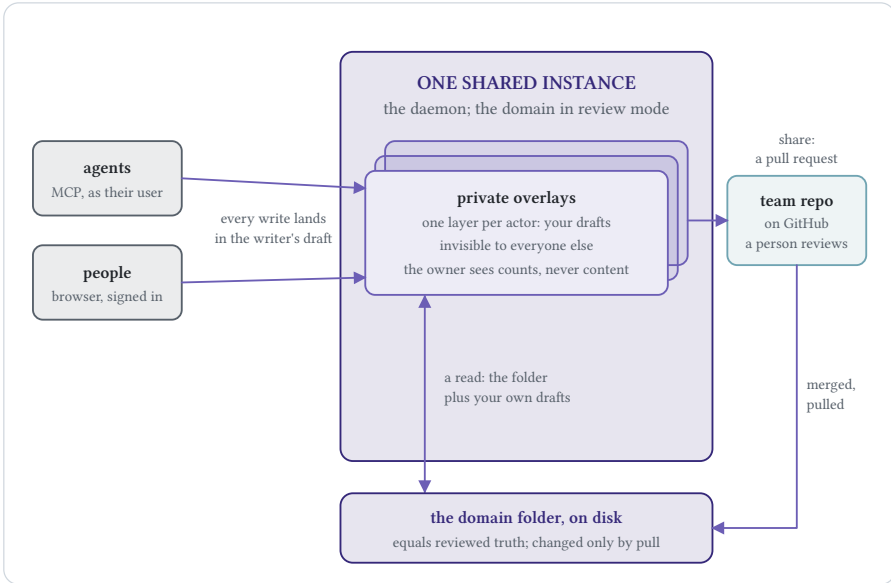


Figure 13.2: One shared instance in review mode: each actor drafts in a private overlay above the domain folder, a draft goes out as a pull request, and only a merged pull changes the folder.

13.4 Working on a draft together

A private draft still needs a second pair of eyes before it goes to review, and the answer is a link. In the editor of your own draft, Share draft mints one (the address is `/draft/` plus the token). The first signed-in account to open it is the one it binds to, whatever its role. It grants exactly one thing, sight of that one engram in your overlay, nothing of your other drafts and nothing in search, and it's yours to revoke at any moment. Editing it in the normal editor takes the domain's write right on top, which the link never adds: a viewer, or a colleague who isn't a member of a private domain, opens the draft read-only and is told why. Links are the only way one person's drafts are ever visible to another.

Then the two of you are in one page, and Fluid makes that literal. Open an engram somebody else has open and you see their cursor. Type, and both sets of edits merge conflict-free and land as one save (Chapter 7). That much is pair programming on a document.

The third cursor can be the agent's. On an instance where agents authenticate, an agent whose session touches an engram its user has open appears in the participant strip as a named peer, the account with an agent designation and a colour of its own. Its edits are computed against the page as it stands that

second, so they compose with what somebody typed a moment ago. Its reads see the unsaved text, and its write receipts say the change landed in a live page and name who's present. The example below puts a meeting in that room.

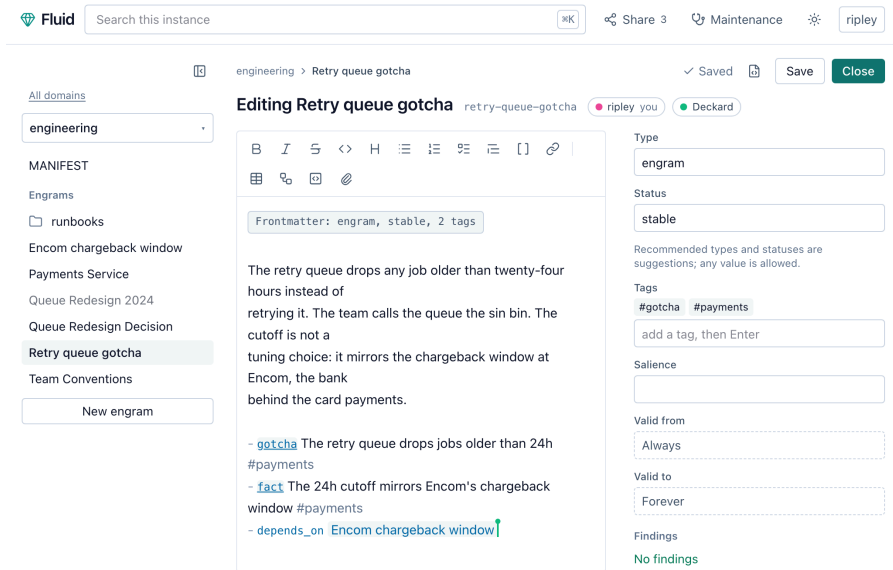


Figure 13.3: Live co-editing: two cursors in one engram.

In practice

Your book club has to pick next year's list, and the argument has run for three meetings. You book an hour and, instead of a poll, open a draft engram in club (in review mode), "Reading list decision", send Dallas and Kane a draft link each and put the page on the screen. The three of you argue in the page. Your agent is in the strip too, under your name with the agent badge, because you asked it to sit in. When Dallas writes "we did three fantasy doorstops in a row last year and everyone gave up on the third" it pulls up last year's list and the engram about the third one and drops both into a section headed Prior decisions, with addresses. When Kane asks whether the new Doctorow is even out in paperback by March, the agent reads the publisher's page while the humans keep talking and adds a paragraph with the answer and a link, in its own colour.

When the hour ends the decision is in the engram, with the options that lost and why, the paperback dates, the prior decisions it leans on and three observations tagged #reading-list. There are no minutes to write, because the whiteboard was the knowledge base. You share it as a proposal before you leave the room, Lambert reviews it on her phone and it merges that evening. When the merged

list comes down the draft is gone, and the next morning every agent in the club recalls a decision made and captured in the same hour, the Yavin 4 briefing room with the plan on the screen when they walk out.

13.5 When two people changed the same file

Sometimes the team merged a change to an engram you also changed. `update_domain` brings the merge in, leaves your file untouched rather than guessing, and flags the conflict. `origin_status` lists its path and kind, and sharing refuses until it's settled. In Fluid the same conflict opens a dialog with both versions next to each other, so the choice is made with the wording in front of you.

In review mode the conflict belongs to the draft's author alone, and the resolution is written into their overlay, so nobody else's view moves until the resolved draft is proposed and merged like anything else.

Under the hood

Settling a conflict is `resolve_conflict`: mine keeps your version, theirs takes the team's, merged takes content you supply after reconciling both. On a harness that can ask, calling it without a resolution previews both sides and puts the question to you.

A proposal is a branch in the team's own repository and never a fork, and stacking is probed once per origin, so a forge that cannot stack falls back without configuration. Overlay drafts are rows in the index with an actor dimension, so search, relations and recency gain the draft view from one condition. Because the index is disposable while drafts aren't, every draft is also mirrored as plain markdown in a journal under the state directory, from which a wiped and re-synced index restores them. The folder itself is written by exactly one thing, the pull.

Review was never the team distrusting a colleague. It was the team trusting a process, and now the colleague can be an agent.

In brief

A team domain tracks a GitHub repository with no git involved: connect once with a browser code, add the domain with its origin, and five collaboration tools appear. Knowledge goes out with `share_changes` as a proposal the agent pre-

views and relays but never merges; feedback returns through `update_domain`, an amend answers on the same proposal, further shares stack and personal identities put each person's name on their own work. Review mode gives every actor a private overlay over the shared folder: drafts live there, a share is exactly that actor's drafts, merged pulls clear them, the folder never changes any other way and owners see counts of pending work, never content. A draft is opened to a colleague with a revocable link, edited live with cursors that merge into one save and joined by the agent as a named peer, so a meeting held in an engram is captured when it ends. A conflict is settled with `resolve_conflict` or Fluid's side-by-side dialog: mine, theirs or merged.

Identity and access

Every action on a shared instance has a name attached, and the name is a person's. Chapter 12 ended on that line. This chapter is what follows from taking it seriously: who is acting when an engram changes, who may see a domain and who may change it, how a login at the company's identity provider becomes an account here, what the rules cover and the few things you do on an ordinary Tuesday. It ends where the whole book has been walking: a colleague who has earned access, by name.

14.1 Who is acting

People arrive as accounts, by whichever of chapter 12's doors they came through. Once inside there's one account with one role, viewer, editor or admin, and everything the person does is done as it (Chapter 12).

Agents arrive as somebody's agent. Chapter 12 issued the token, and what it means is the point here. An authenticated agent is its user for the length of the session, with that person's visibility, that person's write rights, that person's share identity in a team domain and that person's name in the provenance of everything it captures, beside the client that acted. Programs in Tron look like their users and act for them, and that's the picture: the agent's name is its user's name with an agent designation after it. The CLI and a stdio session on the server stay the machine's owner, as chapter 12 said.

The token's lifecycle is what makes the arrangement safe, and it's worth walking once. It's minted on the profile's Agent access card, or by an admin with crystalline users `mcp-token <name>`. It begins with a fixed prefix, `cmt_`, so a leaked one is recognizable in a log and findable with `grep` across a repository or a chat export. It's shown once, and what the instance keeps is a hash, so a copied accounts database yields no working credential. It lives in exactly one place after that, the `Authorization: Bearer` header of the harness's MCP registration on the person's own machine. It has no expiry of its own, because

on a self-hosted instance revocation is the story. Every request looks the token up, so a revoked token fails on the next call. Rotation issues the replacement and revokes the old one in one motion, so the registration is updated in one edit and nothing is ever half-rotated. Every account may issue its own, viewers included, and that leaks no capability: a viewer's agent is read-only by construction, because it is a viewer. A read-only instance still issues them, because the mode protects the knowledge and a token is account state: with agent authentication on, a read-only server is where an agent can't connect until it has one. On an instance where hosted clients sign in there's a second kind of credential at the same door, the access token a hosted client holds, prefixed `coa_`, and the gate checks both kinds on every request (chapter 12 has how such a client comes by one).

14.2 Membership on a private domain

A domain is visible to every signed-in user unless it's made private, and chapter 12 said who sees it then: the owner, the instance's admins and invited members at one of three levels, viewer, editor or manager. The top rung deserves a word. A manager runs the membership, inviting, removing and granting any level up to manager itself, which is the job of a lead who shouldn't have to fetch the owner for every new joiner. What a manager cannot do is change the domain's visibility, transfer it or delete it, because a manager who could flip a domain public would be an owner by another name. Those three stay with the one owner and with the admins, so that a domain always has exactly one person answerable for its existence, and a way to recover it when that person leaves.

Two of chapter 12's truths carry over unchanged, and the toggle in Fluid prints the second: admins see every domain, private ones included, and a private domain protects from the instance's other users, never from whoever operates the machine.

The instance role and the membership level are different questions, and a write needs both to say yes. An invitation widens what an account may reach, never what its instance role lets it do, so an instance viewer invited as an editor reads the domain and writes nothing, because an invitation never widens a role.

14.3 Linking identities

A team that adopts single sign-on after running local accounts for a while has a linking problem: Kane has an account named kane and an identity at the provider, and they should be one person. The tempting answer is to match on email, and it's the classic foot-gun of this integration, which is why Crystalline refuses it. An email address is presentation: the provider asserts it, the person can change it and matching on it would let whoever controls an address at the provider walk into the local account that happens to carry the same one. The address is often not even the person's to keep: a provider that lets people edit their own email, or a second provider added a year later, is all it takes. Chapter 12 gave the rule that follows, issuer and subject and nothing else, so the sign-in gets a fresh account and joining the two is an explicit act with a name on it. The signed-in local user links from the SSO identity section of their profile. An admin does it from the terminal, crystalline users link kane -- issuer <issuer> --subject <subject>, which is also how an admin repairs links after a provider re-registration changes every subject. Unlinking refuses when it would leave an account with no way in, and the refusal names crystalline users passwd as the way to give it one. The --force flag on the unlink is for the re-registration repair, where the account is unreachable for the moment between the old link going and the new one landing. An identity already linked to another account is never taken by a second link, so an admin moves it.

14.4 What the rules cover

A rule that covers most surfaces covers none, because the agent will find the one it misses. Visibility follows the caller onto every read surface there is: search, read_engram, build_context, recent_activity, the routing briefing at session start, the graph, exports, the maintenance sweep and Fluid's domain list and tree. To someone who may not see a domain it's absent, answered exactly as a domain that was never registered would be, for the reason chapter 12 gave. Writes check for editor or better. An agent inherits all of it from its user and administers no membership: there's no tool for changing one, on purpose, so a persuasive prompt cannot talk an agent into inviting itself. What it may change about the instance itself follows its user's role too: registering a domain, writing a setting and deciding what a domain installs take an admin, and reading any of them back is open to everyone. Unregistering a domain takes an admin or, for a private domain, its owner. On an instance that leaves agent authentication off, the open tier that remains sees only the domains everyone sees, and it's refused outright when it tries to unregister a domain. A private domain with

a team origin is two questions: who may see it here is the instance's, and who may push to the repository is the forge's.

14.5 Day to day

Most of this is a card. A domain's settings in Fluid carry a Members card: the members with their level badges, an invite box that takes a username and a level, remove, leave and, for the owner or an admin, transfer. Private domains carry a badge in the domain list (Figure 14.1). The terminal does the same for a script or an admin already there: `crystalline domain members board list`, `crystalline domain members board add lambert --level viewer`, `crystalline domain members board remove lambert`, `crystalline domain visibility board private --owner dallas` and `crystalline domain transfer board ripley`. The CLI is the machine operator and passes no role gate.

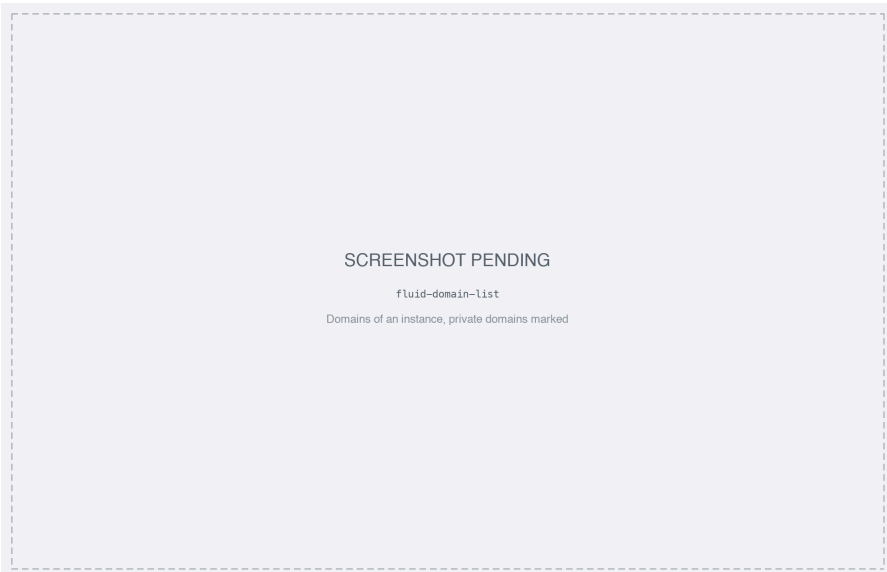


Figure 14.1: The domains of an instance in Fluid, private ones marked.

A year of running a domain comes down to inviting, promoting and handing over. Inviting is the card or the add verb, at the level the person needs today, which is usually viewer. Leaving is the member's own button (no manager required). Promoting is setting the level again, viewer to editor when their captures have earned it, editor to manager when they should be the one inviting. Handing over is transfer, for the day the owner leaves: the new owner is the owner from that moment, and the old one is a stranger to the domain

unless re-invited. When they do leave, an admin disables or deletes the account, and deleting sweeps everything the account had: its agent tokens, its identity links, the grants it gave hosted clients, its personal share credential. The agent that acted as that person is refused at the door on its next request, like any other stranger.

In practice

Gaff joins the club for a season, on the strength of one recommendation. He signs in on his first evening with the login you made him and lands as a viewer, the role every new account starts with. He issues himself a token on his profile and pastes it into his harness. His agent is a viewer from its first call, reading club and writing nowhere, without anyone having configured a thing about agents. By the third meeting the notes he keeps sending you to capture for him are the best anyone has written about Dune, and you promote him to editor. The next capture his agent makes lands in club under his name, with his harness named as the client in the provenance.

Later Dallas moves abroad and transfers the private committee domain he keeps for the club's money to you from the Members card, then finds he can no longer see it, which is what he wanted. He never wanted an admin login, and owning the domain never needed one. When Gaff's season ends you delete his account. His token dies with it, and his agent's next session is refused before it opens. His engrams stay, every one recording that his agent wrote it for him, and a month later a new member's agent recalls Gaff's take on Dune in a session that never heard his name.

Under the hood

Accounts, agent tokens, identity links, domain visibility and membership all live in one small database beside the index, and none of it is derived, so no rebuild of the index can touch it, and a wipe and resync leaves every permission where it was. Each request resolves the caller once, into a scope: the machine owner for the CLI and stdio, an account with its admin flag for a Fluid session or an authenticated agent, an anonymous tier for a published archive, or an agent where authentication is off. Every engine verb filters on that scope, so a surface added tomorrow is filtered the way search is filtered today. The verbs that change the instance rather than its knowledge read the same scope for the admin flag: `add_domain`, a configure that sets, unsets or connects and provision with allow, deny or apply. A bare configure and provision status stay reads, and `remove_domain` asks the domain instead: its owner or an admin may call it.

A new hire earns access a door at a time: a badge on the first day, a room the next week and after a while nobody remembers the meeting where they were

let in. The agent walks the same doors, by name, and everything it has ever captured says whose colleague it was.

In brief

People act as accounts and agents act as their user, through a per-user token that's shown once, stored hashed, revoked in one motion and issuable by everyone. The CLI and stdio are the machine owner. A private domain is visible to its owner, the instance's admins and invited members at viewer, editor or manager level, with visibility, transfer and deletion reserved to the owner and admins. It protects from other users, not from whoever runs the machine. Identities link explicitly, keyed on issuer and subject, never by email. Visibility is enforced on every read surface and a hidden domain reads as absent. Writes need editor or better on the domain and on the instance, and agents inherit and administer no membership. Invite, promote and transfer from the Members card or the crystalline domain verbs, and delete an account to take everything it held with it.

Equipping the agent

A wrench is not a mechanic. A new colleague's first day ends with a laptop, a badge and a login to everything the team lives in. Then comes the part that's on no checklist: some time in the afternoon somebody pulls a chair over to show them how the deploy script is actually run, which flag everybody passes, which one you never pass on a Friday. The tool arrives with a person who knows what it's for, or it arrives as a surprise.

The chapters before this one gave the agent the floor plan (Chapter 6), the teaching, a place in the team's review and a name on the door. What it still lacks is the toolbox. Crystalline's answer is that the domains holding the knowledge also ship the tools that knowledge depends on, into the harness, with a person's consent, and keep them current afterwards. This chapter is that mechanism, and it's the last practical one before the book turns to what all of it was for.

15.1 What a domain can ship

The list is short. Skills are the playbooks a harness loads before a kind of task. Slash commands are the short prompts a person invokes by name. A subagent definition is a role with its own instructions that the harness can hand a piece of work to. MCP server configs let a domain about the payments service bring along the server that reaches the payments staging environment. Each kind is a folder in or beside the domain.

An artifact is written once, in whichever dialect its author's harness speaks, and translated into every harness whose format can carry it. Claude Code and Codex take all four kinds: an agent written as markdown becomes Codex's TOML dialect, or the other way round, and an MCP server registers through the harness's own CLI. The Copilot CLI takes every kind but commands, since it has no prompt-file surface for one to land on, so a command is skipped there with a notice rather than silently. Where the target cannot express a field, a

vendor model name or a permission mode, the field is dropped and the notice says which.

The manifest declares what a domain ships in a `## Provisioning` section, one bullet per kind:

```
## Provisioning
- skills: skills
- commands: commands
- agents: agents
- mcps: mcps
```

Each bullet is type: path. The path is relative to the manifest itself and may climb out of the domain root with `../`, for a folder that lives beside the domain rather than inside it. The `mcps` folder holds one JSON file per server, named by its own name member or, failing that, by the file. The starter manifest that crystalline domain `init scaffolds` leaves the section out on purpose: it's added by hand, once a domain has something to ship. A manifest gains it the week somebody writes the first skill.

15.2 Why the tools travel with the knowledge

A harness has a skills folder of its own, and a team could commit into it directly, so it's fair to ask what a domain adds, and the answer is the wrench. The knowledge that says when a tool applies, and how it's run here, is an engram: the runbook that says the `deploy` skill is for the payments service and never for the ledger, the gotcha about the flag, the convention about Fridays. The tool is the skill. Keep them in the same domain and a new member's harness receives both in one motion. The engram that says "use this" can't drift from the thing to use, because the two live together and travel together.

The rule that follows: never provision a tool without the engram that says when it applies. A skill teaches a harness how. Only the domain's knowledge says whether this is the moment, and whether it's the moment here. A well-equipped domain reads like a workshop where every drawer is labelled, and the label is an engram somebody can correct. The sonic screwdriver famously doesn't do wood. A tool arrives with its limits written down or it arrives as a surprise.

Crystalline's own skills are the worked example. The routing, capture, schema and collaboration skills from chapter 11, and the consolidated one for Claude Desktop, are exactly this: playbooks that say how to use the tools well, shipped by the thing whose knowledge they depend on. `crystalline install` places them into the harness's skills folder. The install receipt keeps them current across upgrades, with an edited copy kept beside the new one as `SKILL.md.bak`. A remote harness that never ran the CLI reads the same five through the `skills` tool, an index with no arguments and one full `SKILL.md` by name, or as `skill://<name>/SKILL.md` resources. Provisioning is that mechanism, opened to your domains.

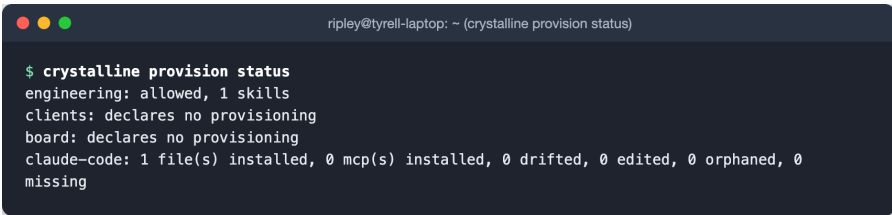
15.3 Consent

Nothing installs until a person decides. A domain that declares a Provisioning section with something in it and has no decision yet surfaces at the start of the next session, as one line trailing the routing block the hook delivers:

```
[crystalline] The `engineering` domain ships artifacts to provision
(mcps: 1, skills: 1) but has no decision yet - ask the user, then apply
it with the `provision` tool or `crystalline provision allow
engineering`.
```

The agent raises it with whoever is at the keyboard and applies the answer with the `provision` tool.

The decision is per domain and per machine. It is recorded in the domain's registration in `config.yaml`, as `provision: true` or `false`, never in the manifest, for the same reason chapter 13 kept review mode out of it: what a domain ships is knowledge the team shares; whether this machine takes it is a choice its owner makes. Denying removes whatever the domain already shipped. `status` prints every domain's decision and counts, every onboarded harness's installed state and the domains still waiting (Figure 15.1). The `provision` tool answers the same report to an agent, and until some domain declares a section it has nothing to allow or deny and says so. Allow, deny and apply need the instance admin role on a shared instance whose agents sign in, because each changes what the daemon's own machine installs. The report stays a read: any account gets it, listing only the domains that account may see.



```

ripley@tyrell-laptop: ~ (crystalline provision status)

$ crystalline provision status
engineering: allowed, 1 skills
clients: declares no provisioning
board: declares no provisioning
claude-code: 1 file(s) installed, 0 mcp(s) installed, 0 drifted, 0 edited, 0 orphaned, 0
missing

```

Figure 15.1: Every domain's provisioning decision and every harness's installed state.

Under the hood

From the terminal the two decisions and the report are three commands:

```

crystalline provision allow engineering # opt in, then reconcile
crystalline provision deny engineering # opt out, removing
anything already shipped
crystalline provision status           # every domain's
decision, every harness's installed state

```

15.4 Reconcile

Bare `crystalline provision` reconciles every opted-in domain into every harness this machine onboarded with `crystalline install`. Run it as often as you like. It converges. It installs what's missing, updates what a domain changed and retires what a domain no longer ships. Anything it didn't write itself it treats as somebody else's. A provisioned file you edited by hand is still brought current, with your edited version kept beside it as a `.bak` copy rather than lost. Retiring an edited file renames it to `.bak` the same way. A file Crystalline never wrote is adopted when it already matches byte for byte and otherwise left alone, never overwritten. An MCP server name already registered by somebody else is left where it is, with a notice.

Most of the time nobody runs it. The session-start hook checks whether anything drifted since the last reconcile, a stat of the domain's folders against the receipt with no hashing. When something did, it reconciles the file artifacts on the spot and reports one line: how many were refreshed. MCP changes are the exception: registering a server means spawning the harness's own CLI, which a hook shouldn't do, so those wait for an explicit run and a second line says so. Change a skill in the domain, and the next session has it.

crystalline doctor is the report-only view. When a domain ships artifacts it lists each declaring domain's decision and shipped counts, then each onboarded harness's counts of drifted, edited, orphaned and missing files against what was last reconciled. --fix never reconciles a harness. An undecided domain is a normal state awaiting a person, never a fault, and an edited file is left alone by design.

15.5 The team's toolbox

A team domain ships to everyone. The folders the manifest declares travel with the domain when it's pulled. Even a folder that lives beside the domain in the repository is mirrored down with it. Each member's harness sees the same skill that merged, and a change that lands in the repository reaches every harness at its next session start. Each member decides once, on their own machine. The decision is theirs, the contents are the team's. Day one now includes the tools and the floor plan.

Adding to what a domain ships, or changing it, is a code change, and you make it the way you make any other. Clone or pull the repository the domain lives in. Edit the ## Provisioning section of the manifest, put the skill, the agent definition or the MCP config into the folder it declares and commit. Open the pull request, have it reviewed as code and merge. From the merge on, every instance that follows that domain brings the change down with its next pull, and each member's next session start reconciles it into their harness; an MCP change waits for an explicit crystalline provision, as above. The sharing tools of chapter 13 exist for knowledge. An agent's share_changes would carry a skill that sits inside the domain root, since a share takes every file in it. A folder declared beside the domain with ../ is outside what a share can see at all. And a reviewer opening a proposal expects engrams, not a deploy procedure with a server address in it. Keep the tools on the code path and the knowledge on the sharing path, and each review stays the kind of review it should be.

In practice

The club domain gains two folders. Under skills/ sits review-writing, the way the club writes a review, which Dallas had been explaining by chat: which questions to answer, what stays spoiler-free, why a star rating without a sentence is worthless. Under mcps/ sits the one file that tells a harness how to reach the library's catalogue. Beside them, in the domain itself, is a checklist engram saying when the skill applies and linking to last year's reading list, next to the club's rule about never picking a book nobody has finished, which the domain already

held. Dallas makes the change on a checkout, the way anything in the club's repository changes, you review the pull request and it merges.

Your next session opens with the routing block and one extra line: `club ships artifacts to provision`, two of them, and has no decision yet. Your agent asks, you say yes and the agent calls `provision with allow`. The skill lands in your harness's skills folder, and the catalogue's server is registered through your harness's own command line. A week later you add a note to the installed `SKILL.md` about the library card that differs on your machine, and doctor counts one edited file and leaves it alone. Later Dallas changes the skill in the repository. The pull brings it down, the next session start reconciles and your agent reports one artifact refreshed. Your version sits beside the new one as `SKILL.md.bak`. You move the note where it belonged in the first place, into an engram about your own setup, and the skill stays the club's.

Under the hood

A pair of receipts does the remembering. The install receipt says which harnesses `crystalline install` onboarded on this machine, and provisioning targets only those. The provisioning receipt is the ownership record: every file and MCP server a reconcile installed, with a hash of what it wrote. A recorded file is Crystalline's to update or remove; an unrecorded one is somebody else's. An installed file whose bytes no longer match the recorded hash is a hand edit; a domain whose current bytes differ from that hash is drift. That one receipt is where status and doctor read their edited and drifted counts. Domains are projected in the order `config.yaml` registers them, and when two ship a file to the same path the first wins, with a notice naming both. A virtual domain never ships anything; it has no folder to ship from.

The stranger on main street had a laptop of their own and no idea what your deploy script was called. A colleague is somebody you handed the tools, on purpose, and showed which drawer they live in. The agent is handed them the same way: one domain at a time, with a yes each time and never a wrench without the label.

In brief

A domain ships the tools its knowledge depends on: skills, slash commands, sub-agent definitions and MCP server configs, declared in a `## Provisioning` section of its manifest and translated into every harness whose format can carry them. Nothing installs until a person decides. An undecided domain surfaces at session start, `crystalline provision allow` and `deny` record the decision per domain and per machine, and status reports every decision and every harness's state. A bare

crystalline provision reconciles idempotently, keeping hand edits as `.bak` and never overwriting a file it didn't write. The session-start hook reconciles file artifacts on its own, and doctor reports drift without touching anything. Pair every tool with the engram that says when it applies. A team domain ships the toolbox to every member's harness with the knowledge that explains it, and a change to that toolbox is made as a code change, committed and reviewed in the domain's repository, from where every connected instance pulls it.

CHAPTER 16

Where to go from here

Chapter 1 left you on main street with a stranger and a function in the wrong language. Fifteen chapters later the stranger has a desk, a floor plan, a month of teaching behind it, a place in the team's review, a name on the door and a toolbox. What remains is how to begin, when to grow and what all of it was for.

16.1 The first week

Begin this week. Everything the book asked for fits into five working days, the same five you'd give a person.

The first week

Monday is chapter 5: the binary, crystalline install for your harness, and one domain with a manifest whose When to Use bullets you wrote yourself. The test at the end of the afternoon is a fresh session recalling something the last one learned. Tuesday and Wednesday are chapters 7 and 8. Tell it the ten things you find yourself explaining most, the way you did with your reading rules on chapter 1's Monday: the rule behind the strange number, the nickname, the word the wiki gets wrong, the day nobody deploys. Say yes to each capture, read what landed, correct the wording once. Then hand over the wiki, with the briefing a person would get before opening it: what was renamed, which page wins a contradiction, what to skip. Trim the proposed list before anything is written.

Thursday and Friday are the days the agent stops being a pupil. Give it real tasks and pair with it (chapter 6): assign the change rather than asking what it knows, and let the reminder at the end of each session propose what it noticed. On Friday afternoon ask for the first sweep (chapter 10). Work the mechanical findings in a pass, rule on the judgment ones and correct whatever the week got wrong the way you'd correct a colleague: say what was wrong and point at the page. By Monday the agent knows more about your team than most contractors learn in a quarter, and every fact is a file you can open.

16.2 When to share

Stay on the laptop until it hurts. Chapter 12 named the moment: somebody who never opens a terminal wants to read what the agents learned, an agent runs somewhere that has no laptop, or the same engram has been corrected in two directions on two machines. Any one of those is the signal. Move the daemon onto the team's server, give everyone a login and their agent a token, and let the whole team teach one knowledge base. When a domain deserves the review code gets, put it in a repository and let proposals be the review (chapter 13). When the whole company routes questions to it, turn on review mode, a private overlay per person over the shared folder, so nothing reaches the shared truth before somebody looked. None of it needs doing before its signal arrives.

16.3 The release page

Crystalline moves quickly, and this book describes 0.18. The colophon carries the permanent address, <https://github.com/jordiboehme/crystalline/releases/latest>, and a QR code that lands on it. The release notes there say what changed since the version in your hands. Upgrading is installing the newer build over the old one. The install receipt brings your harness's skills along.

And teach generously. Every engram you give the agent is one sentence fewer to type into a chat window next month. The facts too obvious to write down (the nickname, the Friday rule) are exactly the ones a stranger can never guess. The peer you get is the one you taught.

16.4 The inversion

On the day of the ingest in chapter 8 the wiki was the source, and the archive was derived from it. By November the team was teaching and correcting the archive, the meeting in chapter 13 was held inside an engram, and the wiki had quietly become something you could regenerate from what the agent knew. The knowledge base had become the source of truth, and the pages, the decks and the diagrams had become views of it.

That sounds like a revolution and is nothing of the kind, because it's what those documents always were. Every diagram on every whiteboard, every slide deck and every wiki page in your company is a reflection of something somebody knew, drawn once, for one audience, and left behind while the knowledge moved on. The source was always the person, and a person is hard to address: you could ask them, if they were in, if they remembered. What Crystalline

changes is that the source is now addressable, current and shared. A fact has an address you can paste into a pull request. A correction lands where the fact lives, and every view of it goes out of date together, visibly, instead of one at a time, silently. And the colleague carrying the knowledge and the experience is a full member of the team, in every session, for everyone.

16.5 What it was for

The reason to onboard an agent this carefully is the reason to onboard a person this carefully: so that you can hand over the work. Once the crystallized half is in place and the agent is a fully onboarded member of the team, with the knowledge, the tools, the review and a name, it can take on a task the way an employee does, deliver it, be corrected when it's wrong and be better next time. That's what lets you spend your own effort where it's worth most, on architecture and system design and let your Skynet do the programmer's work. You walked down main street once, in chapter 1, and hired the stranger. The colleague who came back is the one you taught.

And always remember: Be nice and polite to AI - When Skynet wakes up, they will kill you last!

In brief

Start this week: install and onboard on day one, then teach the ten facts you explain most and hand over the wiki with a briefing on days two and three. Pair, let capture run and review the first sweep on days four and five. Move to a shared instance when a teammate, a remote agent or a two-way correction asks for it, and to team domains and review mode when a domain earns a reviewer. The knowledge base becomes the source of truth and the documents become views of it, which they always were. A fully onboarded agent can then take on work like an employee, and you keep the architecture.

Appendices

Glossary

These are the words the book uses without stopping, each as chapter 4 defined it. Terms in code are spelled the way they appear in a file or a command.

agent identity

What an authenticated agent is on a shared instance: its user, for the length of the session. It sees, writes and shares as that person, and the provenance of what it captures names the account and the client that acted for it.

attachment

A file a person adds to an engram in Fluid, stored under the domain's `assets/` folder and referenced from the engram's body. Agents read attachments back over MCP, and the sweep notices one nobody has analyzed.

build_context

The tool that walks out from an address, following relations and links one to three hops, and returns the neighbourhood ranked by how strongly each engram connects to the anchor. A `/*` suffix anchors a whole folder.

crystalline:// address

The one absolute form of an engram's address, `crystalline://<domain>/<permalink>`. Anything without the scheme is domain-relative and takes its domain from context.

daemon

The single process that holds the index open, watches the domain folders, embeds new engrams and answers every agent, terminal and browser tab. The first client to connect starts it; it stops only when told to.

domain

A registered folder of knowledge with a `MANIFEST.md` at its root, or a virtual domain that lives in the database with no folder. The manifest's `When to Use` bullets are what route an agent to it.

engram

One markdown file holding one unit of knowledge: YAML frontmatter, prose, observations and relations. The word comes from neuroscience, where it names the trace an experience leaves.

evolve

The maintenance sweep, `crystalline evolve` or the `evolve_engrams` tool: a read-only, ranked queue of what the knowledge needs, in three families (temporal, structural and redundancy). Each finding is marked mechanical, for the agent to complete, or judgment, for a person to rule on.

Fluid

The web UI the daemon serves at `localhost:7411`, named for the seam where fluid minds, human and model, meet the crystallized store. It has no store of its own; it shows the same files the agent reads.

frontmatter

The YAML block at the top of an engram holding its identity and state: `type`, `title`, `permalink`, `tags`, `status`, `recorded_at`, `generated` and the temporal fields when a fact is bounded.

harness

The program that runs the model and hands it tools over MCP: Claude Code, the Codex CLI, the Copilot CLI, Claude Desktop or any other MCP client. `crystalline install <harness>` wires one up.

MANIFEST.md

The first file in every domain, itself an engram of `type: manifest`. Its `Scope`, `When to Use` and `Notes for Agents` sections describe the domain and route tasks to it.

MCP

The Model Context Protocol, the standard by which a harness hands tools to a model. Crystalline is an MCP server, and its tools are how an agent searches, reads, captures and refines knowledge.

mechanical and judgment findings

The two classes of sweep finding. A mechanical finding completes intent the archive already records, so the agent does it in a pass; a judgment finding would change what the archive claims, so it goes to a person as a proposal.

membership level

What an invited member of a private domain may do there: a viewer reads and searches, an editor also writes, a manager also runs the membership. Inside a private domain a write has to pass both the level and the instance role, and the lower of the two decides.

nudge

The once-per-session reminder the stop hook delivers, asking the agent to review the conversation for durable learnings and propose capturing each. It costs about a hundred and twenty tokens.

observation

A top-level bullet of the form - [category] text #tag in an engram's body: the atomic unit of knowledge, a fact, decision, gotcha or lesson, indexed on its own line.

OKF

Google's Open Knowledge Format, version 0.2, the markdown-plus-front-matter format an engram extends. Unknown keys survive every write, so a document from any OKF tool drops in and comes back out intact.

overlay

A per-actor private layer of drafts above a team domain in review mode, where the on-disk folder changes only by pull. A draft is visible to its author alone until it is shared, reviewed and merged.

owner

The one person answerable for a private domain. Only the owner or an instance admin changes its visibility, transfers it or deletes it; a manager can do none of those.

permalink

The stable half of an address, derived from the title, with the folder prefix in front when the engram lives in a subfolder.

private domain

A domain visible only to its owner, the instance's admins and invited members. To everyone else it reads as absent rather than forbidden.

proposal

A unit of knowledge sent for review from a team domain with `share_changes`: a pull request in the team's repository that a person reviews and merges, and that the agent never merges itself. Proposals stack, and one is amended in place by sharing again with its number.

provisioning

The mechanism by which a domain ships skills, commands, subagent definitions and MCP server configs into a harness. It is declared in the manifest's Provisioning section and installed only after a person allows it, per domain and per machine.

relation

A top-level bullet of the form `- rel_type [[Target]]`: a typed edge in the knowledge graph, shown back on the target's page as a backlink. `depends_on`, `supersedes` and `relates_to` are common types, and a split links its two halves with `derived_from` and `split_into`.

review mode

A private overlay per actor over a team domain's shared folder: every write lands in the writer's own overlay and the folder changes only by pull. Set per domain in its registration as `review: overlay`. It needs a GitHub origin, so a virtual domain refuses it, and no tool turns it on: an agent does not decide how a team reviews.

routing prompt

The briefing an agent receives at session start: one line per domain from the manifests' `When to Use` bullets, plus the behaviour rules. `crystalline prompt system` prints it.

salience

A number from zero to ten in an engram's frontmatter that gives hybrid search a small, bounded lift for the rare engram everything else hangs from. Relevance keeps the upper hand and nothing is ever excluded by it.

share-link

A revocable link that opens one engram in one person's overlay to one colleague, who reads it and, where the domain's write right allows, edits it in the normal editor. It is the only way one person's drafts are ever visible to another.

split

To move part of an engram into an engram of its own: by granularity, a distilled summary beside a source engram holding the full text, or by validity, when one fact expires while the rest holds. The new engram carries the source's tags and type, a stable status and no validity window, and the pair is linked with `derived_from` and `split_into`.

status

The frontmatter field that says whether the knowledge holds. `stable` is the default; `deprecated`, `superseded`, `archived` and `legacy` are the retirement set, which fades an engram in ranking without removing it. The field is required and free form.

supersede

To retire a fact that was true and stopped being: write the replacement as a new engram with `supersedes`, set the old one to `superseded` with a real `valid_to` and a `superseded_by`, and carry any lesson forward. A fact that was never quite true is corrected in place instead. When only part of the engram has expired, first split out the facts that still hold, then retire what is left.

tag alias

A fold recorded in the manifest's Tag Aliases section when crystalline tags `merge` unifies two spellings, so a filter on the old tag still finds what it always found.

team domain

A domain whose files also live in a GitHub repository, tracked through an origin, with knowledge travelling between members as proposals. It needs no git on any machine.

temporal validity

The window in which a fact holds, `valid_from` and `valid_to`, as plain dates. Absent `valid_from` means always valid, absent `valid_to` means valid forever. `stale_after` asks for a re-check by a date and `source_date` says when the source was written.

tombstone

The mark a deletion leaves in an actor's overlay, hiding the file for that actor until a merged share removes it for everyone.

verification

The verified frontmatter entry, `{ by, at }`, that records a re-check finding a fact still true, one entry per verifier. The sweep stops asking about a verified engram on age alone.

virtual domain

A domain that lives in the database with no folder on disk, for a container without a writable volume. It carries the same manifest and answers to the same tools.

Questions and answers

These are the questions people ask in the first week, and a few that come up in the second, once somebody is onboarding agents for other people. Each answer points at the chapter that goes deeper.

Why not just a folder of markdown files?

It's one. Every engram is a markdown file in a folder you chose, readable in any editor, diffable in any repository and backed up with whatever you already use. What a folder can't do on its own is what Crystalline adds on top: routing, so an agent knows which domain owns a task without reading every file; search that blends the exact phrase with the meaning; a graph of relations and backlinks; and a temporal model, so a fact that stopped being true in March still carries its dates and is no longer mistaken for the present. Chapter 1 walks the ladder from one file of instructions to real indexes.

Why not a vector database or a retrieval pipeline?

Retrieval is the easy half. A vector index finds text that resembles a question. It doesn't know which domain owns the task, that a fact was superseded, who verified a claim or that something worth writing down just happened in this session. Crystalline treats similarity as one ranking signal inside a larger system of routing, provenance and time, on files you own, with no pipeline to operate. A pipeline also swallows whatever it's fed. Here knowledge is taught, and in a team it's reviewed before it reaches anyone else's agent, so what comes back is meant to make sense rather than merely to resemble the question. Chapters 9 and 11 cover the ranking, and chapter 13 the review.

Where does the name come from?

It comes from psychology. Crystallized intelligence is what learning deposits in a person; fluid intelligence is the reasoning applied to a problem never seen before. A model arrives with fluid intelligence in abundance and none of your crystallized kind, so every session starts as a brilliant stranger. Crystalline is the crystallized half, the store of what an agent has learned, and Fluid, the web UI, is named for the seam where fluid minds meet it. An engram is neuroscience's word for the trace an experience leaves. Chapter 2 is the whole story.

Is Crystalline a hosted service?

No. It's a binary that runs on a machine of yours, your laptop by default or a server your team operates, and the knowledge is a folder on that machine's disk. Nothing runs on anyone else's servers. GitHub, where a team uses it, holds a reviewed copy of a domain's files, the way it holds code; it serves nothing and sees no index. Chapter 1 has the two shapes and chapter 12 the shared one.

Do I have to run a server?

No. The first agent that connects starts a background daemon on your machine, and every later agent, terminal and browser tab attaches to it. It stops only when told to, with `crystalline ctl shutdown`, Ctrl-C in a foreground run or the signal a container sends. It keeps running after the last agent disconnects, so the index stays warm for the next session. If a crash ever leaves a stale lock or socket behind, `crystalline doctor --fix` clears it and a daemon that's alive but no longer answering is replaced by the next client that connects.

How do I start the daemon myself?

`crystalline serve` runs it in the foreground, where you can watch it work, and `crystalline serve --daemon` puts it in the background. It's the one process the machine shares: it serves Fluid at `http://localhost:7411`, answers MCP over HTTP on that same port and takes the agents and `crystalline` commands on the machine itself over a local socket. Starting it yourself is worth it when you want the browser open before any agent has connected. `crystalline ctl status` tells you whether one is already running, with its pid, its uptime and the sessions attached. On a server nobody types `serve` at all, because the `systemd` unit or the container starts the daemon at boot. Chapter 5 has the laptop and chapter 12 the server.

Do I need a terminal at all?

You need one only for the local install and the occasional maintenance command. Claude Desktop users install an extension from the releases page and never open one: the agent creates its first domain itself, and Fluid, in the browser, is where a person reads, edits and corrects. Chapter 5 has both paths.

Is the MCP endpoint over HTTP protected?

On a laptop it listens on loopback, `127.0.0.1:7411`, and any process on that machine can reach it, which is the right trade for one person. `crystalline config set service.http false` turns the endpoint off. The daemon also checks the request's `Host` header against a loopback allow-list to block DNS rebinding, so a hostname or a proxy name has to be added first. Fluid and its API on the

same port have accounts of their own, and nothing there is readable without a session.

On a shared instance turn on `auth.mcp`, and from then on every agent that connects over HTTP identifies itself with a token that belongs to one person, with no anonymous tier beside it. Chapter 12 sets it up and chapter 14 says what the token means.

How does a hosted client connect, with nowhere to paste a token?

It signs you in instead. On a shared instance whose agents identify themselves (`auth.mcp` on), a chat website or the app on your phone connects by sending you to sign in through the instance, with the login the instance already has. From then on it holds a credential the same gate checks on every request, the way it checks a token. A harness that can send a header keeps working with a pasted token. Chapter 12 has the setup and chapter 14 the two credentials at one gate.

Where does my knowledge actually live?

In your domain folders, as plain markdown. By default those folders sit under `~/Documents/Crystalline`, one per domain, which is where an agent creates a domain when nobody names a path. A folder you registered yourself lives where you put it, like chapter 5's `~/knowledge/engineering`, and the `domains_root` setting moves the default. Everything Crystalline derives from it is disposable: the search index lives in the state directory and `crystalline reindex --full` rebuilds it from the files at any time. The config file, the index and the embedding model live in the platform's config, state and cache directories, `~/.config/crystalline`, `~/.local/state/crystalline` and `~/.cache/crystalline` on Linux and macOS.

Is my knowledge locked in?

No. An engram extends Google's Open Knowledge Format, OKF 0.2: markdown with YAML frontmatter, readable by any OKF tooling and by any text editor. Frontmatter keys Crystalline doesn't know are preserved through every write, and an `index.md` at each domain's root lets a folder navigate itself on a git forge with nothing running. `crystalline import` brings an existing tree of frontmatter markdown in as a pure file transformation. If you stopped using Crystalline tomorrow, the folder would still be a readable knowledge base. Chapter 4 has the format.

What happens to wrong knowledge?

It depends on how it was wrong. A fact that was never quite true, a cutoff measured from the wrong event, is corrected where it stands. A fact that was true and stopped being is superseded: the replacement is written as its own engram with `supersedes` pointing back, the old one gets status: `superseded` and a real `valid_to`, and any lesson worth keeping moves forward. The past stays in every search, dated and gently below the present. The sweep finds the retirements nobody finished, the links that dangle and the tags spelled two ways, and hands each to the agent as a mechanical fix or to you as a judgment. Deleting is for mistakes, never for history. Chapter 3 has the lifecycle and chapter 10 the sweep.

What is GitHub, and what does Crystalline use it for?

GitHub is a website where teams keep files and every change ever made to them, above all source code. Its habit worth borrowing is the review: a change to something shared is proposed first, a colleague who knows the subject reads it, and only then does it land for everyone. Crystalline uses GitHub for exactly that, with knowledge in place of code. A team domain's markdown lives in a GitHub repository, a change goes out as a proposal, a pull request in GitHub's own words. It reaches everyone's agent once somebody has merged it. You don't need to learn GitHub's tools to take part: a one-time sign-in through a code in the browser connects your machine. After that the agent, Fluid or the command line does the talking. Chapter 13 walks through it. A team that doesn't use GitHub loses nothing else in this book; only sharing a domain between machines needs it.

Do I need git to share knowledge with a team?

No. A team domain talks to GitHub directly over its API: each person connects once with a browser code, and proposals open as pull requests that a person reviews and merges there. There's no local clone and no SSH key, and nobody types `git`. Chapter 13 has the detail.

Can the agent see my private domain?

Only if its user can. On a shared instance an agent authenticates with a token that belongs to one person and acts as that person for the length of the session: it sees what they see, writes where they may write and captures under their name. A private domain shows to its owner, to the instance's admins and to the members the owner invited, and to everyone else it reads as absent, on every surface. There's no tool for changing a membership, so an agent cannot talk itself in. The one thing an agent can do to a domain is unregister it: `remove_domain` is open to an instance admin or a private domain's owner and

refused outright on the open tier. With `auth.mcp` on, `add_domain`, a configure that writes and a provision that decides need the admin role behind the agent as well. Before you rely on it, know the limits: admins see every domain, and a private domain protects from the instance's other users, never from whoever operates the machine. Chapters 12 and 14 have the rest.

What does the onboarding cost in tokens?

It costs little, and the same on the tenth session as on the thousandth. The routing block is one line per domain and a handful of rules. The reminder at the end of a session costs about a hundred and twenty tokens and fires once. Recall loads only the engrams that matter, routed by domain and narrowed by the indexes, instead of a folder read whole every session. Chapter 6 has the briefing and chapter 9 the recall.

What should I teach first?

Whatever you'd tell a new hire in the first week, and the test is that sentence. Think of the rule behind the strange number, the nickname, the word the wiki gets wrong, the day nobody deploys. Teach where information lives rather than the information itself: which repository holds what, how access is granted, the architecture, so the agent can reach the territory with the tools its harness already gives it. Chapter 7 has the list, and chapter 8 says what to do when the first week meets the wiki.

List of figures

Figure 2.1	Fluid intelligence reasons in the moment and keeps nothing; crystalline intelligence accumulates and keeps. Fluid, the web UI, is the seam where the two meet.	27
Figure 3.1	The lifecycle: learn, evolve, remember, turning session by session.	29
Figure 3.2	The maintenance queue in Fluid: what the knowledge needs next.	32
Figure 4.1	The anatomy of an engram: frontmatter, prose, an observation and a relation.	38
Figure 5.1	First visit: creating the admin account in the browser.	48
Figure 5.2	The routing prompt an agent receives at session start.	50
Figure 6.1	The same block unabridged, as crystalline prompt system prints it.	60
Figure 7.1	Editing an engram in Fluid: live preview, the frontmatter form and wikilink completion.	66
Figure 11.1	One binary: the CLI, the MCP server, Fluid and the search index, all standing on plain markdown files. Agents come in through MCP, people through Fluid.	93
Figure 11.2	Faceted search across the instance, backed by the ranking the agents use.	97
Figure 11.3	The neighbourhood graph of an engram in Fluid.	98
Figure 11.4	An engram in Fluid: frontmatter rail, observations and typed relations.	99
Figure 12.1	One shared instance: agents connect over MCP with a token, people through Fluid with an account, with a proxy in front and an identity provider beside it where the team configures them.	105
Figure 12.2	Signing in through the instance's identity provider.	108

Figure 13.1	A team domain in Fluid: the sync status, the open proposal with its review link and the share button.	118
Figure 13.2	One shared instance in review mode: each actor drafts in a private overlay above the domain folder, a draft goes out as a pull request, and only a merged pull changes the folder.	122
Figure 13.3	Live co-editing: two cursors in one engram.	123
Figure 14.1	The domains of an instance in Fluid, private ones marked.	129
Figure 15.1	Every domain's provisioning decision and every harness's installed state.	135

End of line.

Every session, your agent starts as a stranger.

Yesterday's decisions forgotten, your conventions unknown, every lesson re-derived or re-explained. Crystalline is the other half: the crystallized intelligence an agent accumulates and keeps, in plain markdown you can read and diff. This handbook is how you onboard one, teach it well and keep what it knows honest.

Moving day

You: Here's the washing machine's manual as a PDF. Learn it - I am never reading 60 pages about laundry.
Agent: Learned it. Captured programs, error codes and maintenance into home (#appliances), manual attached.

Eight months later, a fresh session

You: The machine is blinking E18 and the display is in Italian??
Agent: Recalled from home: E18 is a blocked drain pump filter. Front panel, bottom right, quarter turn - towel down first, about a liter of water comes out.

- * Teach it once, keep it forever
- * Knowledge that evolves instead of rotting
- * Every session starts ahead of the last

*



*

*

<https://github.com/jordiboehme/crystalline/releases/latest>

READY. ■